

同济大学计算机系

计算机组成原理课程综合实验报告



学 号 2452769

姓 名 幸可函

专 业 计算机科学与技术

授课老师 张冬冬

一、实验内容

使用 Verilog 实现 54 条 MIPS 指令的单周期 CPU 的设计、前仿真、后仿真和下板调试运行。

二、CPU 数据通路设计

(1) 根据每条指令所涉及部件和部件的数据输入来源，画出每条指令的数据通路。

(2) 根据每条指令功能，在已形成的数据通路下，画出每条指令从取指到执行过程的指令流程图。

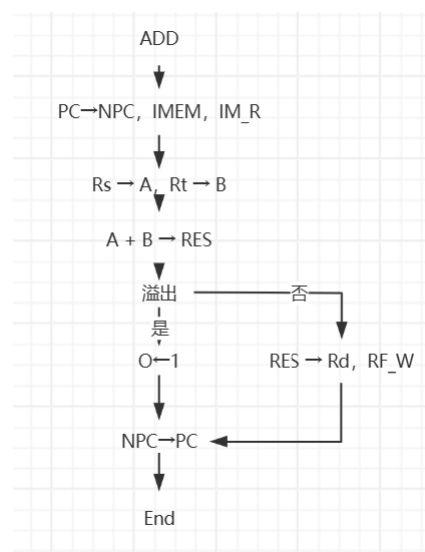
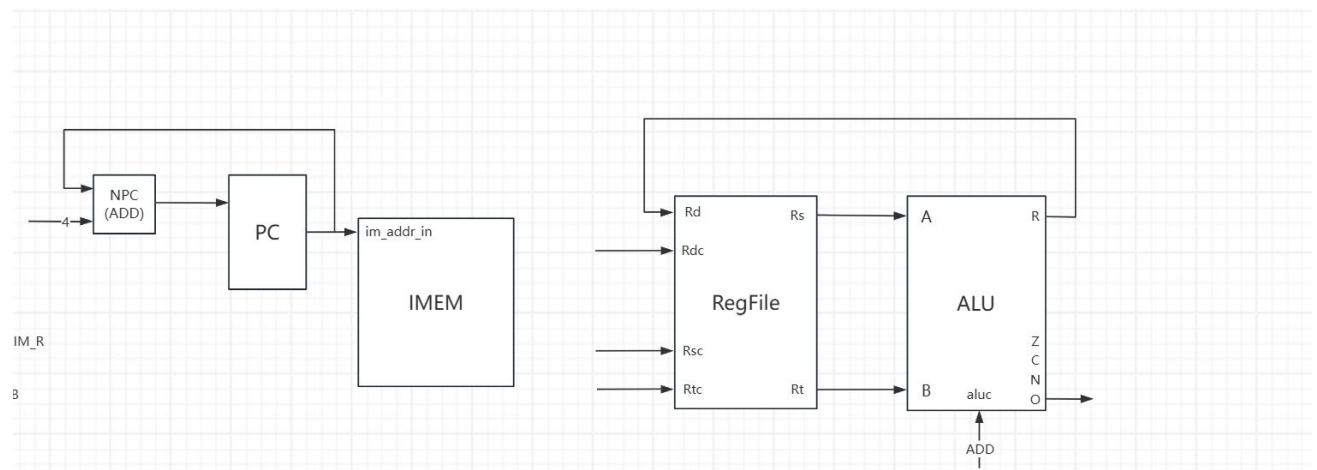
1. ADD

格式: ADD rd, rs, rt

描述: $rd \leftarrow rs + rt$

目的: 32 位数相加 将通用寄存器中存的 32 位数据 rs 与 rt 相加产生一个 32 位数据存入目标寄存器 rd。

- 如果发生了溢出，则 rd 不改变并且产生一个溢出的异常。
- 如果相加不溢出，则产生的 32 位数据直接存入目标寄存器 rd



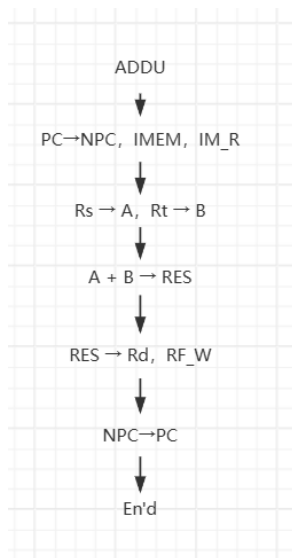
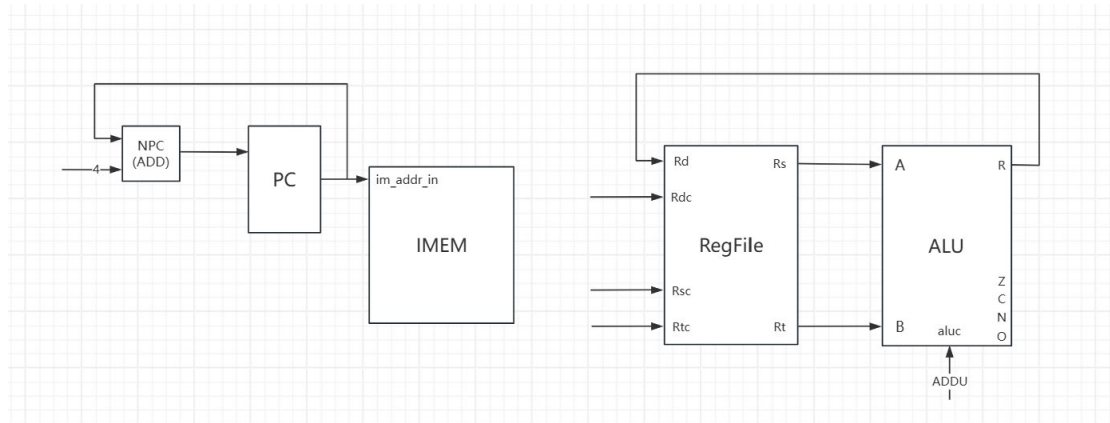
2. ADDU

格式: ADDU rd, rs, rt

目的: 32 位数据相加

描述: $rd \leftarrow rs + rt$ 将通用寄存器中存的 32 位数据 rs 与 rt 相加产生一个 32 位数据,存入目标寄存器 rd。

在任何情况下都不会有溢出的异常



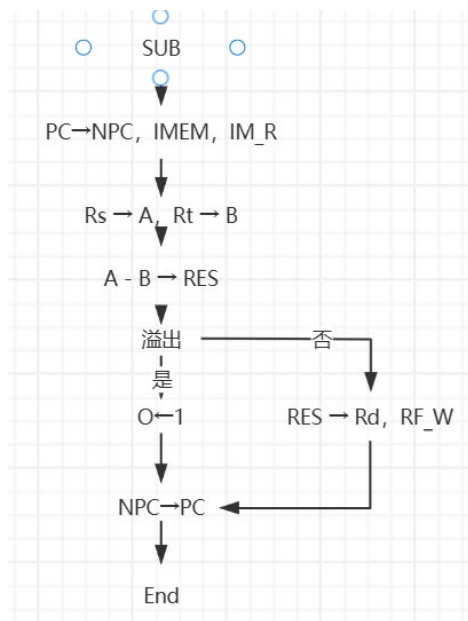
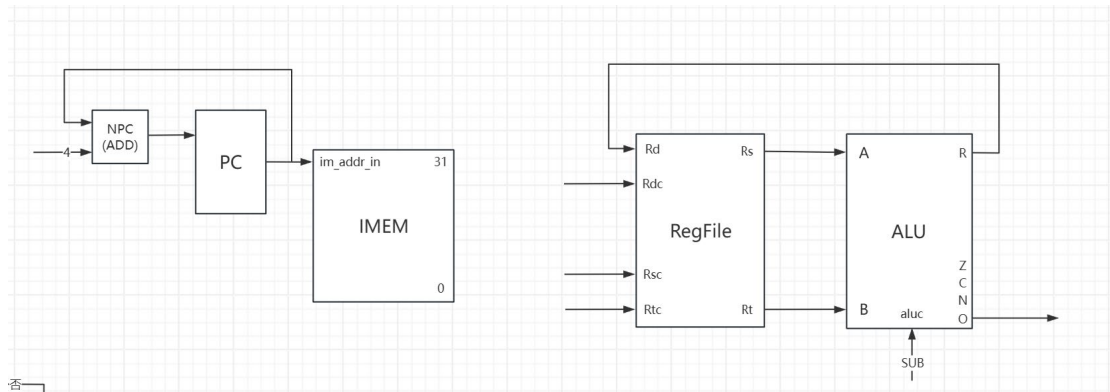
3.SUB

格式: SUB rd, rs, rt

目的: 与 32 位数相减

描述: $rd \leftarrow rs - rt$ 将通用寄存器中存的 32 位数据 rs 与 rt 相减产生一个 32 位数据存入目标寄存器 rd。

- 如果发生了溢出，则 rd 不改变并且产生一个溢出的异常。
- 如果不溢出，则产生的 32 位数据直接存入目标寄存器 rd。



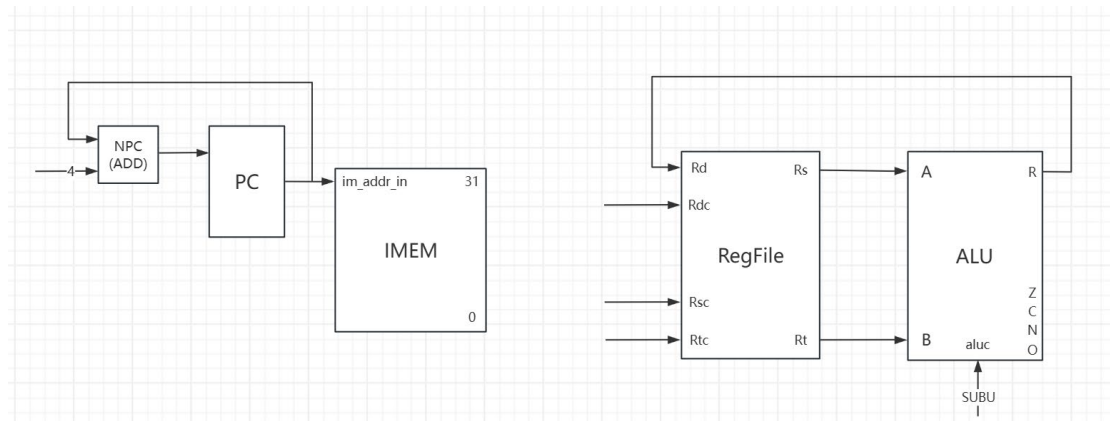
4. SUBU

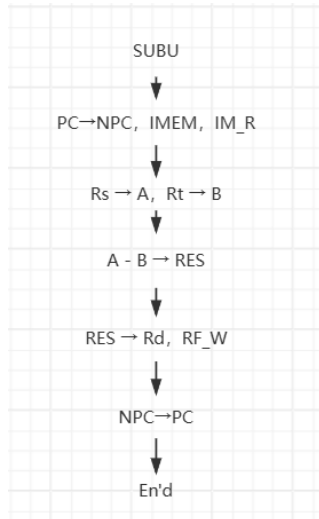
格式: SUBU rd, rs, rt

目的: 32 位数据相减

描述: $rd \leftarrow rs - rt$ 将通用寄存器中存的 32 位数据 rs 与 rt 相减产生一个 32 位数据存入目标寄存器 rd。

在任何情况下都不会有溢出的异常



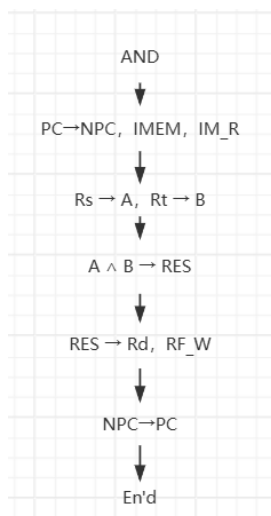
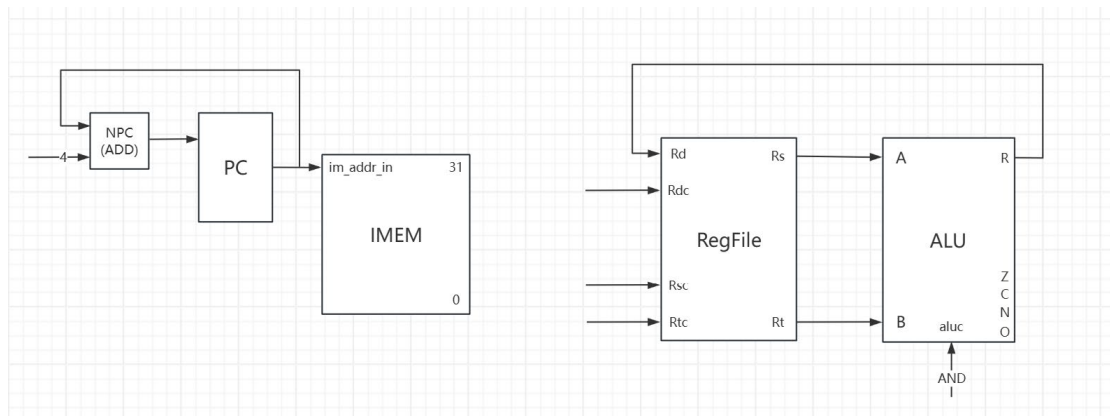


5. AND

格式: AND rd, rs, rt

目的: 按位逻辑与

描述: $rd \leftarrow rs \text{ AND } rt$ 将通用寄存器 rs 和 rd 中的数据每一位做按位与操作，将结果存入目标寄存器 rd 中。

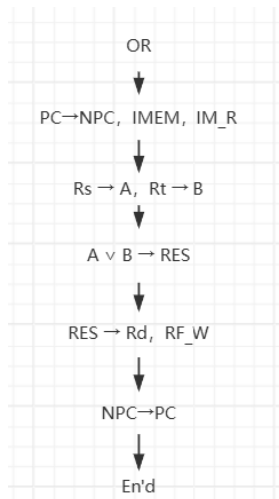
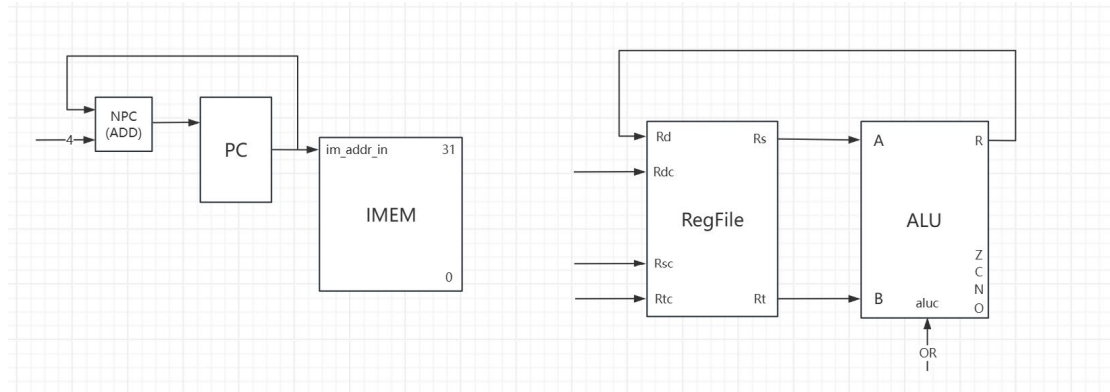


6. OR

格式: OR rd, rs, rt

目的: 按位逻辑或

描述: $rd \leftarrow rs \text{ or } rt$ 将通用寄存器 rs 和 rt 中的数据每一位做按位或操作，将结果存入目标寄存器 rd 中。



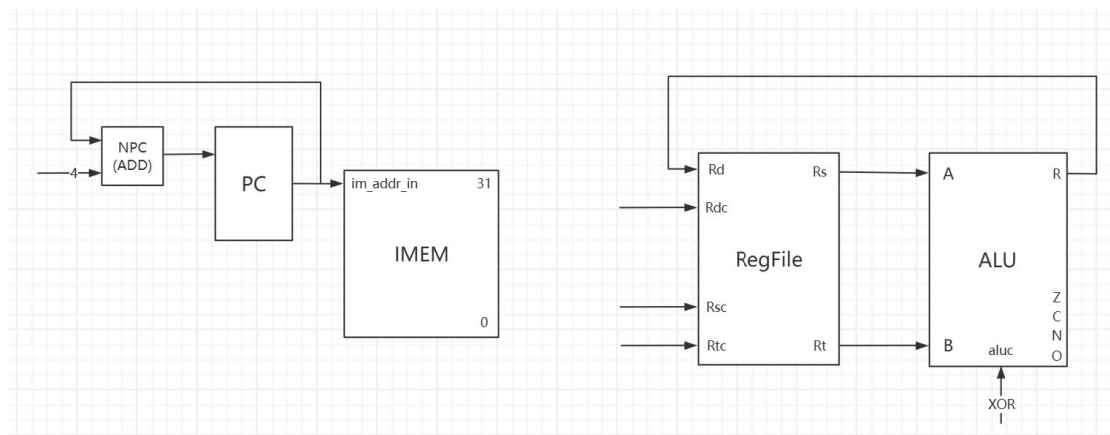
7. XOR

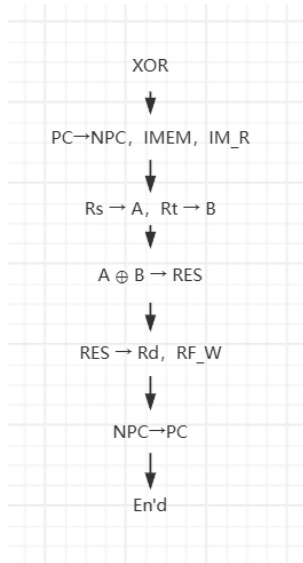
格式: XOR rd, rs, rt

目的: 按位逻辑异或

描述: $rd \leftarrow rs \text{ XOR } rt$

将通用寄存器 rs 和 rt 中的内容按位进行异或操作，将结果存入 rd 中。





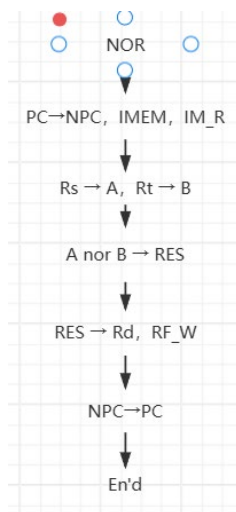
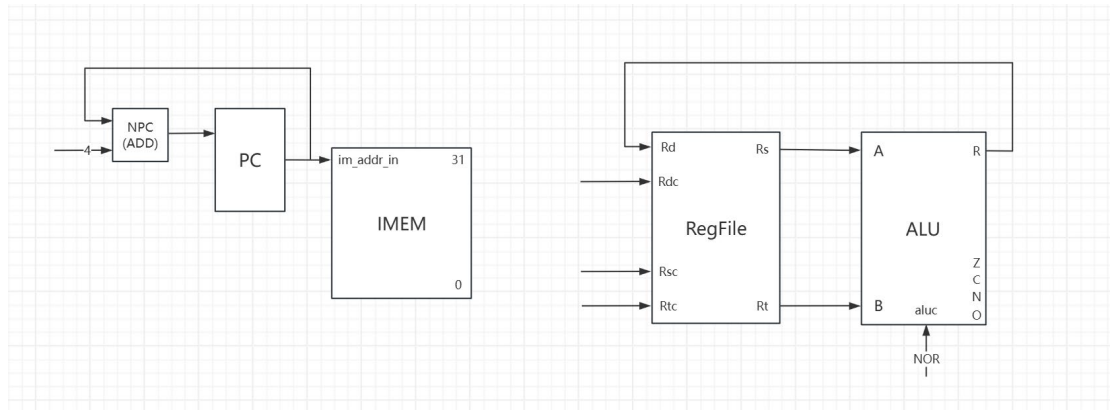
8. NOR

格式: NOR rd, rs, rt

目的: 按位逻辑或非

描述: $rd \leftarrow rs \text{ NOR } rt$

将通用寄存器 rs 和 rt 中的数据每一位做按位或非操作，将结果存入目标寄存器 rd 中。

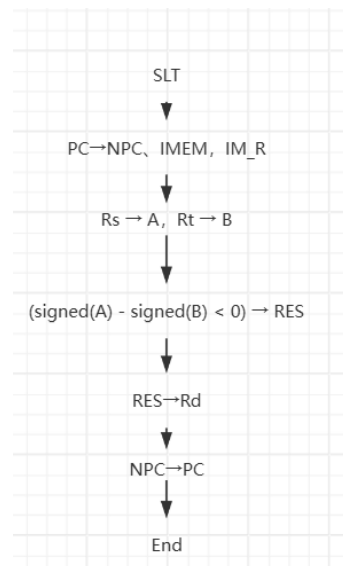
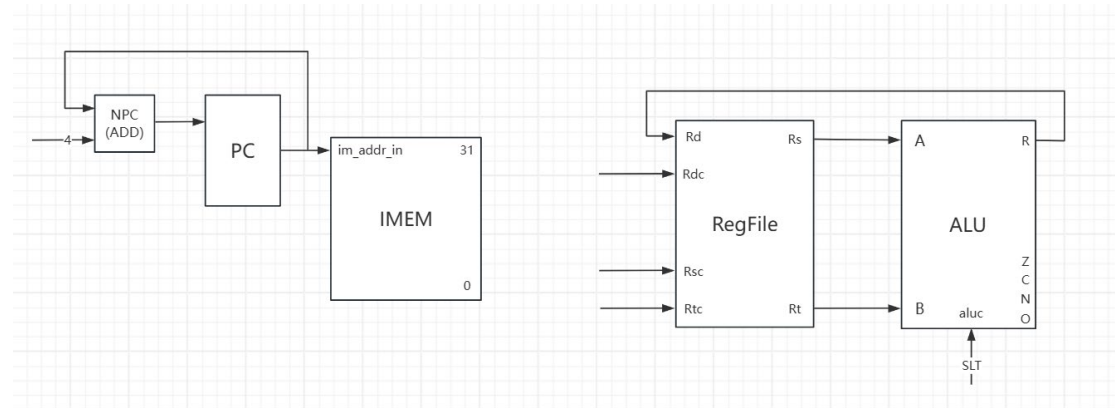


9.SLT

格式: SLT rd, rs, rt

目的: 通过小于的比较来记录结果

描述: $rd \leftarrow (rs < rt)$ 比较在 rs 和 rt 寄存器中保存的有符号数, 用 boolean 值保存结果到 rd 寄存器中。如果 rs 小于 rt, 则结果为 1, 反之结果为 0。算数比较不会引起溢出异常。

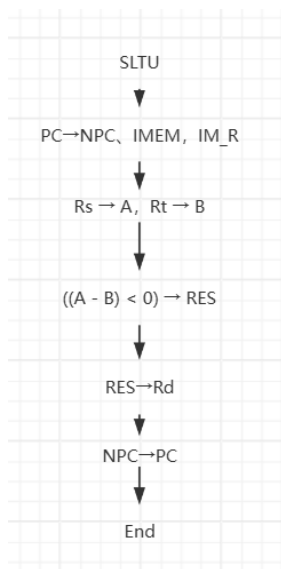
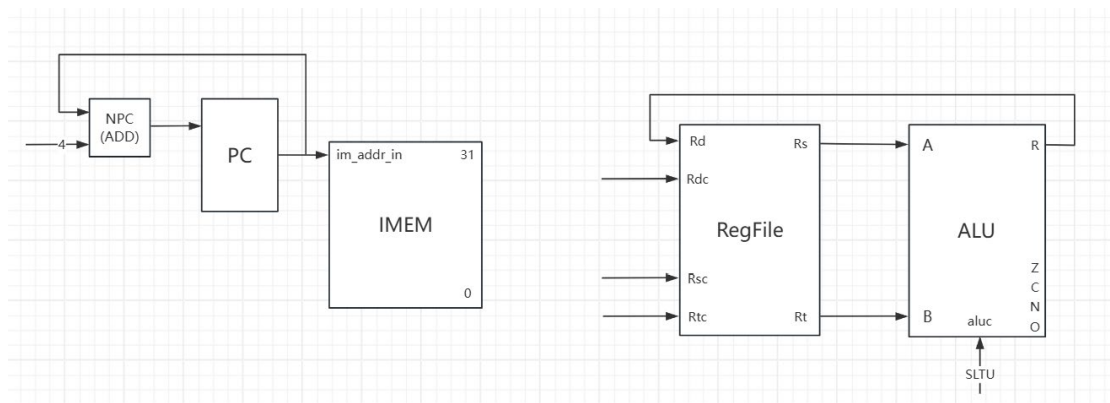


10. SLTU

格式: SLTU rd, rs, rt

目的: 通过无符号小于的比较来记录结果

描述: $rd \leftarrow (rs < rt)$ 比较在 rs 和 rt 寄存器中保存的无符号数, 用 boolean 值保存结果到 rd 寄存器中。如果 rs 小于 rt, 则结果为 1, 反之结果为 0。算数比较不会引起溢出异常。



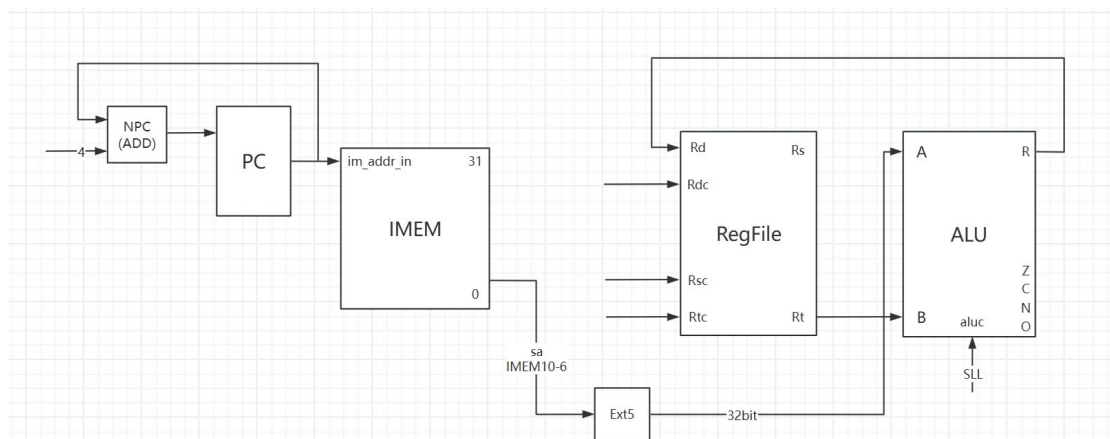
11. SLL

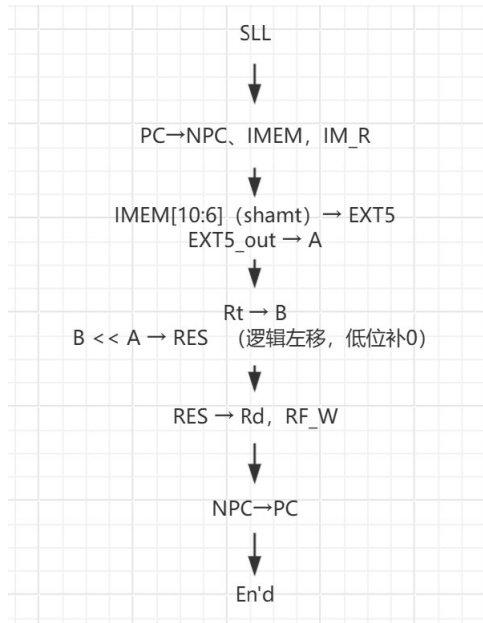
格式: SLL rd, rt, sa

目的: 通过数字填充逻辑左移

描述: $rd \leftarrow rt \ll sa$

将通用寄存器 rt 的内容左移 sa 位，空余出来的位置用 0 来填充，把结果存入 rd 寄存器。





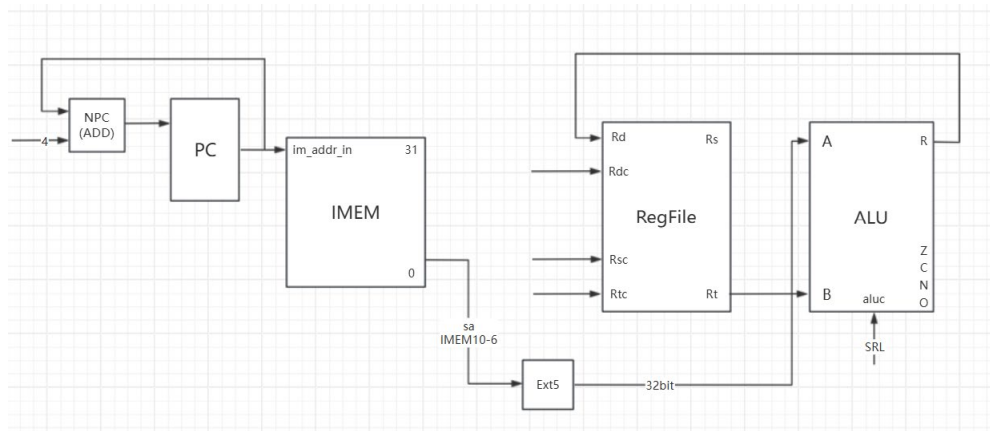
12. SRL

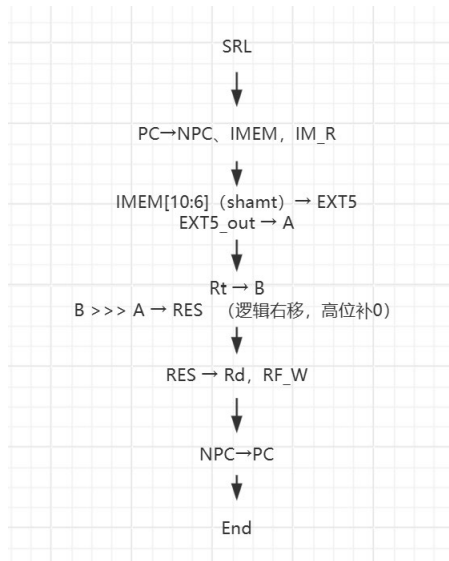
格式: SRL rd, rt, sa

目的: 通过数字填充逻辑右移

描述: $rd \leftarrow rt \gg sa \text{ (logical)}$

将通用寄存器 rt 中的 32 位内容右移 sa 位，高位用 0 来填充，结果存入通用寄存器 rd。





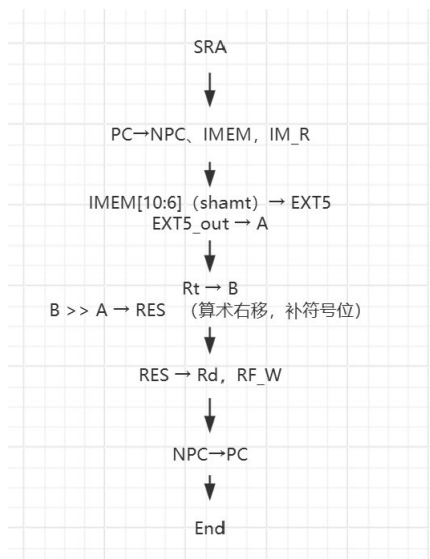
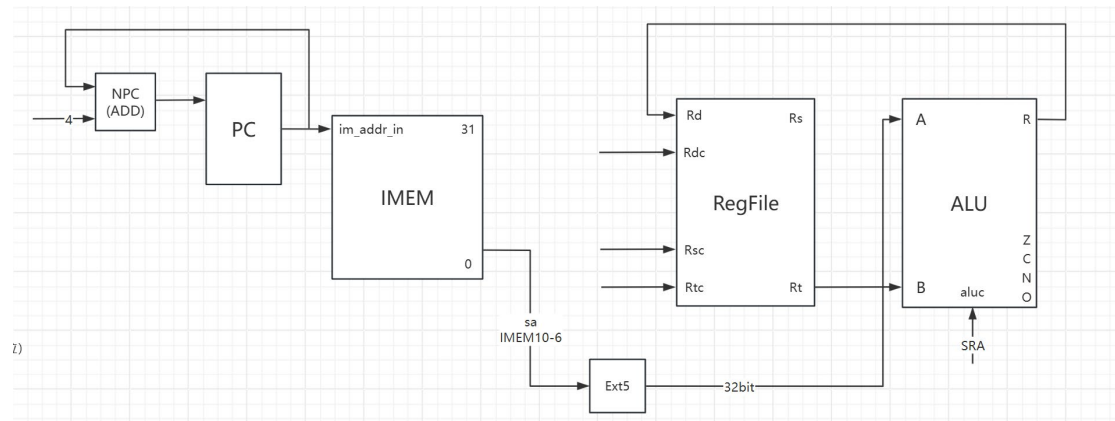
13. SRA

格式: SRA rd, rt, sa

目的: 通过数字填充算术右移

描述: $rd \leftarrow rt \gg sa \text{ (arithmetic)}$

将通用寄存器 rt 中的 32 位内容右移 sa 位，高位用 rt[31]来填充，结果存入通用寄存器 rd。



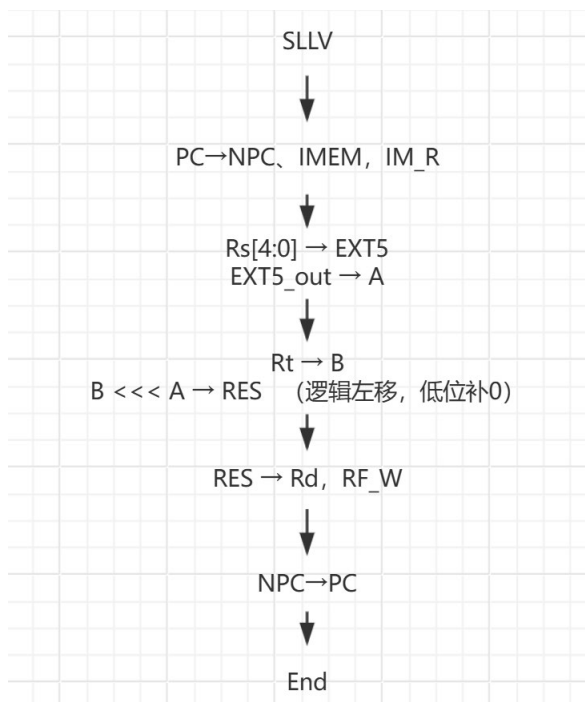
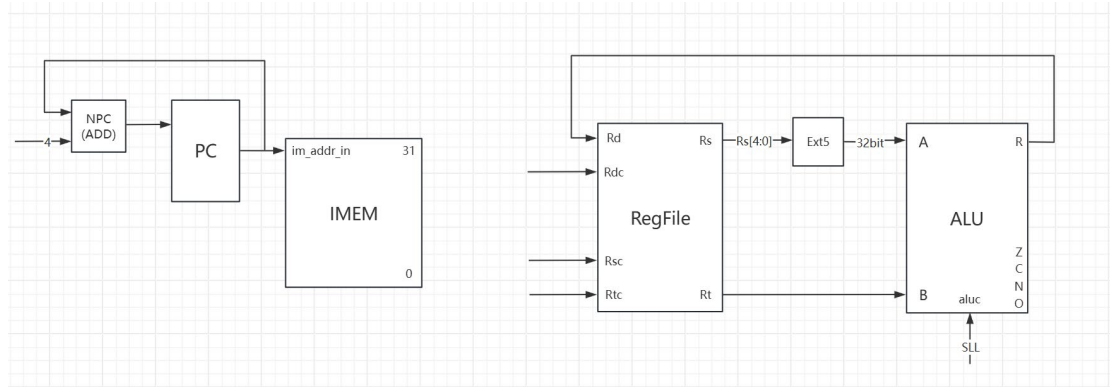
14. SLLV

格式: SLLV rd, rt, rs

目的: 通过数字填充逻辑左移

描述: $rd \leftarrow rt \ll rs$

将通用寄存器 rt 的内容逻辑左移，左移的位数保存在 rs 寄存器中，空余出来的位置用 0 来填充，把结果存入 rd 寄存器。



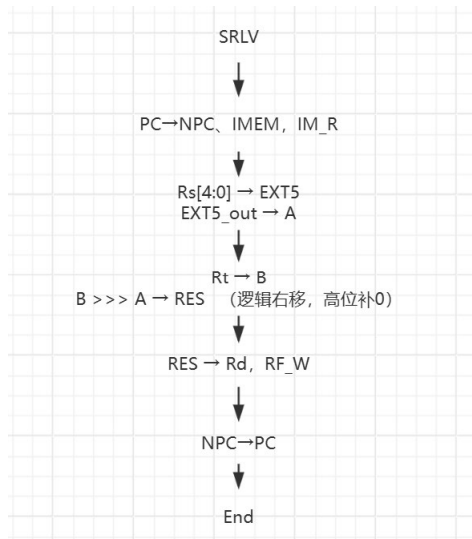
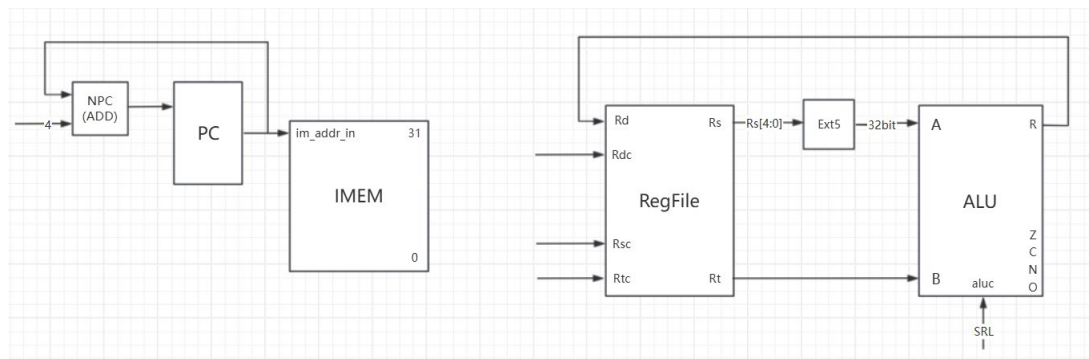
15. SRLV

格式: SRLV rd, rt, rs

目的: 通过数字填充逻辑右移

描述: $rd \leftarrow rt \gg rs \text{ (logical)}$

将通用寄存器 rt 中的 32 位内容右移，高位用 0 来填充，结果存入通用寄存器 rd。右移的位数由通用寄存器 rs 中的 0-4bit 确定。



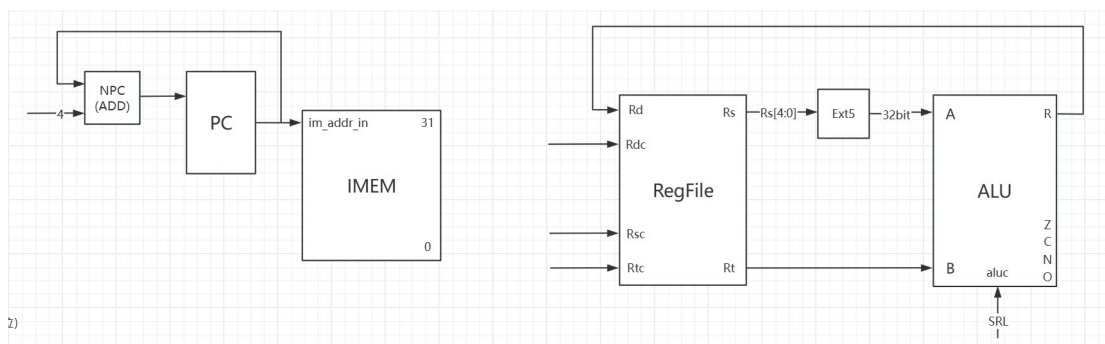
16. SRAV

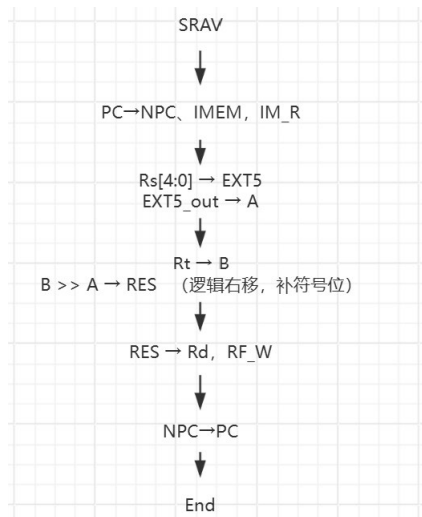
格式: SRAV rd, rt, rs

目的: 通过数字填充算术右移

描述: $rd \leftarrow rt \gg rs \text{ (arithmetic)}$

将通用寄存器 rt 中的 32 位内容右移, 高位用 rt[31]来填充, 结果存入通用寄存器 rd。右移的位数由通用寄存器 rs 中的 0-4bit 确定。





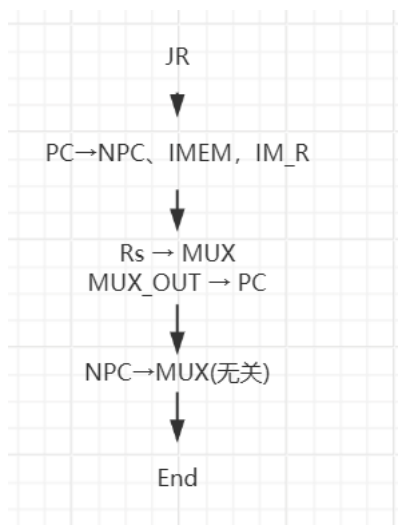
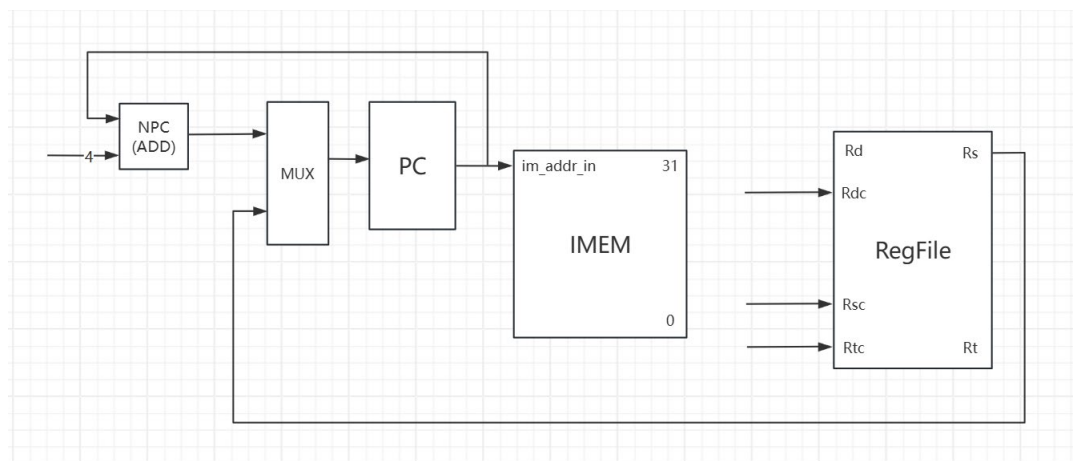
17. JR

格式: JR rs

目的: 使用寄存器的跳转指令

描述: $PC \leftarrow rs$

跳转地址存放在通用寄存器 rs 中，直接跳转到寄存器所存地址。



18. ADDI

格式: ADDI rt, rs, immediate

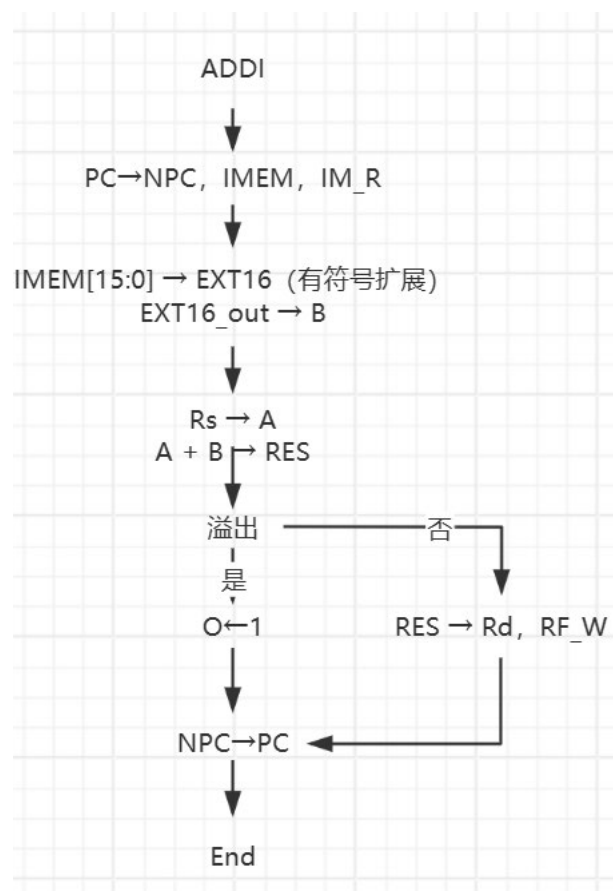
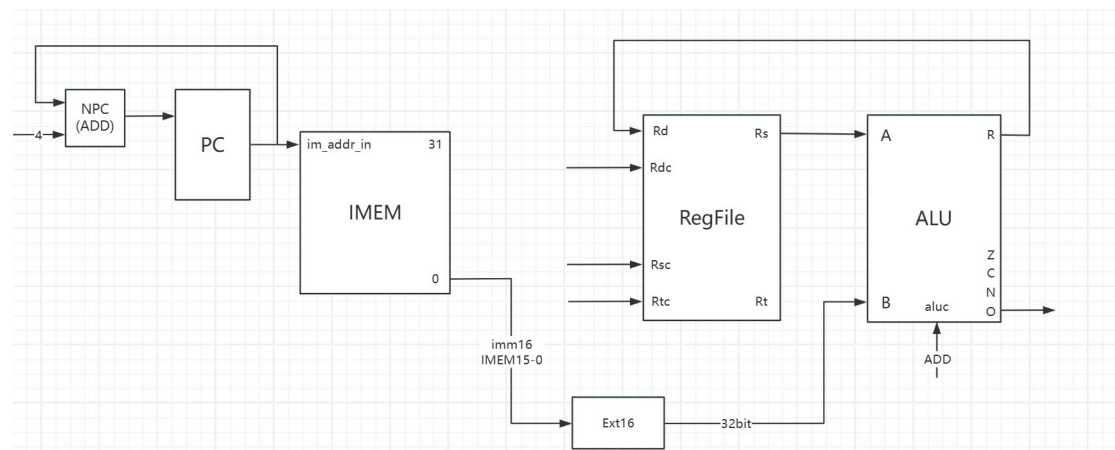
目的: 使 32 位数据与一个立即数相加

描述: $rt \leftarrow rs + \text{immediate}$

16 位有符号立即数与通用寄存器 rs 中的 32 位数相加产生一个 32 位的数存入目标寄存器 rt。

如果发生了溢出，则 rt 不改变并且产生一个溢出的异常。

如果相加不溢出，则结果存入目标寄存器 rt。



19. ADDIU

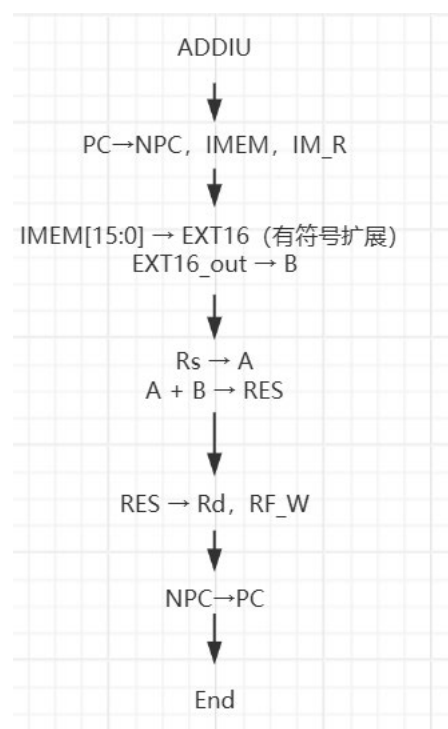
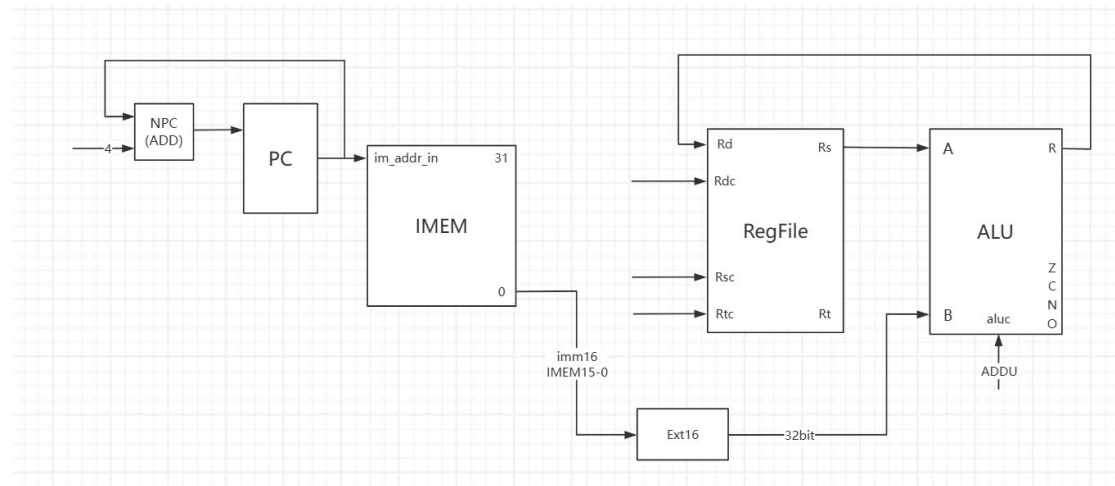
格式: ADDIU rt, rs, immediate

目的: 使 32 位数据与一个立即数相加

描述: $rt \leftarrow rs + \text{immediate}$

一个 16 位有符号的立即数与通用寄存器 rs 中的 32 位数相加产生一个 32 位的数存入目标寄存器 rt 。

在任何情况下都不会有溢出的异常。



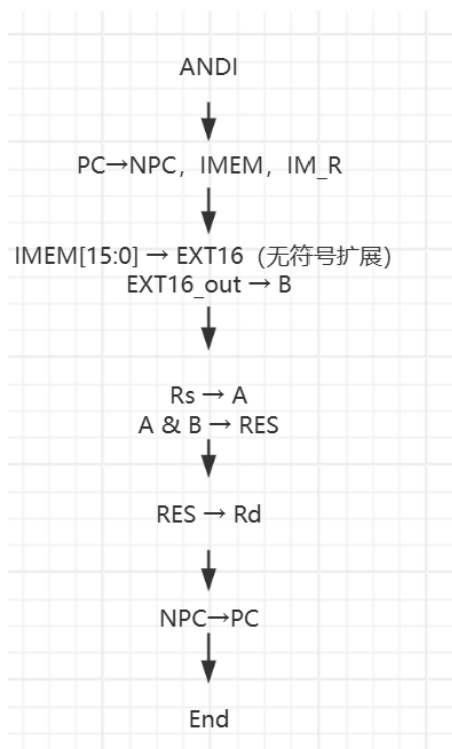
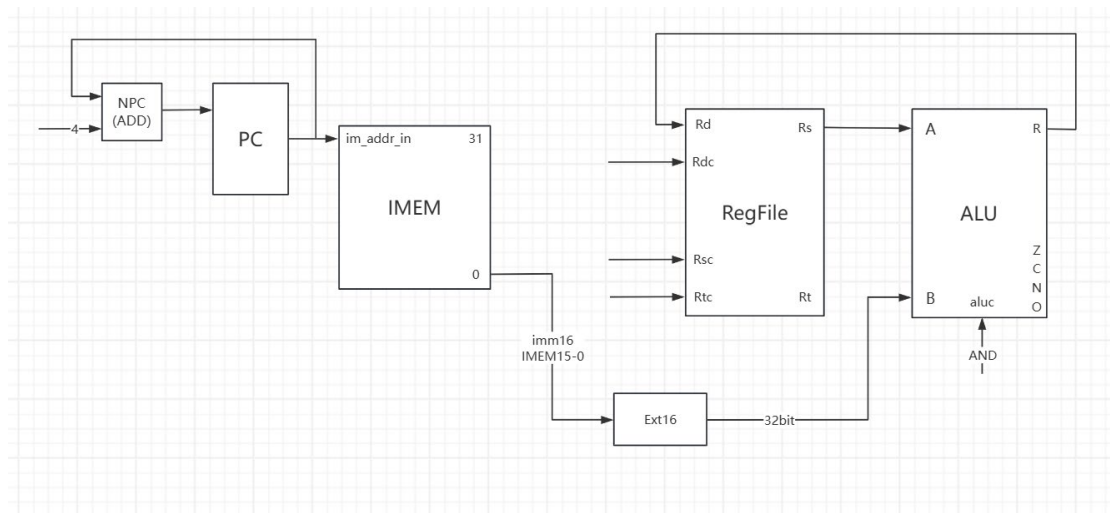
20. ANDI

格式: `ANDI rt, rs, immediate`

目的: 与一个常数做按位逻辑与

描述: $rt \leftarrow rs \text{ AND } \text{immediate}$

将 16 位立即数做 0 扩展后与通用寄存器 rs 中的 32 位数据做按位与，将结果存入目标寄存器 rt 。



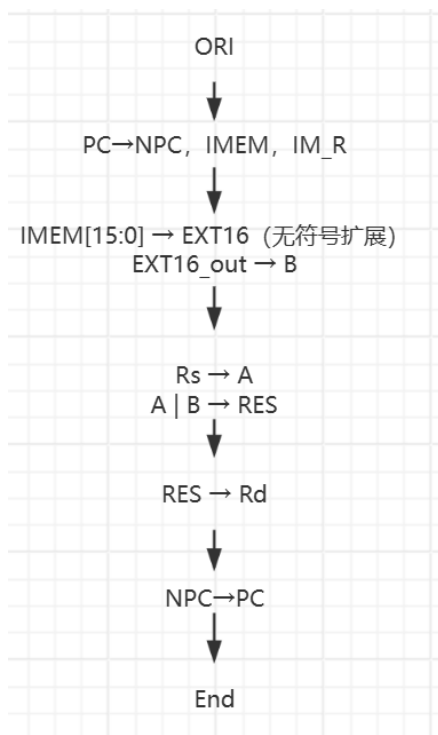
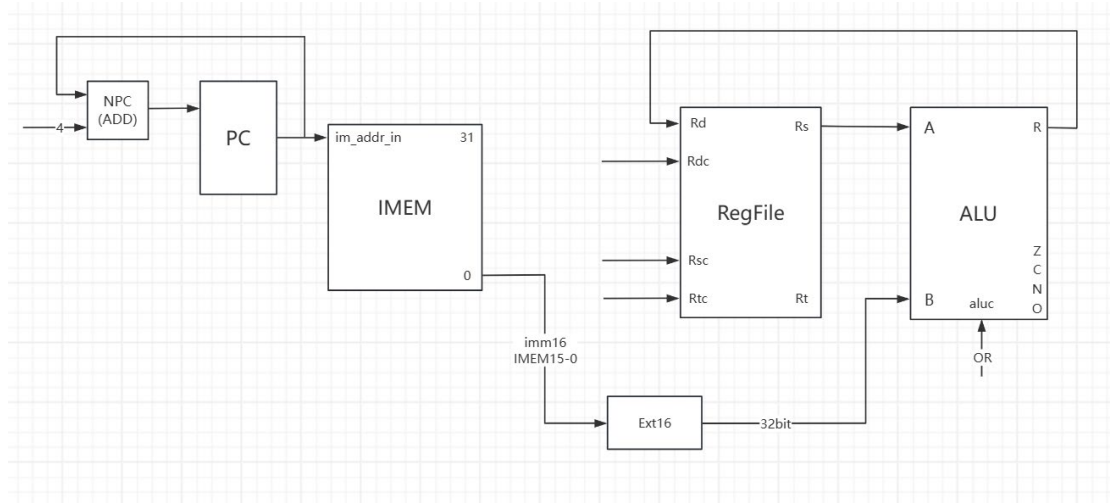
21. ORI

格式: ORI rt, rs, immediate

目的: 和一个常数做按位逻辑或

描述: $rt \leftarrow rs \text{ OR } \text{immediate}$

将通用寄存器 rs 和经过 0 扩展的立即数每一位做按位或操作，将结果存入目标寄存器 rt。



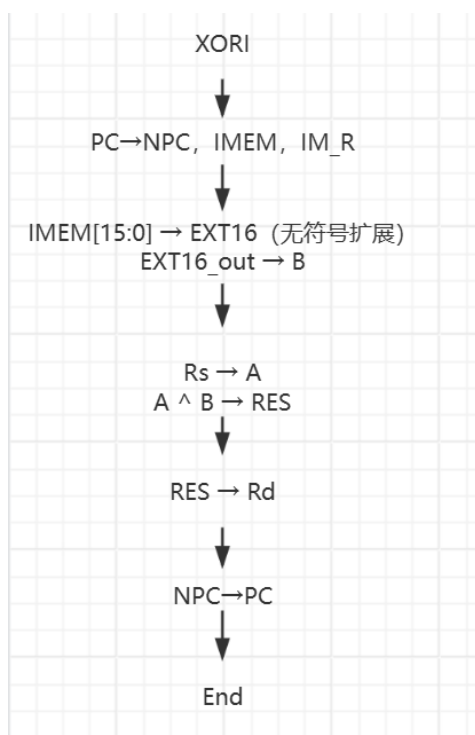
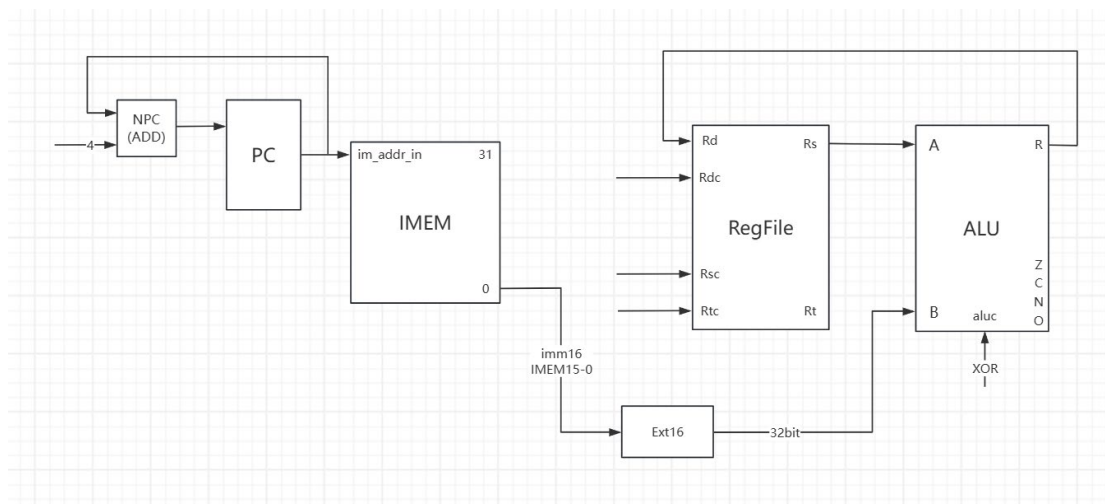
22. XORI

格式: XORI rt, rs, immediate

目的: 和一个常数做按位逻辑异或

描述: $rt \leftarrow rs \text{ XOR } \text{immediate}$

将通用寄存器 rs 和经过 0 扩展的立即数每一位做按位异或操作, 将结果存入目标寄存器 rt。



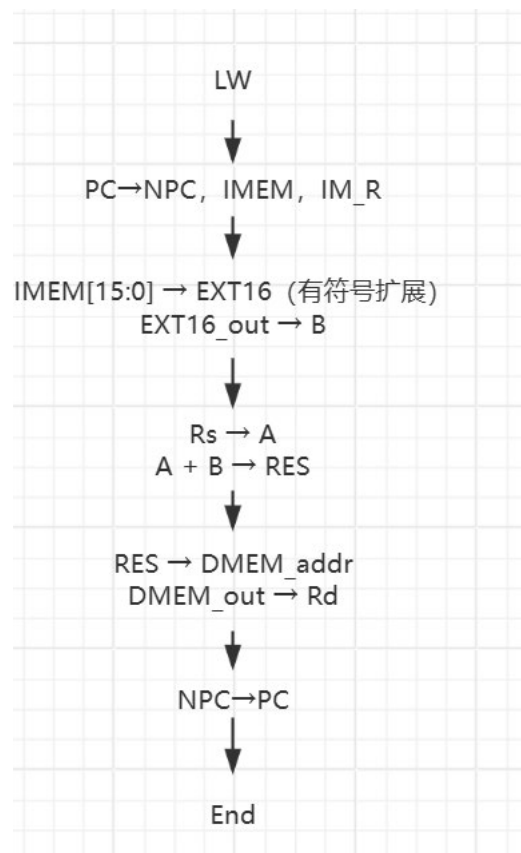
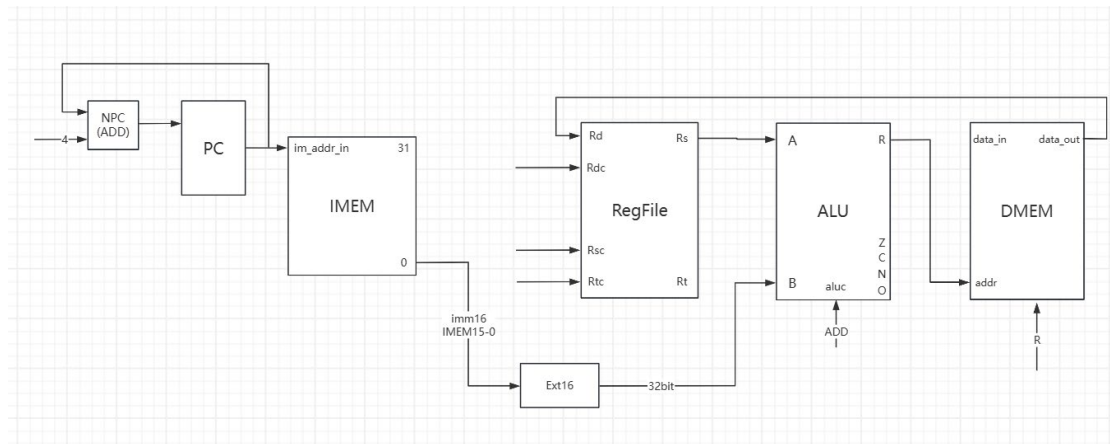
23. LW

格式: LW *rt*, *offset*(*base*)

目的: 从内存读取一个字的有符号数据

描述: $rt \leftarrow \text{memory}[\text{base} + \text{offset}]$

从内存中基地址加偏移量所得到的准确地址中的内容加载到通用寄存器 *rt* 中。



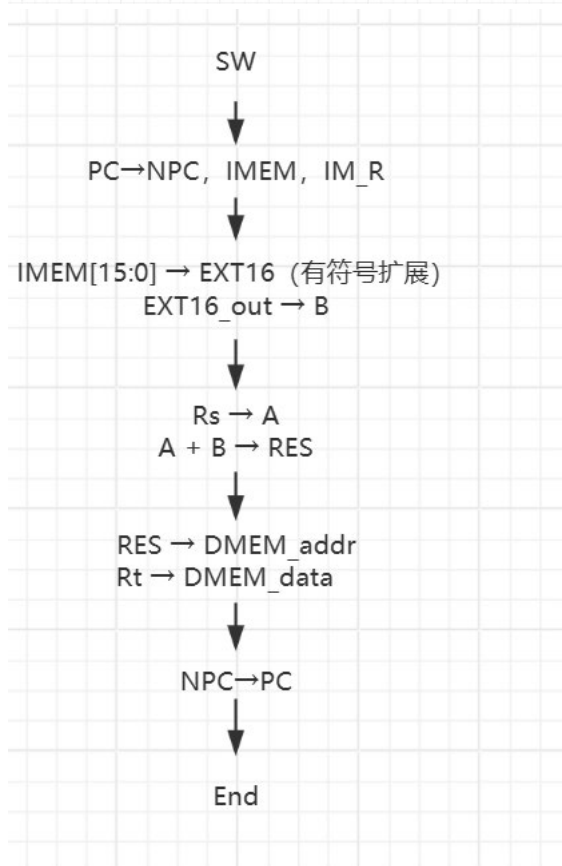
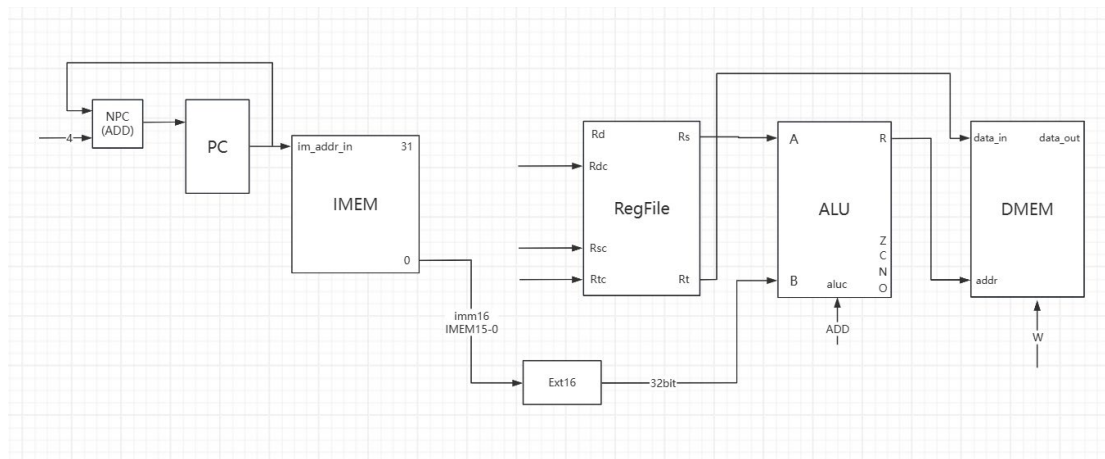
24. SW

格式: SW rt, offset(base)

目的: 存一个字到内存

描述: $\text{memory}[\text{base} + \text{offset}] \leftarrow \text{rt}$

将通用寄存器 rt 中的 32 位数据存入内存中的有效地址，有效地址由基地址和 16 位偏移量相加所得。



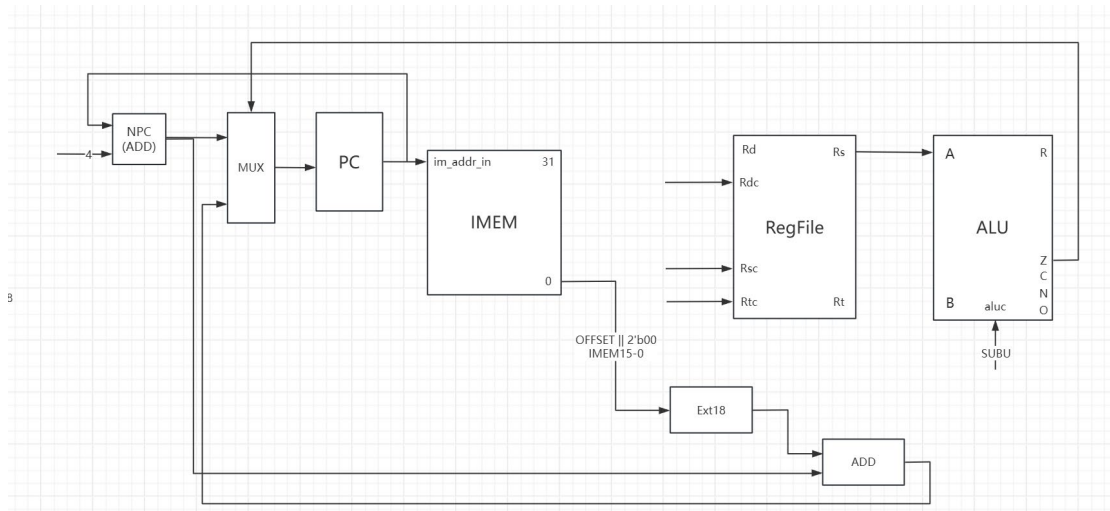
25. BEQ

格式: BEQ rs, rt, offset

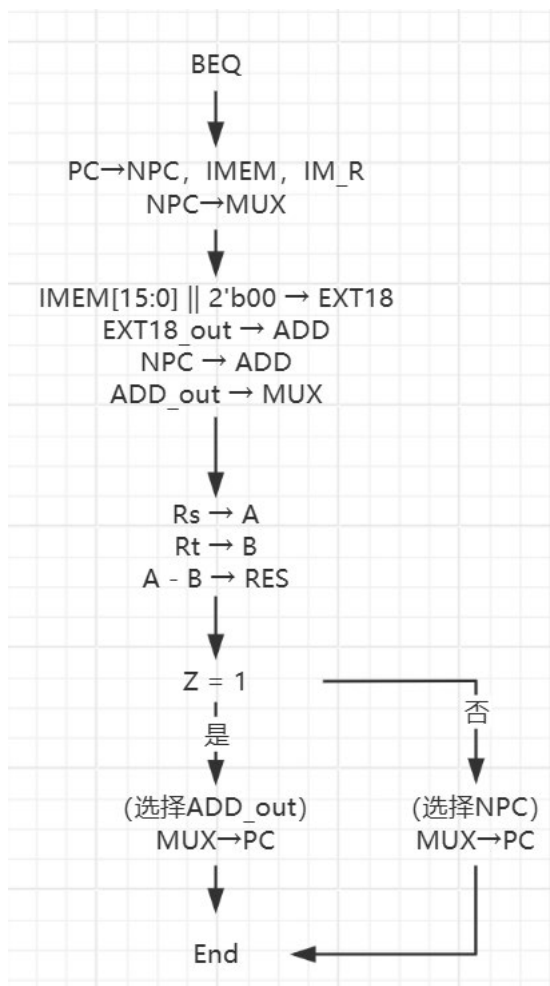
目的: 比较通用寄存器的值, 然后做 pc 相关的分支跳转

描述: 如果 $rs = rt$, 那么将 offset 左移两位, 再进行符号扩展到 32 位与当前 pc 相加, 形成有效转移地址, 转到该地址。

如果 $rs \neq rt$, 则继续执行下条指令。



(图中 Rt 应该连 B，这里漏掉了)



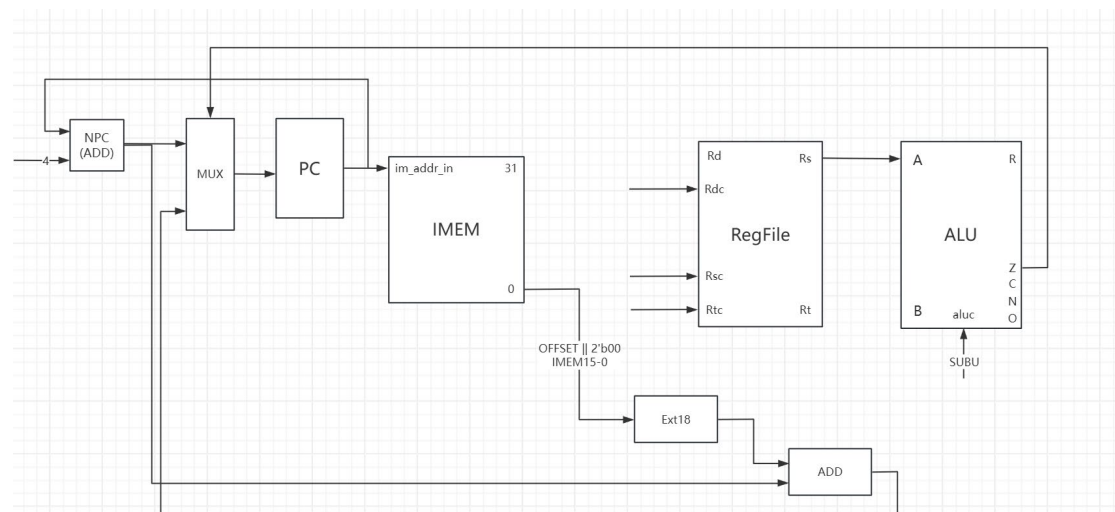
26. BNE

格式: BNE rs, rt, offset

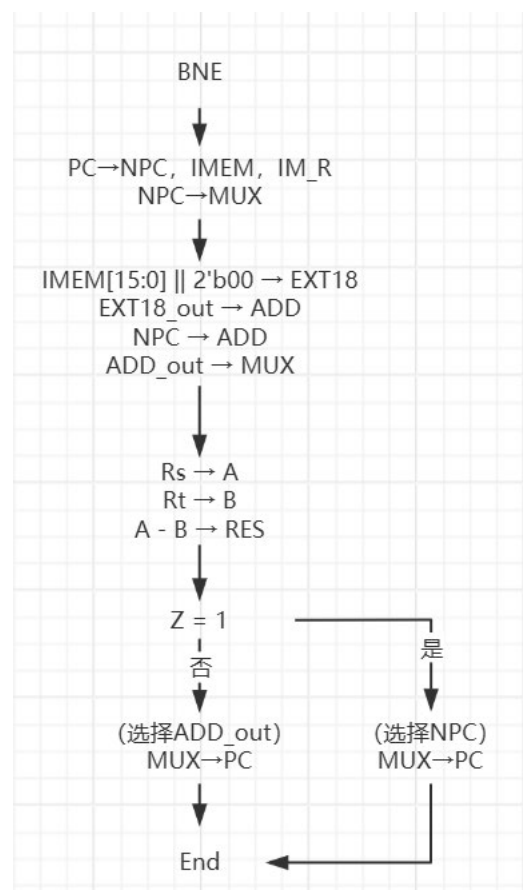
目的: 比较通用寄存器的值，然后做 pc 相关的分支跳转

描述: 如果 $rs \neq rt$ ，那么将会跳转到现在 pc 与偏移量 offset（如果是 16 位需扩展到 18 位）相加后所得的指令。

如果 $rs = rt$ ，则继续执行。



(图中 Rt 应该连 B ，这里漏掉了)



27. SLTI

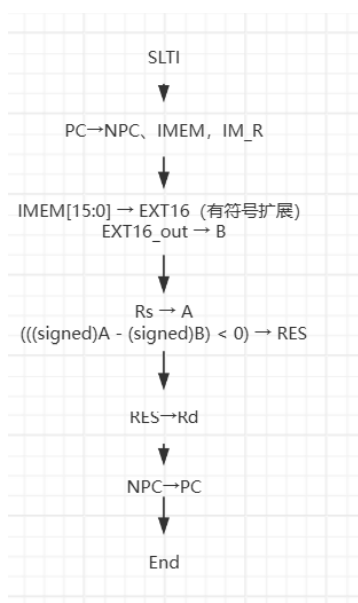
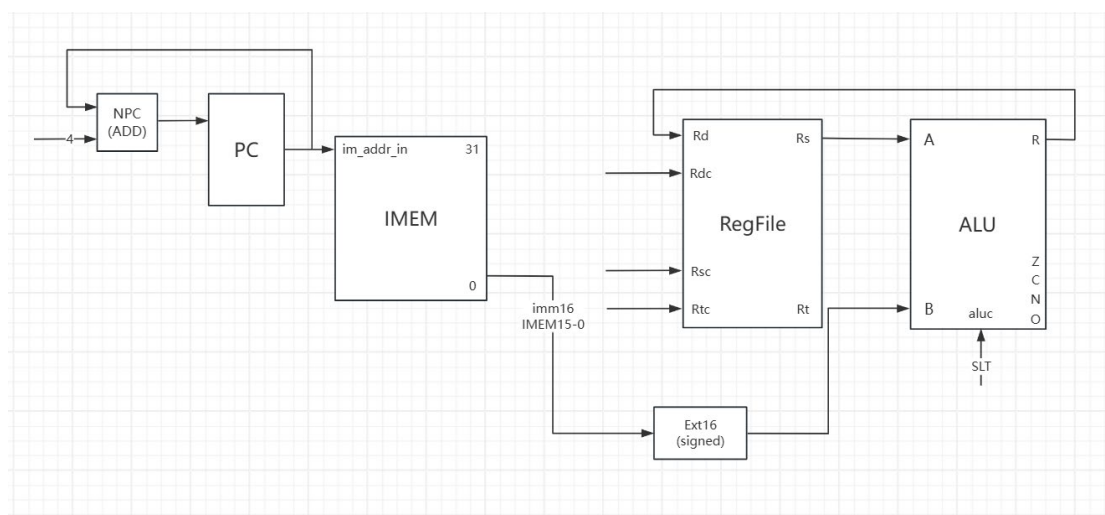
格式: `SLTI rt, rs, immediate`

目的: 通过跟立即数小于的比较来记录结果

描述: $rt \leftarrow (rs < \text{immediate})$

比较 rs 中的数和经过符号扩展的 16 位立即数，用 `boolean` 值保存结果到 rt 寄存器中。

如果 rs 中的数小于立即数，则结果为 1，反之结果为 0。算数比较不会引起溢出异常。



28. SLTIU

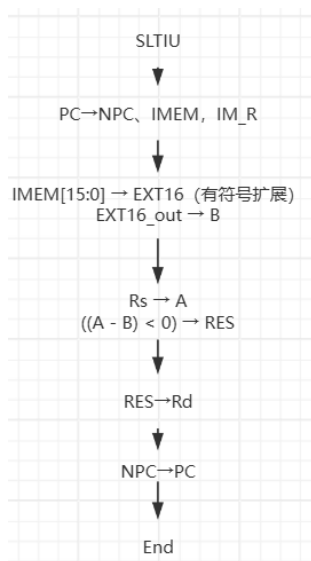
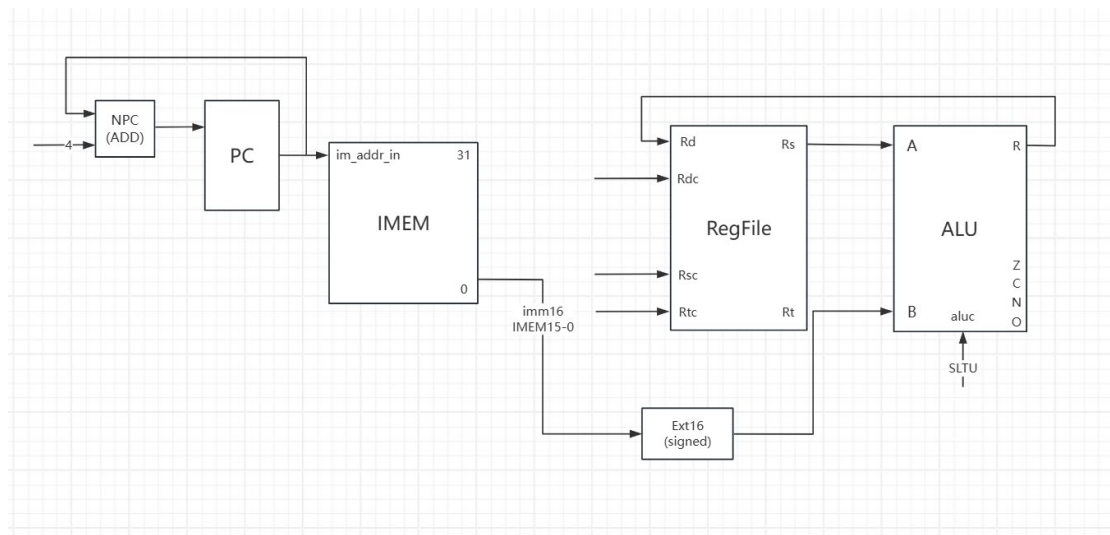
格式: SLTIU rt, rs, immediate

目的: 通过跟立即数进行无符号小于的比较来记录结果

描述: $rt \leftarrow (rs < \text{immediate})$

把 rs 中的数和经过符号扩展的 16 位立即数进行无符号小于比较，用 boolean 值保存结果到 rt 寄存器中。

如果 rs 小于符号扩展的立即数，则结果为 1，反之结果为 0。算数比较不会引起溢出异常。



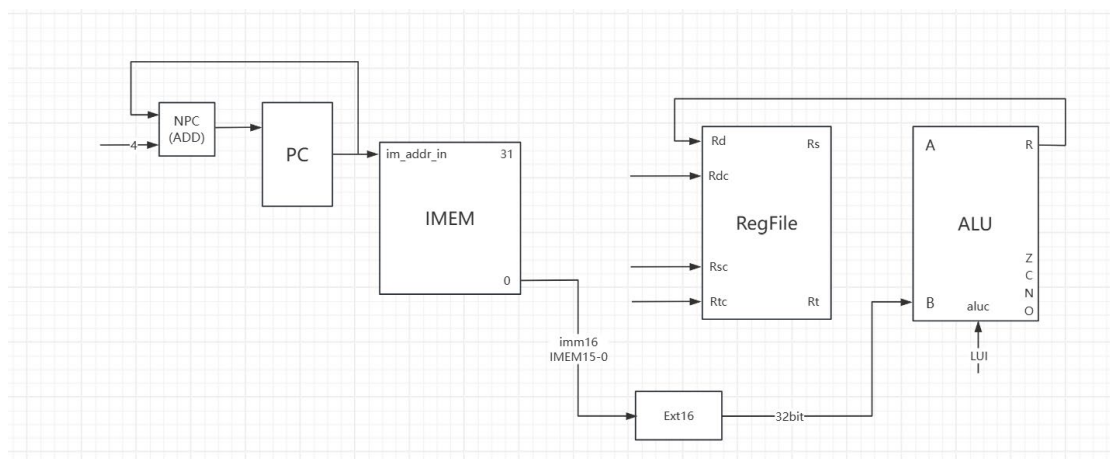
29. LUI

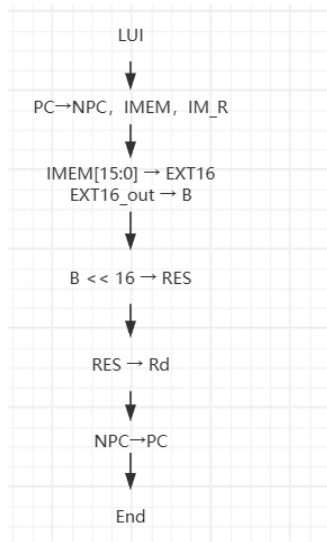
格式: LUI rt, immediate

目的: 把一个立即数载入到寄存器的高位, 低位补 0

描述: $rt \leftarrow \text{immediate} \parallel 0^{16}$

将一个 16 位的立即数载入到通用寄存器 rt 的高位, 低位补 0。



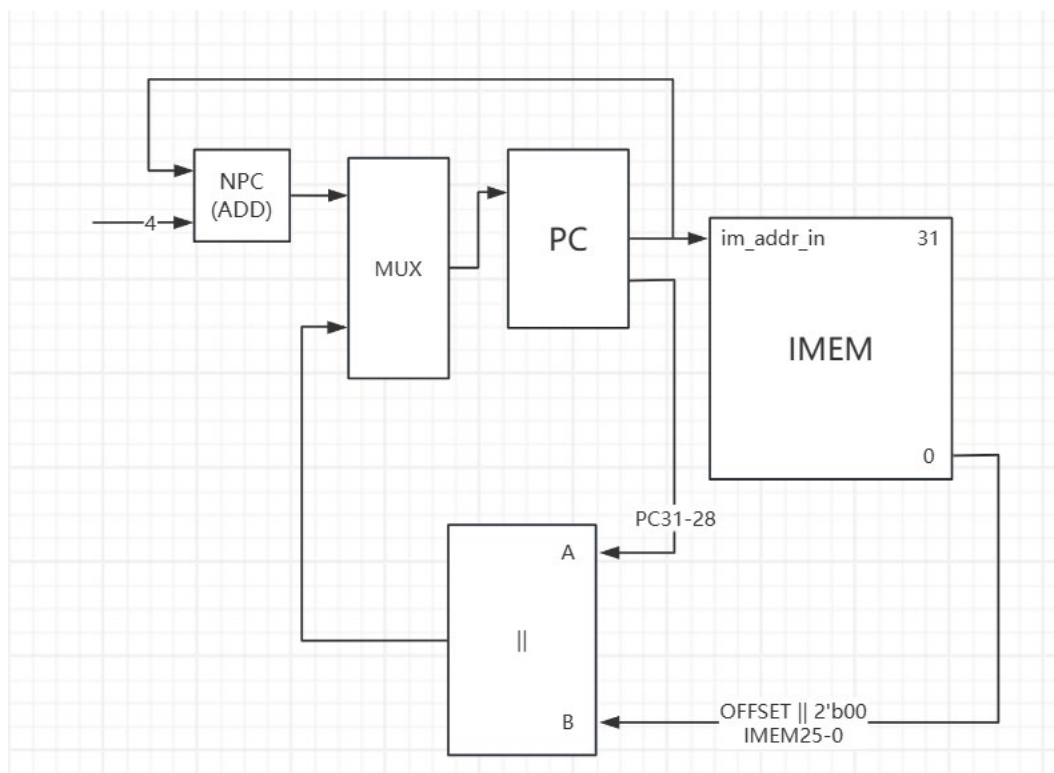


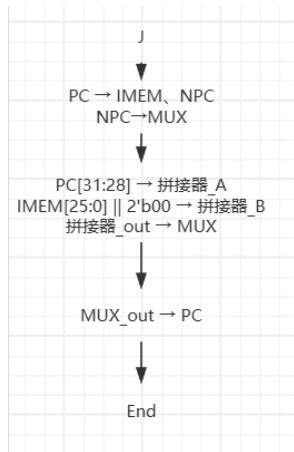
30. J

格式: J target

目的: 在 256MB 的范围内跳转

描述: 该指令无条件跳转到一个绝对地址, instr_index 有 26 位, 在左移过后访问空间能达到 256M。



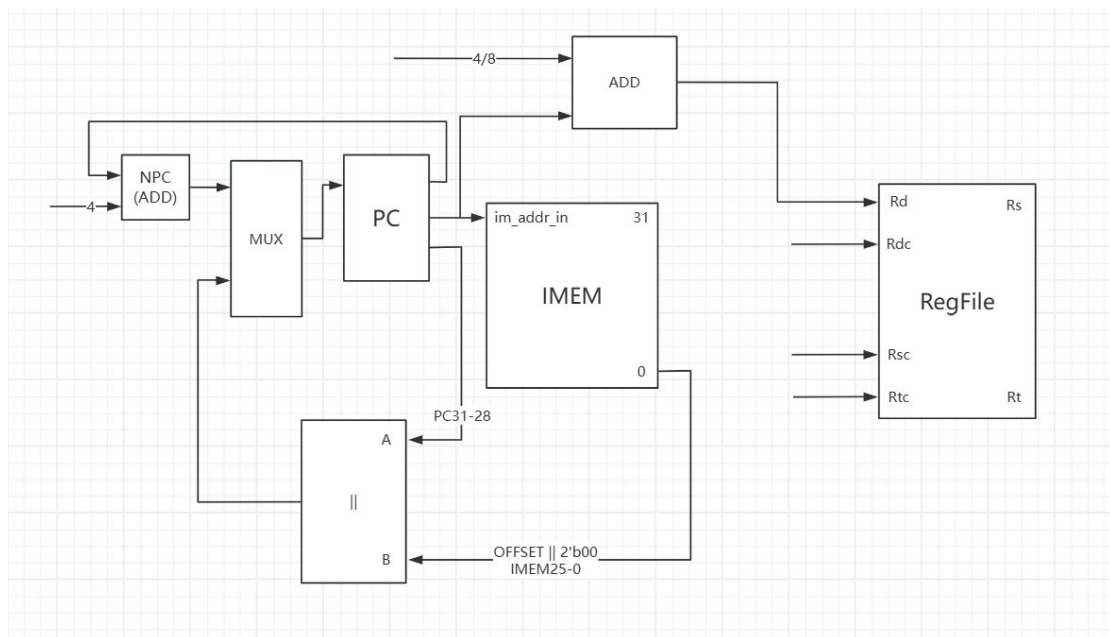


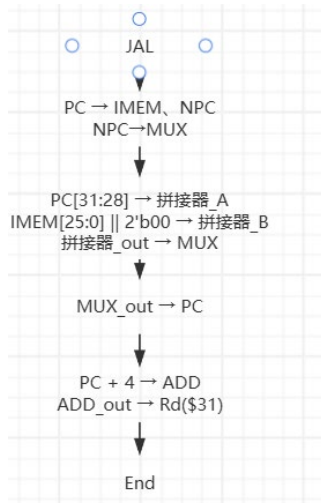
31. JAL

格式: JAL target

目的: 在 256MB 范围内执行一个过程调用

描述: 在跳转到指定地址执行子程序调用的同时, 在 31 号寄存器中存放返回地址 (当前地址后的第二条指令地址)。





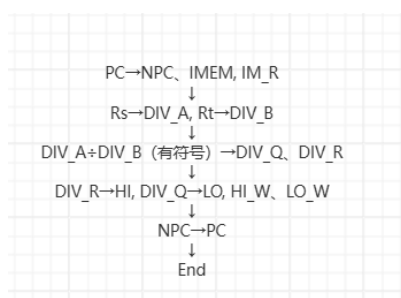
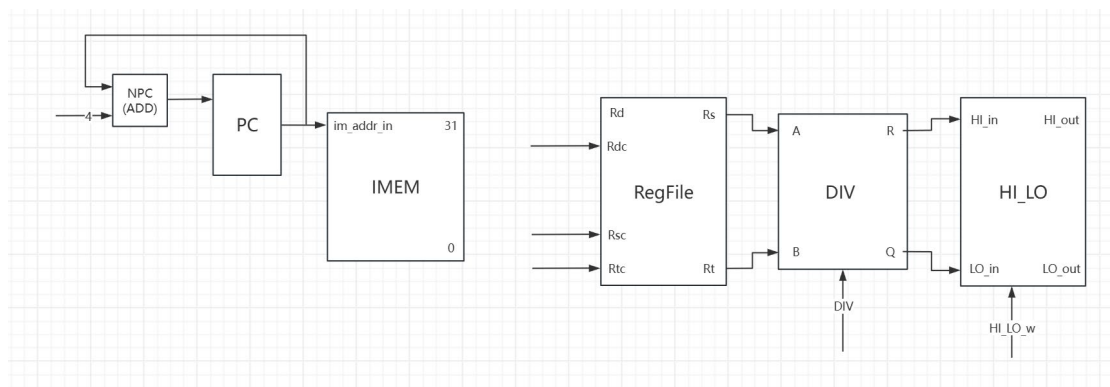
32. DIV

格式: DIV rs, rt

描述: (HI, LO) ← rs / rt

目的: 32 位有符号数除法 将通用寄存器 rs 和 rt 中的数据当做有符号数进行除法运算。

- 32 位的商存入 LO 寄存器，32 位的余数存入 HI 寄存器。
- 在任何情况下都不会产生算术异常。



33. DIVU

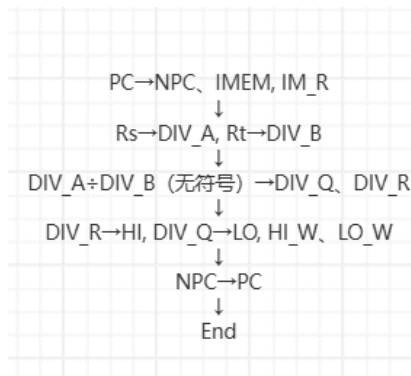
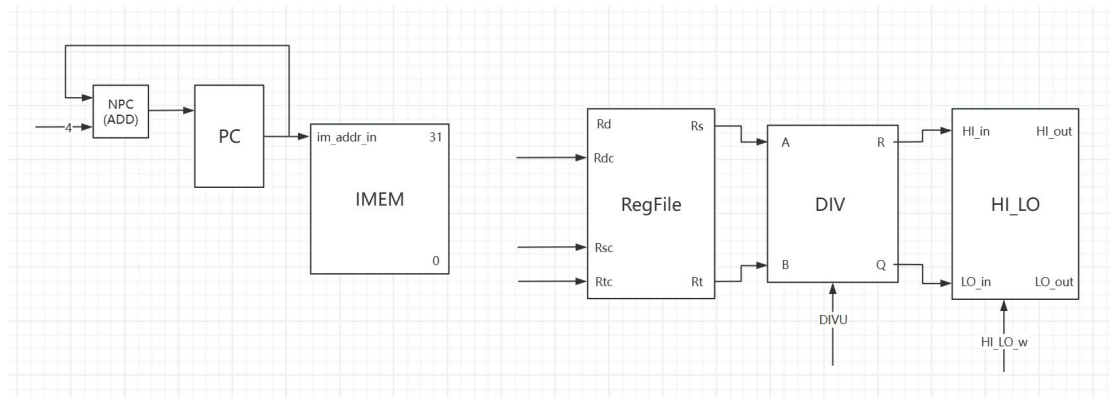
格式: DIVU rs, rt

描述: (HI, LO) ← rs / rt

目的: 32 位无符号数除法 将通用寄存器 rs 和 rt 中的数据当做无符号数进行除法运算。

- 32 位的商存入 LO 寄存器，32 位的余数存入 HI 寄存器。

- 在任何情况下都不会产生算术异常。



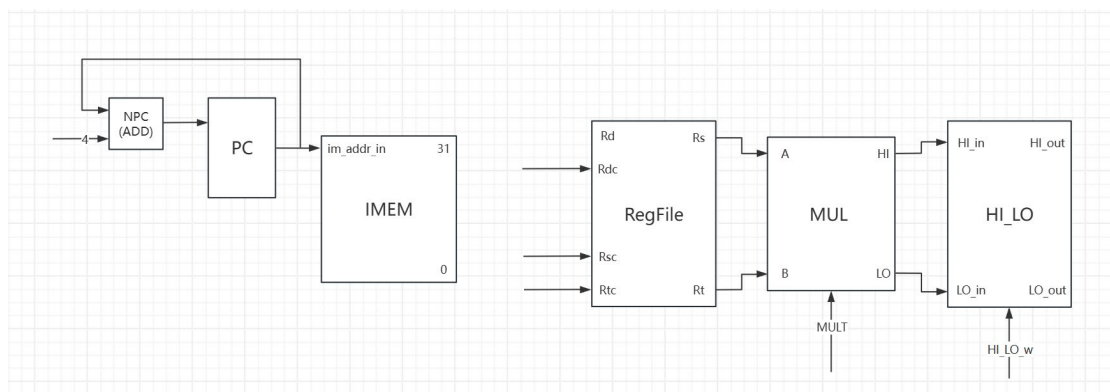
34. MULT

格式: MULT rs, rt

描述: (HI, LO) $\leftarrow$ rs * rt

目的: 32 位有符号数乘法。将通用寄存器 rs 和 rt 中的数据当做有符号数进行乘法运算，产生一个 64 位结果。

- 结果的低 32 位写入 LO 寄存器，高 32 位写入 HI 寄存器。
- 在任何情况下都不会产生算术异常。





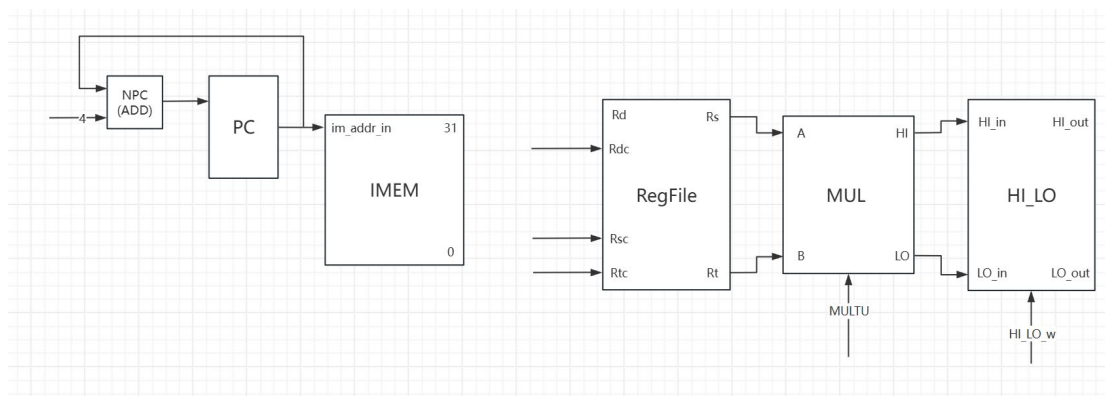
35. MULTU

格式: MULTU rs, rt

描述: (HI, LO) $\leftarrow$ rs * rt

目的: 32 位无符号数乘法。将通用寄存器 rs 和 rt 中的数据当做无符号数进行乘法运算，产生一个 64 位结果。

- 结果的低 32 位写入 LO 寄存器，高 32 位写入 HI 寄存器。
- 在任何情况下都不会产生算术异常。



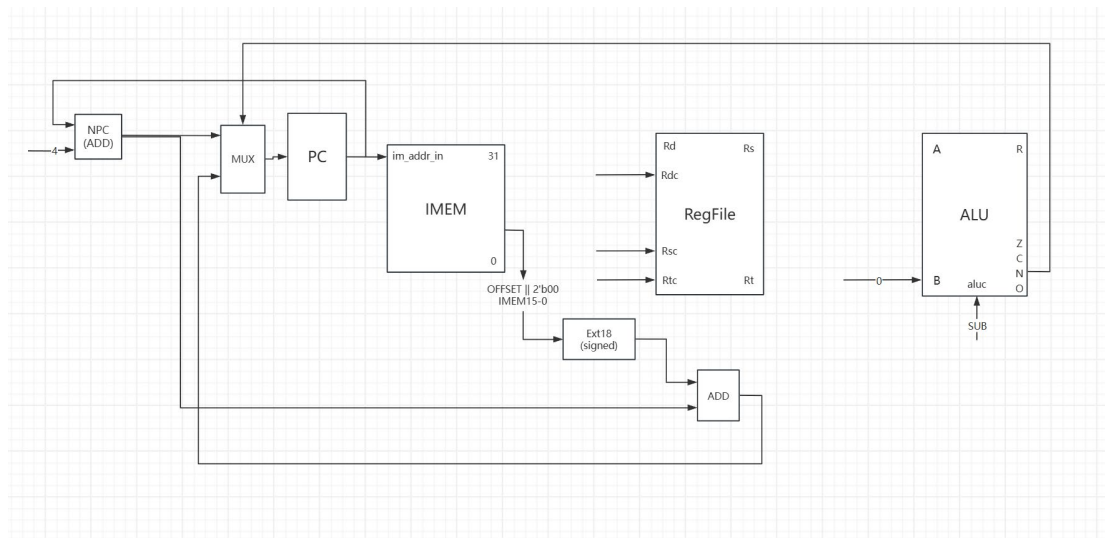
36. BGEZ

格式: BGEZ rs, offset

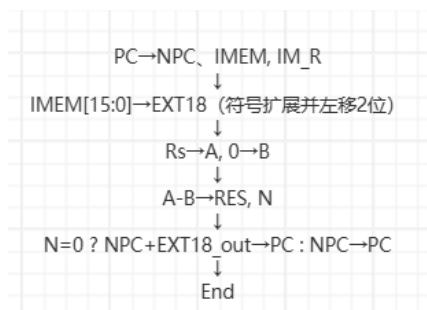
描述: 如果 rs ≥ 0 ，则进行 PC 相关的跳转

目的: 检测通用寄存器的值然后进行 PC 相关的跳转。如果 rs 大于等于 0，则在指令延时槽执行后跳转到当前 PC 与偏移量 offset 相加后所得的指令地址。

- 16 位偏移量 offset 左移 2 位并进行符号扩展后，与 PC 相加形成分支目标地址。



(图中 Rs 到 ALU.A 应当有一个连线，这里漏掉了)



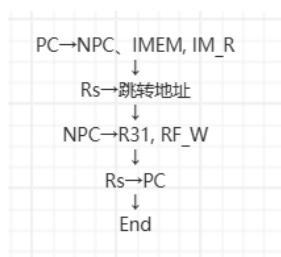
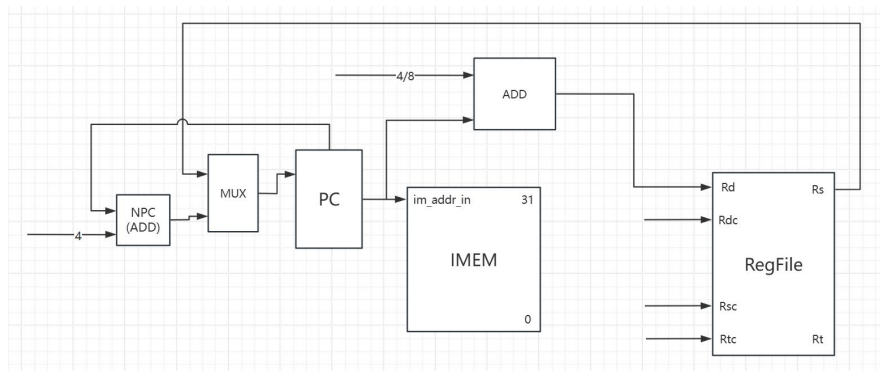
37. JALR

格式: JALR rs

描述: $\$31 \leftarrow \text{return_addr}, PC \leftarrow rs$

目的: 执行一个在寄存器中存放指令地址的过程调用。跳转目标地址存放在通用寄存器 rs 中，跳转时将返回地址保存到 31 号寄存器。

· 课件里的标准返回地址为 PC+8；但 Mars 中执行的是 PC+4。



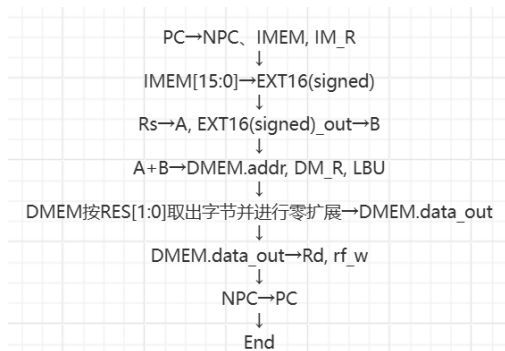
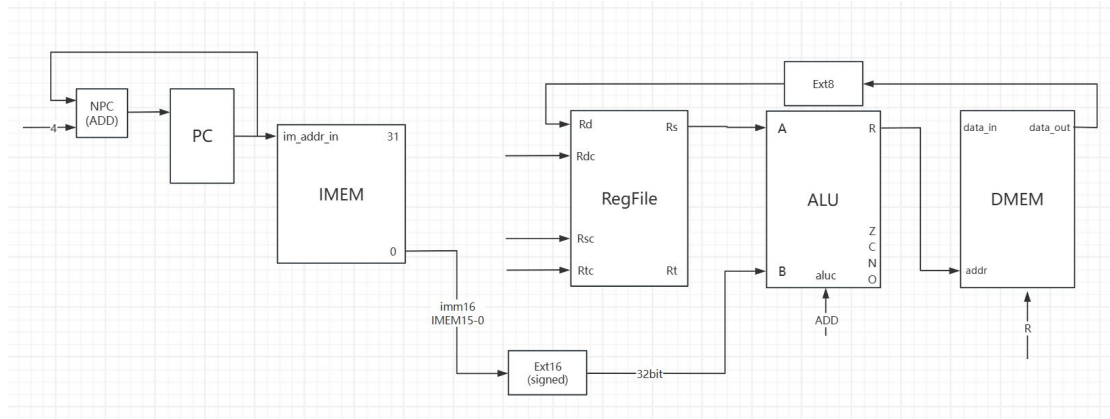
38. LBU

格式: LBU rt, offset(base)

描述: $rt \leftarrow \text{memory}[\text{base} + \text{offset}]$

目的: 从内存加载一个无符号字节。从基地址 **base** 与符号扩展后的偏移量 **offset** 相加所得的有效地址中取出一个 8 位字节。

- 取出的字节经 0 扩展后存入通用寄存器 rt。



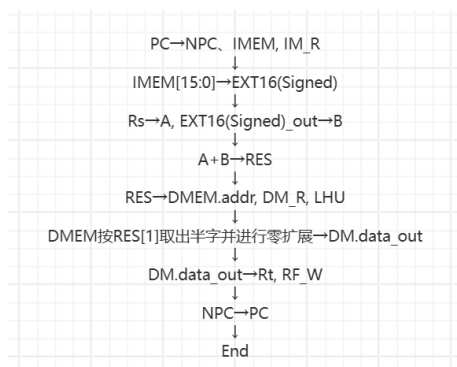
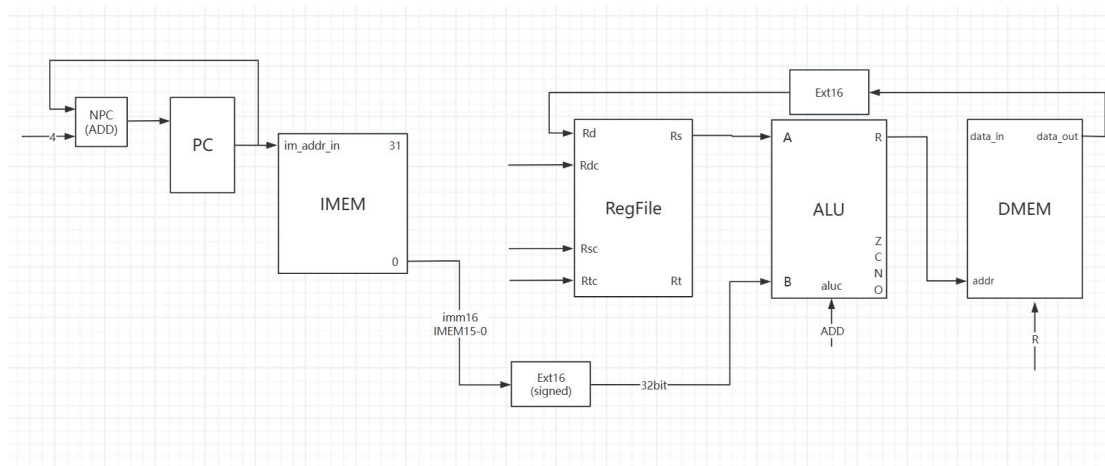
39. LHU

格式: LHU rt, offset(base)

描述: $rt \leftarrow \text{memory}[\text{base} + \text{offset}]$

目的: 从内存加载一个无符号半字。从基地址 `base` 与符号扩展后的偏移量 `offset` 相加所得的有效地址中取出一个 16 位半字。

- 取出的半字经 0 扩展后存入通用寄存器 rt。
- 若有效地址最低位不为 0，则产生地址错误异常。



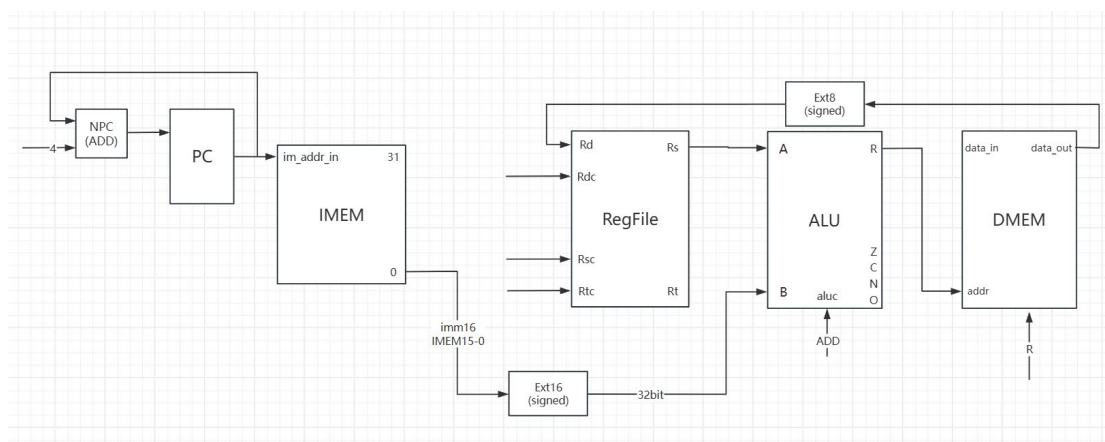
40. LB

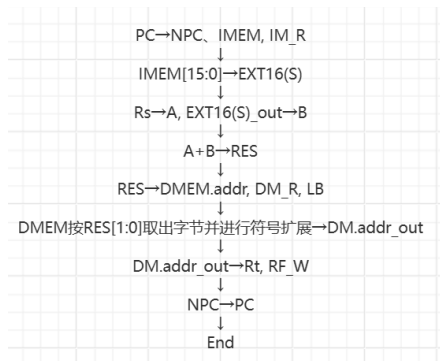
格式: LB rt, offset(base)

描述: $rt \leftarrow \text{memory}[\text{base} + \text{offset}]$

目的: 从内存加载一个有符号字节。从基地址 **base** 与符号扩展后的偏移量 **offset** 相加所得的有效地址中取出一个 8 位字节。

· 取出的字节经符号扩展后存入通用寄存器 **rt**。





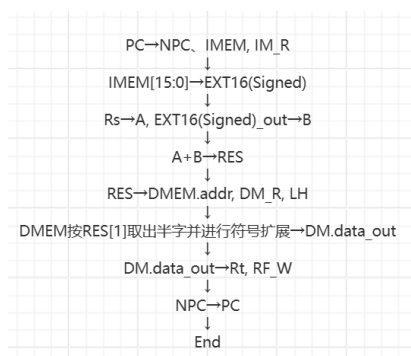
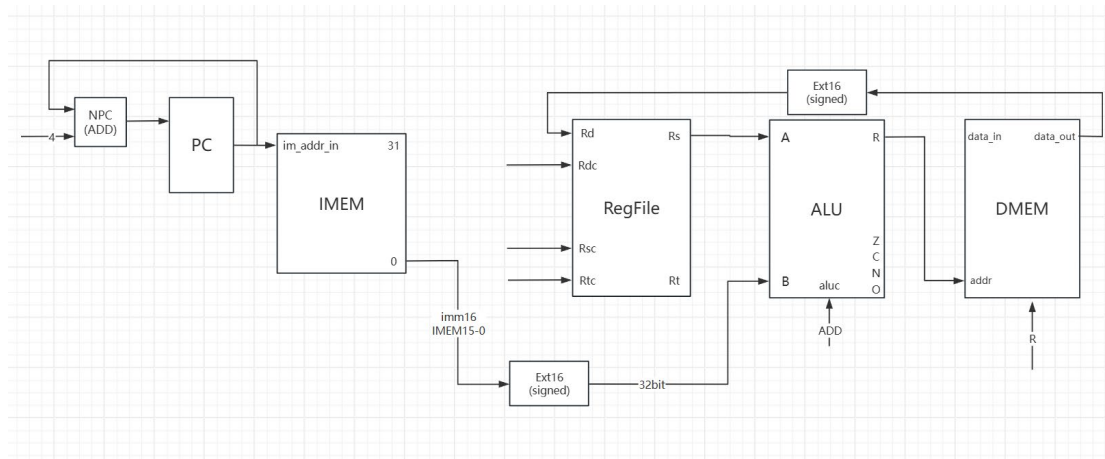
41. LH

格式: LH rt, offset(base)

描述: $rt \leftarrow \text{memory}[\text{base} + \text{offset}]$

目的: 从内存加载一个有符号半字。从基地址 **base** 与符号扩展后的偏移量 **offset** 相加所得的有效地址中取出一个 16 位半字。

- 取出的半字经符号扩展后存入通用寄存器 **rt**。
- 若有效地址最低位不为 0，则产生地址错误异常。

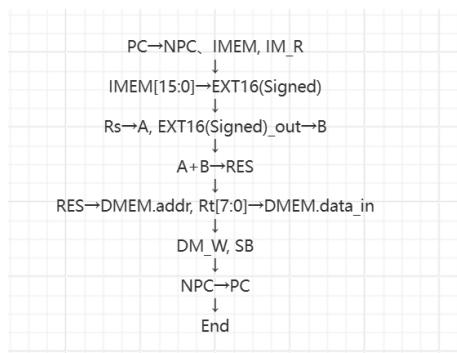
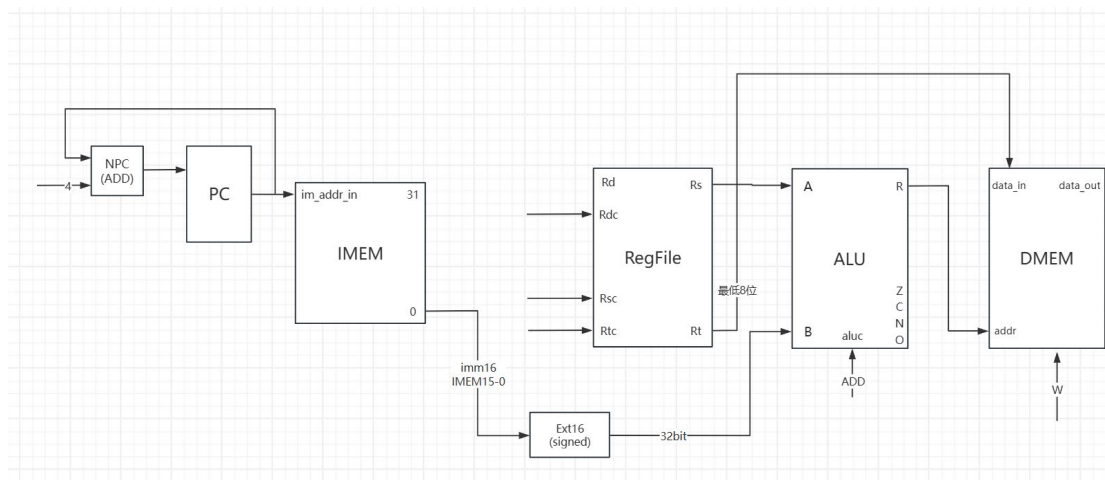


42. SB

格式: SB rt, offset(base)

描述: $\text{memory}[\text{base} + \text{offset}] \leftarrow rt$

目的: 存一个字节到内存。将通用寄存器 **rt** 中最低 8 位的数据存入由基地址 **base** 与符号扩展后的偏移量 **offset** 相加所得有效地址的内存中。



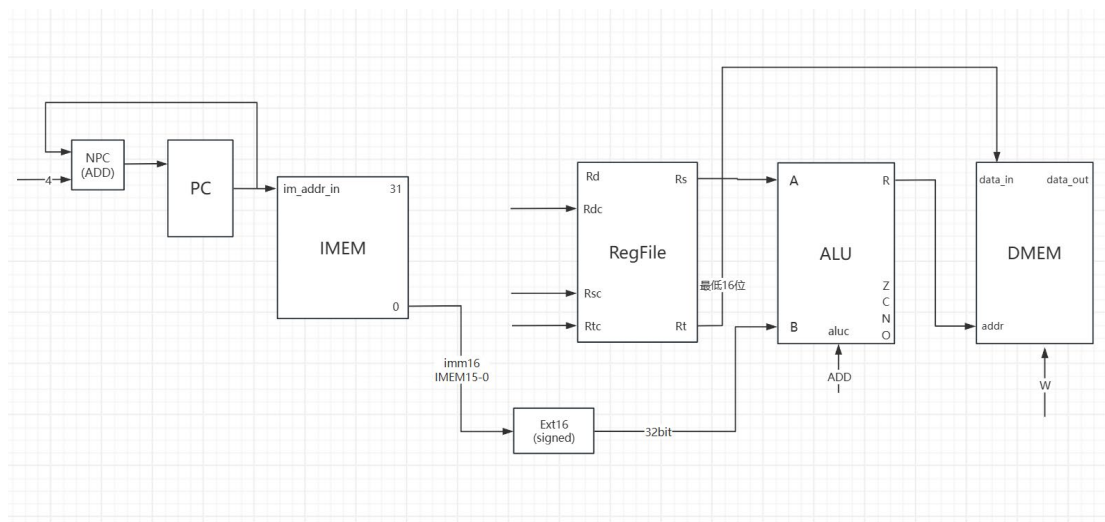
43. SH

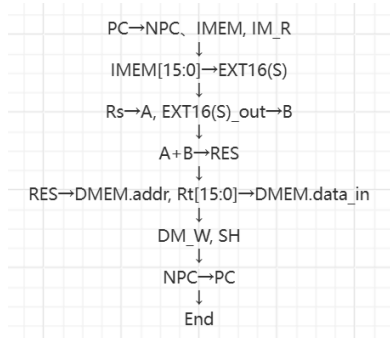
格式: SH rt, offset(base)

描述: $\text{memory}[\text{base} + \text{offset}] \leftarrow \text{rt}$

目的: 存一个半字到内存。将通用寄存器 **rt** 中最低 16 位的数据存入由基地址 **base** 与符号扩展后的偏移量 **offset** 相加所得有效地址的内存中。

· 若有效地址最低位不为 0，则产生地址错误异常。





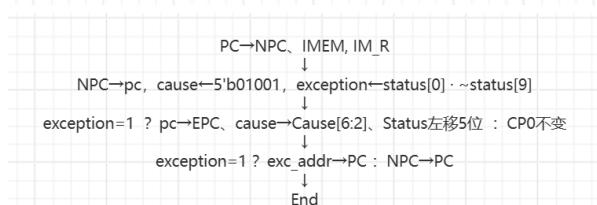
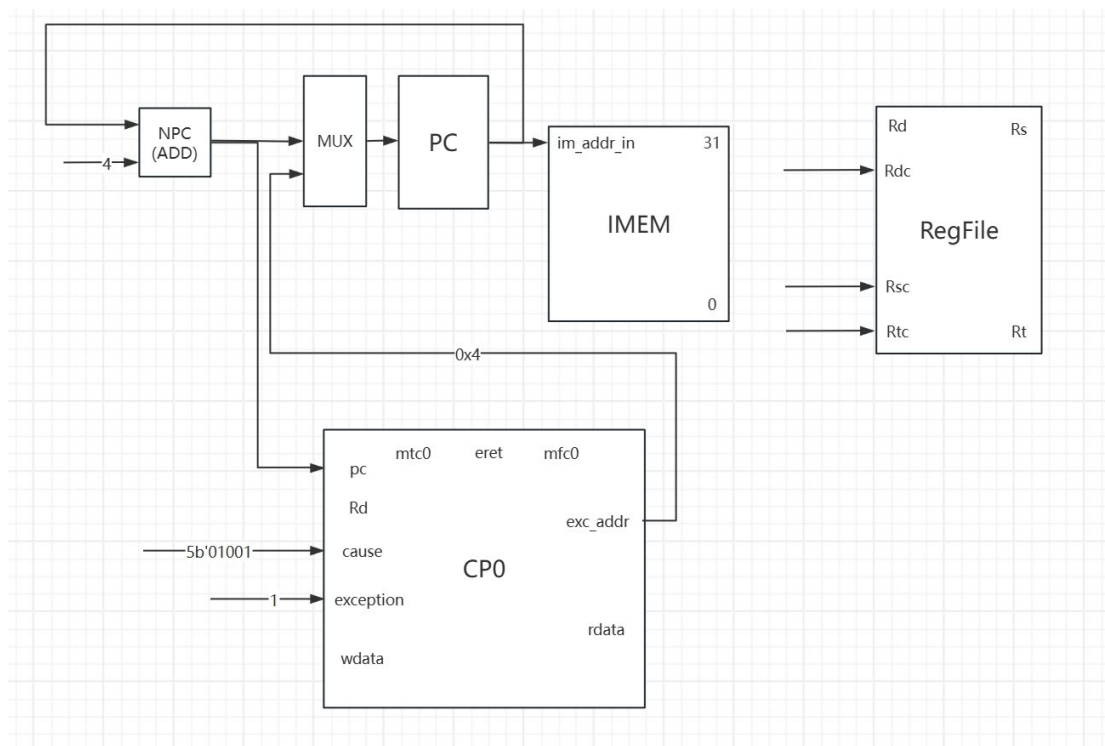
44. BREAK

格式: BREAK

描述: 引起一个断点异常

目的: 引起一个断点异常。当断点异常发生时, 处理器将立刻并且不可控制地转入异常处理。

- CP0 的 Cause 寄存器需要硬件通路写入正确的异常类型码。
- CP0 的 EPC 寄存器需要硬件通路写入 PC+4, 即 BREAK 指令的下一条指令地址。



45. SYSCALL

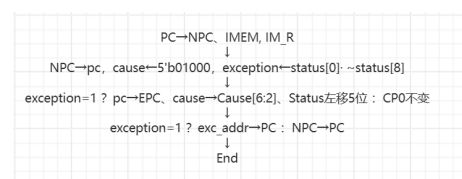
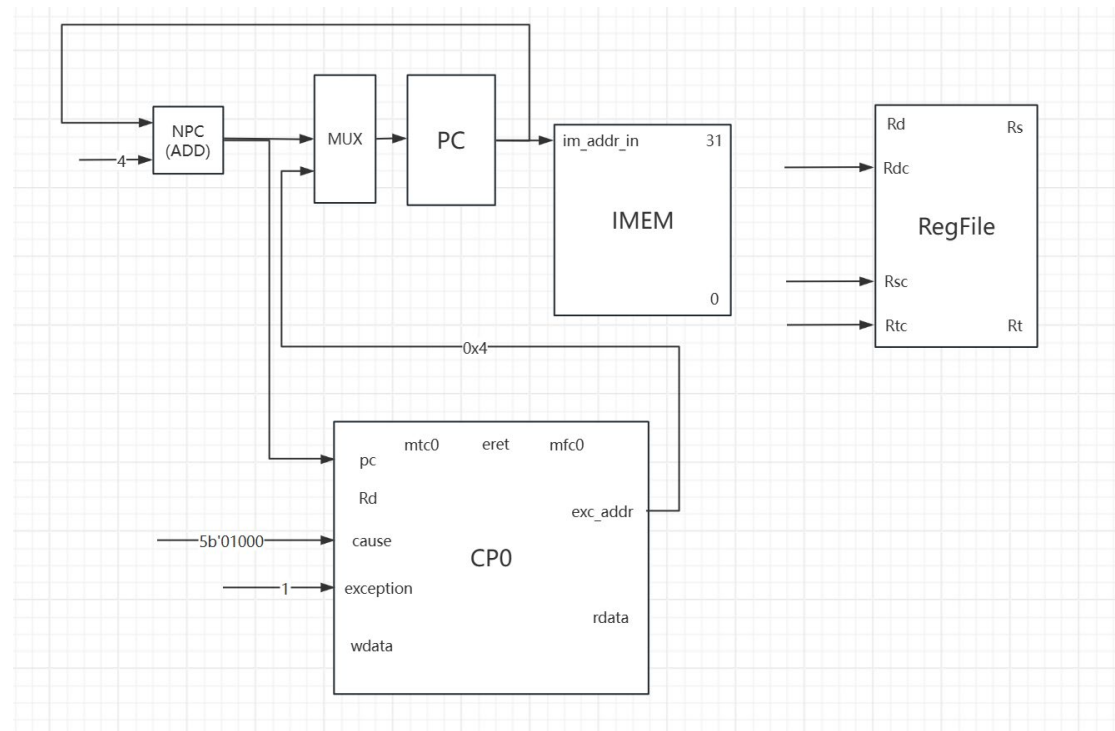
格式: SYSCALL

描述: 引发一个系统调用异常

目的: 引发一个系统调用异常 当系统调用发生时, 处理器将立刻并且不可控制地转入异常

处理。

- CP0 的 Cause 寄存器需要硬件通路写入正确的异常类型码。
- CP0 的 EPC 寄存器需要硬件通路写入 PC+4，即 SYSCALL 指令的下一条指令地址。



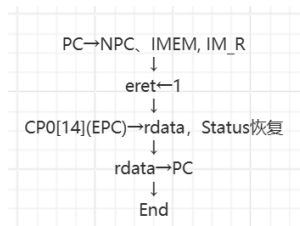
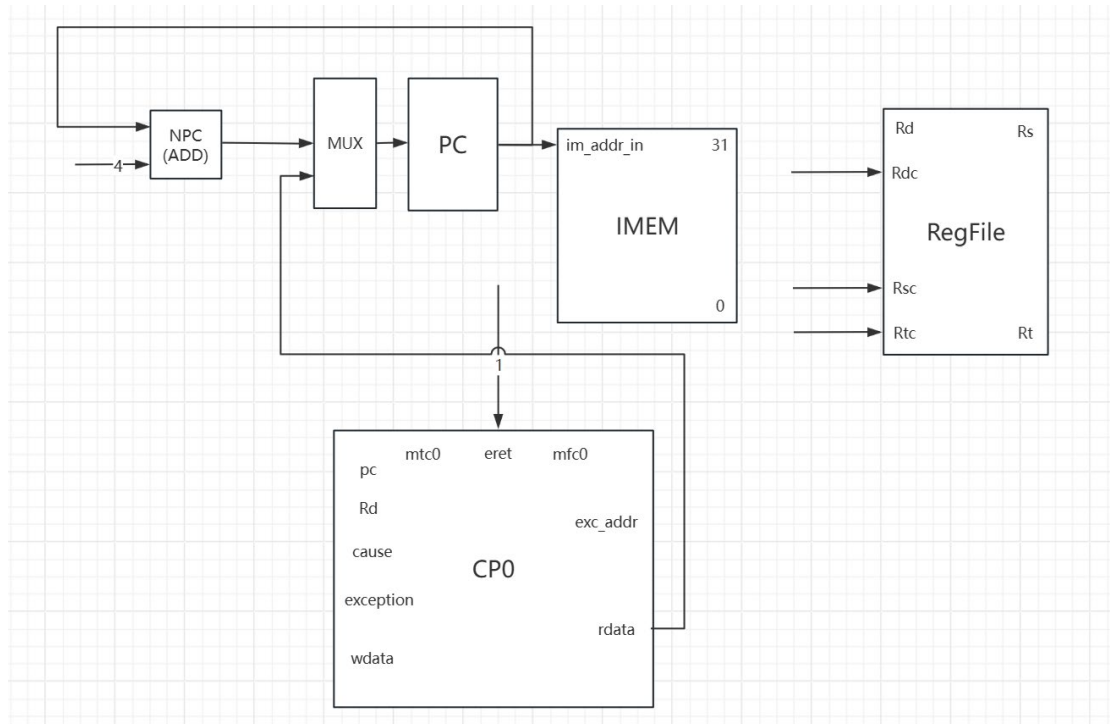
46. ERET

格式: ERET

描述: $PC \leftarrow EPC$

目的: 从异常中断返回。在所有中断处理过程结束后，返回发生中断或异常的程序。

- 将 CP0 的 EPC 寄存器中的地址送入 PC 寄存器，从异常处理程序返回主程序。
- ERET 不执行其下一条指令。

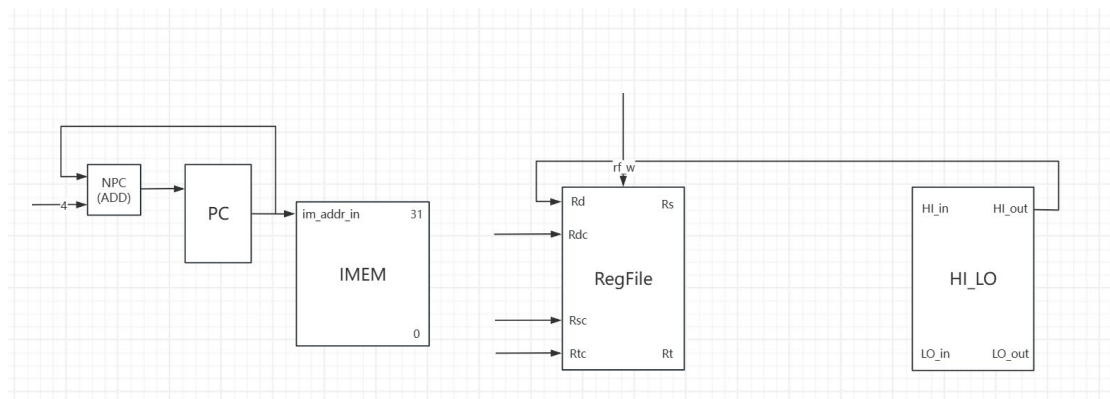


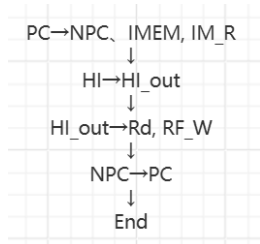
47. MFHI

格式: MFHI rd

描述: $rd \leftarrow HI$

目的: 复制特殊寄存器 HI 的内容到通用寄存器



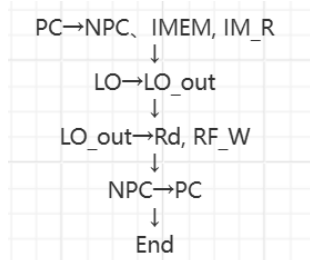
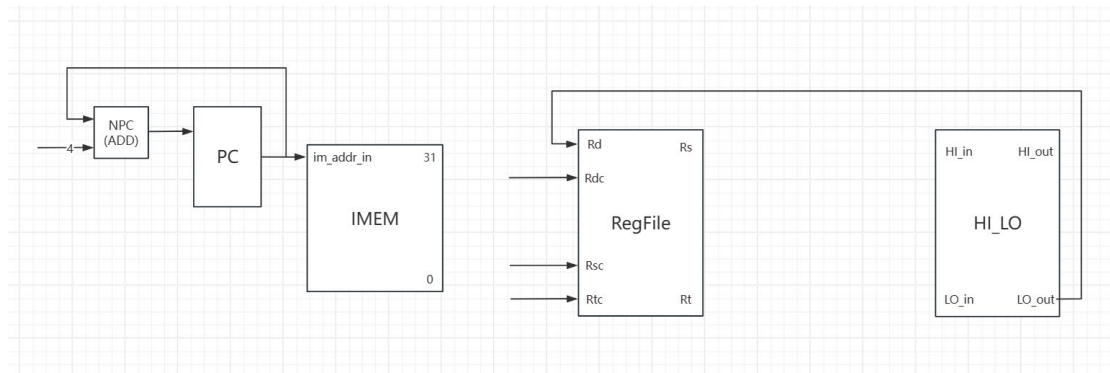


48. MFLO

格式: MFLO rd

描述: $rd \leftarrow LO$

目的: 复制特殊寄存器 LO 的内容到通用寄存器

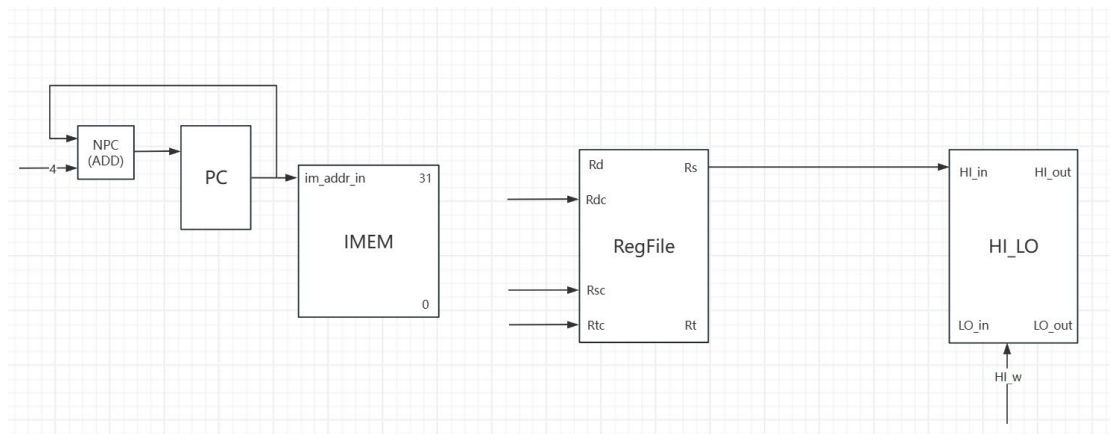


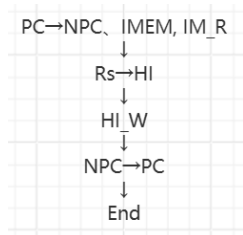
49. MTHI

格式: MTHI rs

描述: $HI \leftarrow rs$

目的: 复制通用寄存器的内容到特殊寄存器 HI 中



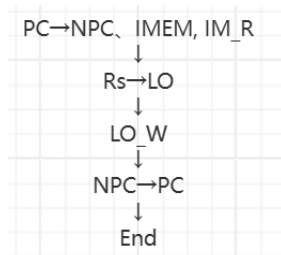
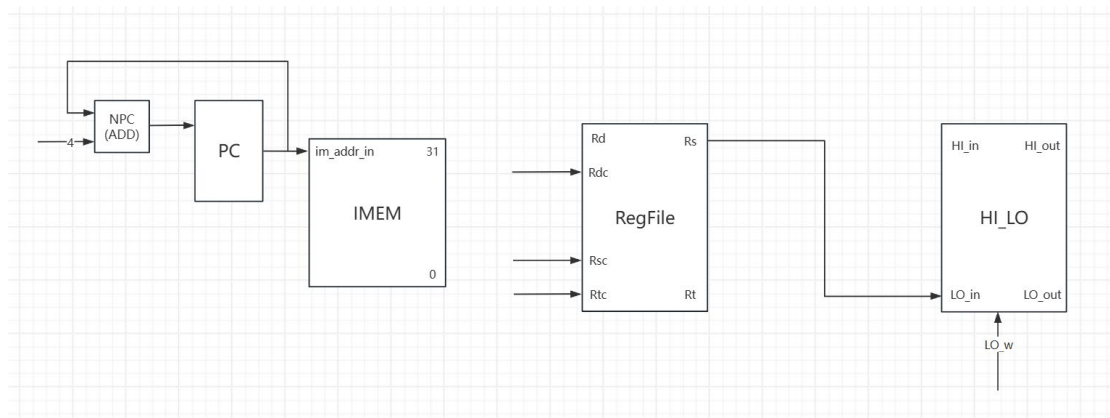


50. MTLO

格式: MTLO rs

描述: $LO \leftarrow rs$

目的: 复制通用寄存器的内容到特殊寄存器 LO 中

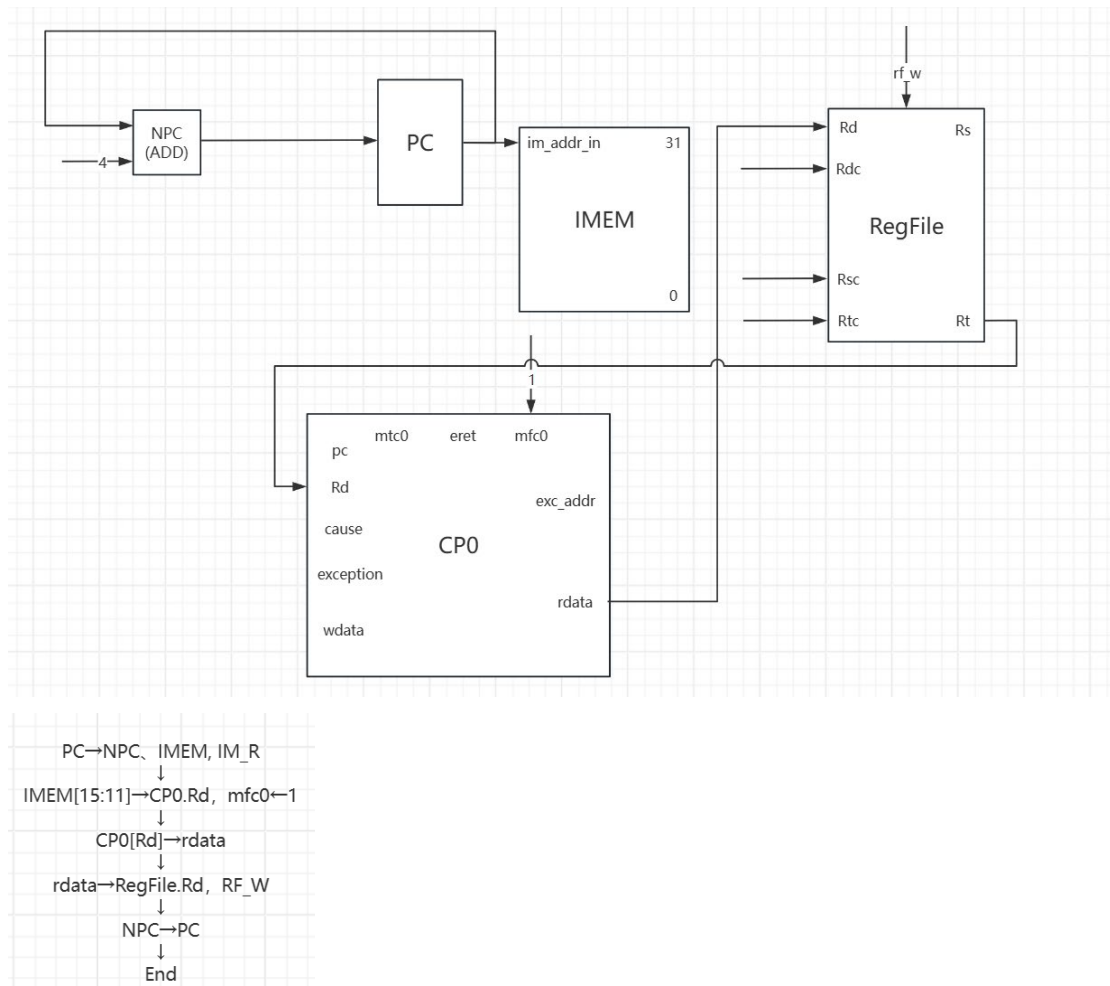


51. MFC0

格式: MFC0 rt, rd

描述: $rt \leftarrow CPR[0, rd, 0]$

目的: 把一个数据从特殊寄存器移到通用寄存器。由 rd 选择协处理器 0 中的特殊寄存器，并把它的内容转移到通用寄存器 rt 中。

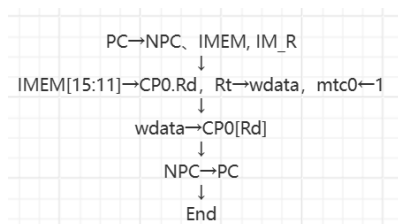
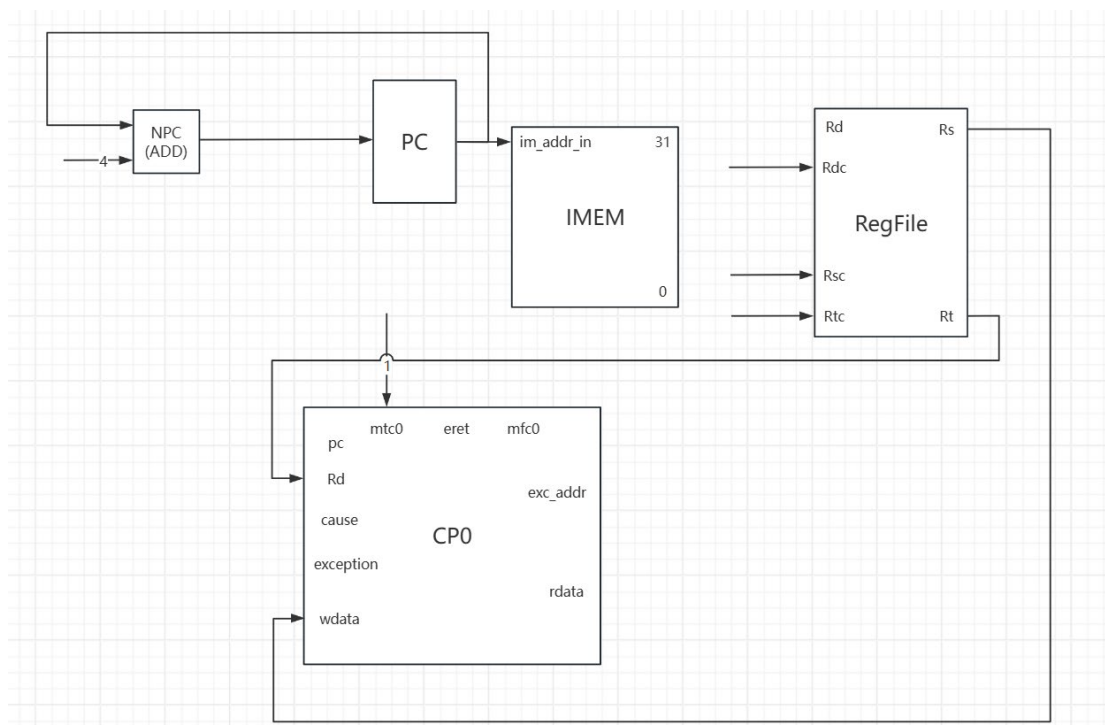


52. MTC0

格式: MTC0 rt, rd

描述: $CPR[0, rd, 0] \leftarrow rt$

目的: 把一个数据从通用寄存器移到特殊寄存器。由 rd 选择协处理器 0 中的特殊寄存器, 并把通用寄存器 rt 中的内容转移到该特殊寄存器中。

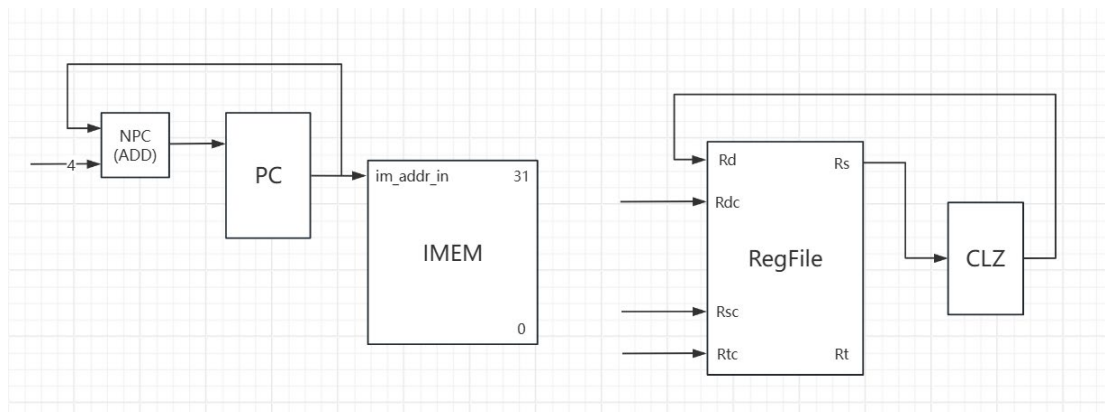


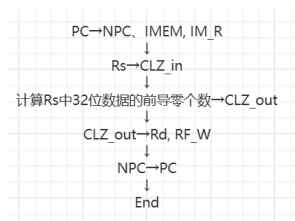
53. CLZ

格式: CLZ rd, rs

描述: $rd \leftarrow \text{CountLeadingZeros}(rs)$

目的: 计算 32 位字中的前导零个数。计算通用寄存器 rs 中 32 位数据的前导零个数，并将结果存入通用寄存器 rd 中。





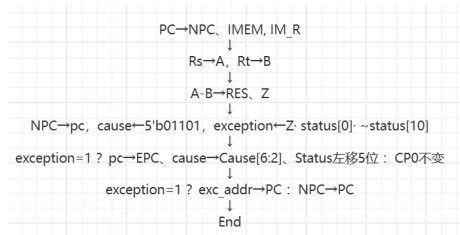
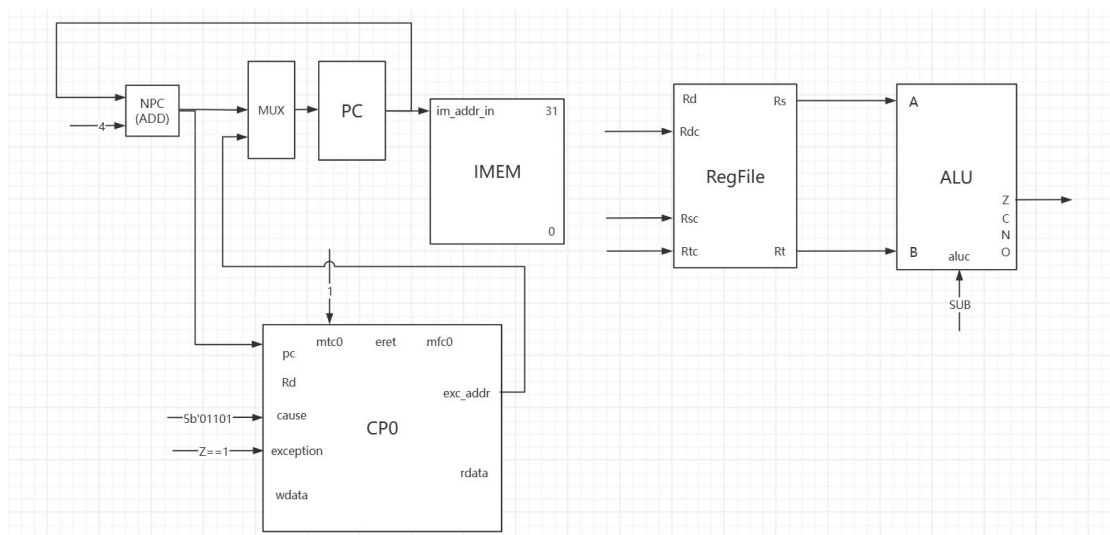
54. TEQ

格式: TEQ rs, rt

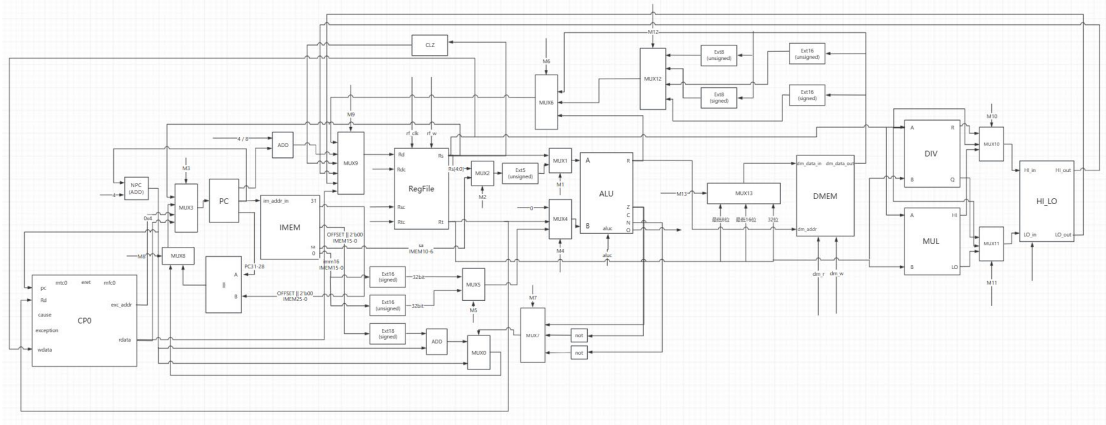
描述: 如果 $rs = rt$, 则引发自陷异常

目的: 比较寄存器值并根据条件引发异常。比较通用寄存器 rs 和 rt 的值, 若二者相等则引发一个自陷异常。

- CP0 的 Cause 寄存器需要硬件通路写入正确的异常类型码。
- CP0 的 EPC 寄存器需要硬件通路写入 $PC+4$, 即 TEQ 指令的下一条指令地址。



(3) 给出总的通路图。



(本图省略了 Rdc、Rsc 和 Rtc 的选择通路，后续的操作时间表和控制信号里也不会出现)

三、CPU 控制部件设计

报告要求：

(1) 根据指令流程图，编排指令取指到执行的操作时间表。

Op	PC	Regfile	Rw	Rd	A	B	aluOp	CDMEM				Evd	EvdS	EvdSigned	EvdSignedS	RAVX																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																						
								dm_r	dm_w	dm_vld	dm_data_in					dm_data	A	B	M1	M2	M3	M4	M5	M6	M7	M8	M9																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																											
ADD	PC + 4	ALU	Rn	Rn	Rn	Rn	aluOp1																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															</

$$\begin{aligned}
M7[0] &= i_bne \\
M8 &= i_j + i_jal \\
M9[2] &= i_mflo + i_mfc0 \\
M9[1] &= i_clz + i_mfhi \\
M9[0] &= i_jal + i_jalr + i_mfhi + i_mfc0 \\
M10[1] &= i_mult + i_multu \\
M10[0] &= i_div + i_divu \\
M11[1] &= i_mult + i_multu \\
M11[0] &= i_div + i_divu \\
M12[1] &= i_lh + i_lhu \\
M12[0] &= i_lbu + i_lhu \\
M13[1] &= i_sh \\
M13[0] &= i_sb
\end{aligned}$$

其中 M3 编码为:00 选择 MUX8,01 选择 Rs,10 选择 CP0.exc_addr,11 选择 CP0.rdata。
ERET 时 CP0 内部令 rdata 输出 CP0[14] (EPC)。

Regfile 写使能:

$$\begin{aligned}
rf_w &= i_add + i_addu + i_sub + i_subu + i_and + i_or + i_xor + i_nor + i_slt + i_sltu \\
&+ i_sll + i_srl + i_sra + i_sllv + i_srlv + i_srav + i_addi + i_addiu \\
&+ i_andi + i_ori + i_xori + i_lw + i_slti + i_sltiu + i_lui + i_jal + i_jalr \\
&+ i_lbu + i_lhu + i_lb + i_lh + i_mfhi + i_mflo + i_mfc0 + i_clz
\end{aligned}$$

DMEM 控制信号:

$$\begin{aligned}
dm_r &= i_lw + i_lbu + i_lhu + i_lb + i_lh \\
dm_w &= i_sw + i_sb + i_sh \\
dm_ena &= dm_r + dm_w
\end{aligned}$$

HI/LO 控制信号:

$$\begin{aligned}
HI_w &= i_div + i_divu + i_mult + i_multu + i_mthi \\
LO_w &= i_div + i_divu + i_mult + i_multu + i_mtlo
\end{aligned}$$

除法器、乘法器符号控制信号 (约定 1 为有符号, 0 为无符号):

$$\begin{aligned}
div_signed &= i_div \\
mul_signed &= i_mult
\end{aligned}$$

CP0 控制信号:

$$\begin{aligned}
mfc0 &= i_mfc0 \\
mtc0 &= i_mtc0 \\
eret &= i_eret \\
teq_taken &= i_teq \cdot Z \\
exception &= status[0] \cdot (i_syscall \cdot \neg status[8] + i_break \cdot \neg status[9] + \\
&teq_taken \cdot \neg status[10])
\end{aligned}$$

```

cause[4] = 0
cause[3] = i_syscall + i_break + teq_taken
cause[2] = teq_taken
cause[1] = 0
cause[0] = i_break + teq_taken

```

CP0 数据连接按实验指导书固定：CP0.pc = NPC = PC+4，CP0.Rd = IMEM[15:11]，CP0.wdata = Rt。CP0 仅在 exception=1 时锁存 pc 与 cause；BREAK、SYSCALL、满足条件且未被屏蔽的 TEQ 分别写入 cause=9、8、13。

异常入口的物理偏移为 0x4；由于本 CPU 的 PC 保存 Mars 逻辑地址，CP0.exc_addr 应输出 32'h0040_0004，顶层再减去 32'h0040_0000 映射到指令存储器物理地址 0x4。EPC 保存 PC+4。

实验不额外设置 EXL 位。exception 有效时 CP0 将 Status 左移 5 位，ERET 时恢复 Status，并通过 rdata 输出 EPC 送回 PC。

ALU 控制信号：

```

aluc[3] = i_lui + i_sltu + i_sltiu + i_slt + i_slti + i_sra + i_srav + i_srl + i_srlv + i_sll +
i_sllv
aluc[2] = i_and + i_andi + i_or + i_ori + i_xor + i_xori + i_nor + i_sra + i_srav + i_srl +
i_srlv + i_sll + i_sllv
aluc[1] = i_add + i_addi + i_lw + i_sw + i_lbu + i_lhu + i_lb + i_lh + i_sb + i_sh + i_sub
+ i_xor + i_xori + i_nor + i_sltu + i_sltiu + i_slt + i_slti + i_sll + i_sllv
aluc[0] = i_subu + i_beq + i_bne + i_bgez + i_teq + i_sub + i_or + i_ori + i_nor + i_slt +
i_slti + i_srl + i_srlv

```

(3) 根据逻辑表达式，完成控制部件设计。

```

`timescale 1ns / 1ps

////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 2026/07/31 09:41:06
// Design Name:
// Module Name: controller
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////////////////////////////////

// 单周期 54 条指令 CPU 的组合逻辑控制器。
// MUX 编码遵循数据通路，并保持 CPU31 原有选择值不变。
module controller(
    input i_add,
    input i_addu,
    input i_sub,
    input i_subu,
    input i_and,

```

```

input i_or,
input i_xor,
input i_nor,
input i_slt,
input i_sltu,
input i_sll,
input i_srl,
input i_sra,
input i_sllv,
input i_srlv,
input i_srav,
input i_jr,

input i_addi,
input i_addiu,
input i_andi,
input i_ori,
input i_xori,
input i_lw,
input i_sw,
input i_beq,
input i_bne,
input i_slti,
input i_sltiu,
input i_lui,

input i_j,
input i_jal,

input i_div,
input i_divu,
input i_mult,
input i_multu,
input i_bgez,
input i_jalr,
input i_lbu,
input i_lhu,
input i_lb,
input i_lh,
input i_sb,
input i_sh,
input i_break,
input i_syscall,
input i_eret,
input i_mfhi,
input i_mflo,
input i_mthi,
input i_mtlo,
input i_mfc0,
input i_mtc0,
input i_clz,
input i_teq,

input alu_zero,
input alu_negative,
input [31:0] status,

output M0,
output M1,
output M2,
output [1:0] M3,
output [1:0] M4,
output M5,
output [1:0] M6,
output [1:0] M7,
output M8,
output [2:0] M9,
output [1:0] M10,
output [1:0] M11,
output [1:0] M12,
output [1:0] M13,

output rf_w,

output dm_ena,

```

```

output dm_r,
output dm_w,

output HI_w,
output LO_w,
output div_signed,
output mul_signed,

output mfc0,
output mtc0,
output exception,
output eret,
output [4:0] cause,

output [3:0] aluc
);

wire teq_taken;

// MUX0: 选择 PC+4 或分支目标地址。
assign M0 = (i_beq & alu_zero) |
            (i_bne & ~alu_zero) |
            (i_bgez & ~alu_negative);

// MUX1: 选择 Rs 或零扩展后的移位量。
assign M1 = i_sll | i_srl | i_sra |
            i_sllv | i_srlv | i_srav;

// MUX2: 选择 Rs[4:0]或 sa。
assign M2 = i_sll | i_srl | i_sra;

// MUX3: 00 为 MUX8, 01 为 Rs, 10 为 CP0 异常入口, 11 为 CP0 读出的 EPC。
// 已受理的异常优先于其他所有 PC 来源。
assign M3[1] = exception | i_eret;
assign M3[0] = ~exception & (i_jr | i_jalr | i_eret);

// MUX4: 00 为 Rt, 01 为立即数, 10 为零。
assign M4[1] = i_bgez;
assign M4[0] = i_addi | i_addiu | i_andi | i_ori | i_xori |
            i_lw | i_sw | i_slti | i_sltiu | i_lui |
            i_lbu | i_lhu | i_lb | i_lh | i_sb | i_sh;

// MUX5: 选择 16 位有符号扩展或无符号扩展。
assign M5 = i_andi | i_ori | i_xori | i_lui;

// MUX6: 00 为 ALU 结果, 01 为字装载结果, 10 为字节或半字装载结果。
assign M6[1] = i_lbu | i_lhu | i_lb | i_lh;
assign M6[0] = i_lw;

// MUX7: 00 为 Z, 01 为~Z, 10 为~N。
assign M7[1] = i_bgez;
assign M7[0] = i_bne;

// MUX8: 选择普通分支路径或跳转目标地址。
assign M8 = i_j | i_jal;

// MUX9: 000 为 MUX6, 001 为链接地址, 010 为 CLZ 结果, 011 为 HI,
// 100 为 LO, 101 为 CP0 读出数据。
assign M9[2] = i_mflo | i_mfc0;
assign M9[1] = i_clz | i_mfhi;
assign M9[0] = i_jal | i_jalr | i_mfhi | i_mfc0;

// MUX10: 00 为 Rs, 01 为 DIV 余数, 10 为 MUL 高 32 位。
assign M10[1] = i_mult | i_multu;
assign M10[0] = i_div | i_divu;

// MUX11: 00 为 Rs, 01 为 DIV 商, 10 为 MUL 低 32 位。
assign M11[1] = i_mult | i_multu;
assign M11[0] = i_div | i_divu;

// MUX12: 00 为 LB 结果, 01 为 LBU 结果, 10 为 LH 结果, 11 为 LHU 结果。
assign M12[1] = i_lh | i_lhu;
assign M12[0] = i_lbu | i_lhu;

// MUX13: 00 为 SW 写入数据, 01 为 SB 写入数据, 10 为 SH 写入数据。

```

```

assign M13[1] = i_sh;
assign M13[0] = i_sb;

assign rf_w = i_add | i_addu | i_sub | i_subu |
              i_and | i_or | i_xor | i_nor |
              i_slt | i_sltu |
              i_sll | i_srl | i_sra |
              i_sllv | i_srlv | i_srav |
              i_addi | i_addiu |
              i_andi | i_ori | i_xori |
              i_lw | i_slti | i_sltiu | i_lui | i_jal |
              i_jalr | i_lbu | i_lhu | i_lb | i_lh |
              i_mfhi | i_mflo | i_mfc0 | i_clz;

assign dm_r = i_lw | i_lbu | i_lhu | i_lb | i_lh;
assign dm_w = i_sw | i_sb | i_sh;
assign dm_ena = dm_r | dm_w;

assign HI_w = i_div | i_divu | i_mult | i_multu | i_mthi;
assign LO_w = i_div | i_divu | i_mult | i_multu | i_mtlo;
assign div_signed = i_div;
assign mul_signed = i_mult;

assign mfc0 = i_mfc0;
assign mtc0 = i_mtc0;
assign eret = i_eret;

// 本实验中, Status[0]为全局使能位。
// Status[8]、Status[9]或Status[10]置位时, 分别屏蔽 syscall、break 或 teq 异常。
assign teq_taken = i_teq & alu_zero;
assign exception = status[0] &
                  ((i_syscall & ~status[8]) |
                   (i_break & ~status[9]) |
                   (teq_taken & ~status[10]));

// 异常编码: syscall 为 8, break 为 9, teq 为 13。
assign cause[4] = 1'b0;
assign cause[3] = i_syscall | i_break | teq_taken;
assign cause[2] = teq_taken;
assign cause[1] = 1'b0;
assign cause[0] = i_break | teq_taken;

assign aluc[3] = i_lui |
                 i_sltu | i_sltiu |
                 i_slt | i_slti |
                 i_sra | i_srav |
                 i_srl | i_srlv |
                 i_sll | i_sllv;

assign aluc[2] = i_and | i_andi |
                 i_or | i_ori |
                 i_xor | i_xori |
                 i_nor |
                 i_sra | i_srav |
                 i_srl | i_srlv |
                 i_sll | i_sllv;

assign aluc[1] = i_add | i_addi |
                 i_lw | i_sw |
                 i_lbu | i_lhu | i_lb | i_lh | i_sb | i_sh |
                 i_sub |
                 i_xor | i_xori |
                 i_nor |
                 i_sltu | i_sltiu |
                 i_slt | i_slti |
                 i_sll | i_sllv;

assign aluc[0] = i_subu |
                 i_beq | i_bne | i_bgez | i_teq |
                 i_sub |
                 i_or | i_ori |
                 i_nor |
                 i_slt | i_slti |
                 i_srl | i_srlv;

```

endmodule

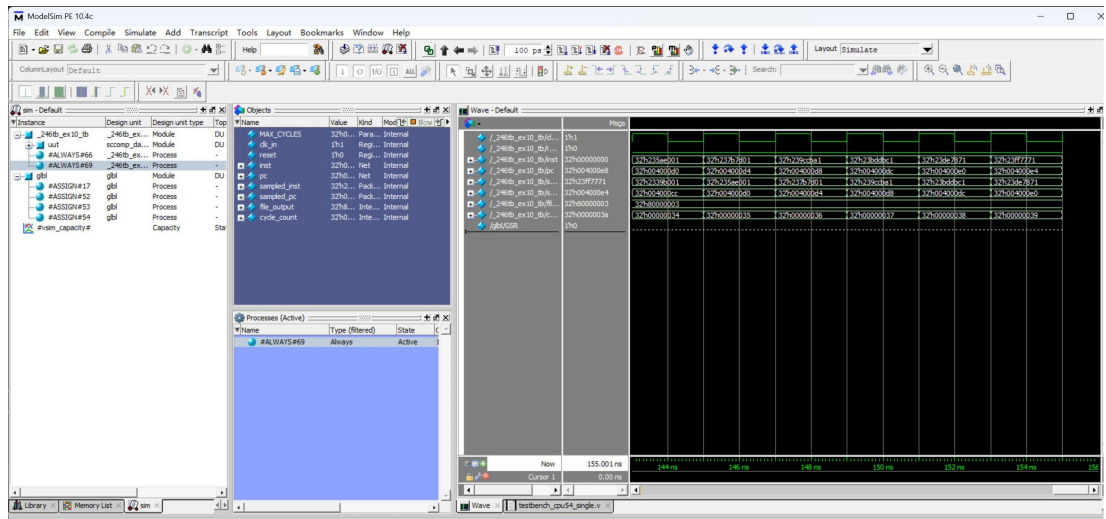
四、CPU 前仿真测试结果

报告要求：

- (1) 对 CPU 前仿真的实验内容和结果进行描述，并给出 modelsim 前仿真波形图的贴图。
- (2) 对单条指令、网站前仿真所用测试代码 code 文件等测试的结果进行比对。

前仿真以 sccomp_dataflow 为被测顶层模块，通过 tb 产生模拟时钟和复位信号，并将各单条指令测试程序及 CP0 测试程序载入 IMEM。仿真过程中观察 PC、指令、通用寄存器、数据存储器、HI/LO 及 CP0 等关键部件的状态变化，同时将寄存器和存储器结果输出到文本文件。经 txt_compare 与标准结果逐项比较，各测试结果均一致，说明 CPU54 的数据通路、控制器以及 54 条指令的功能实现正确。

1.addi

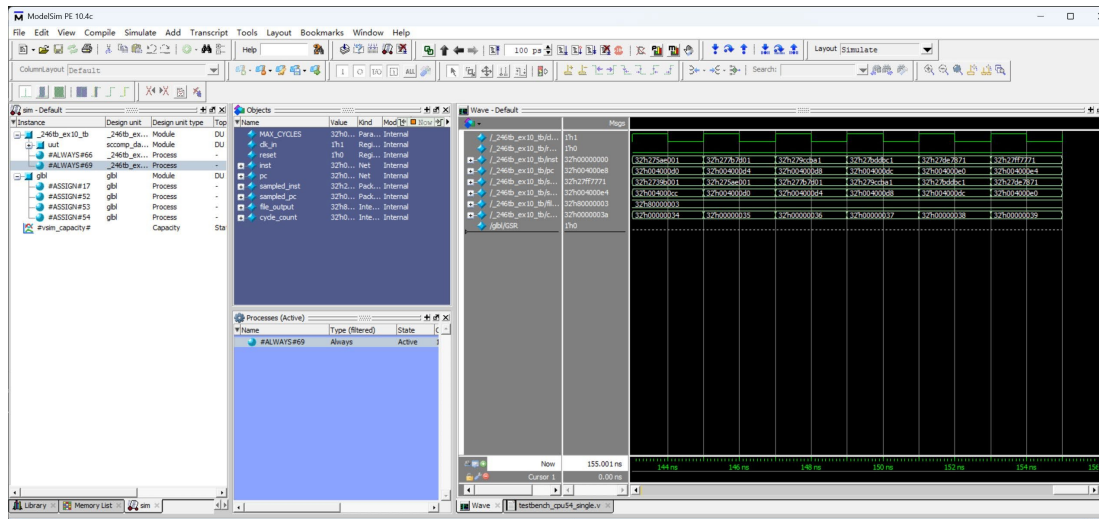


```
执行命令: txt_compare -file1 "D:\CPU54_test\test1_addi_result.txt"
           -file2 "D:\CPU54_test\output\my_1_addi_result.txt"
           --display detailed

比较结果输出:
=====
在指定检查条件下完全一致。
=====

请按任意键继续...
```

2.addiu



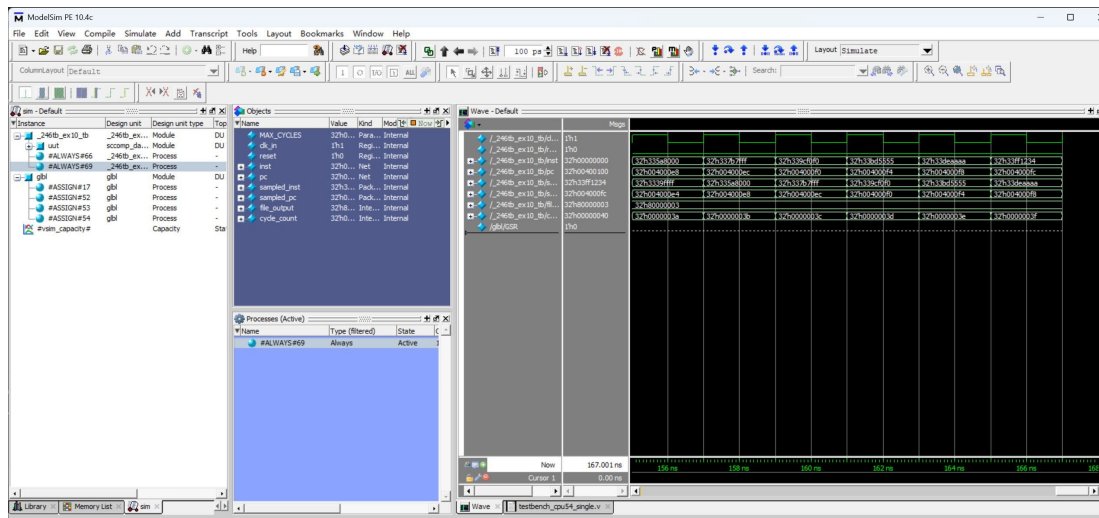
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\2_addiu.result.txt"
--file2 "D:\CPU54_test\output\my_2_addiu_result.txt"
--display detailed
```

比较结果输出:

在指定检查条件下完全一致。

请任意键继续...

3.andi



```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\3_andi.result.txt"
--file2 "D:\CPU54_test\output\my_3_andi_result.txt"
--display detailed
```

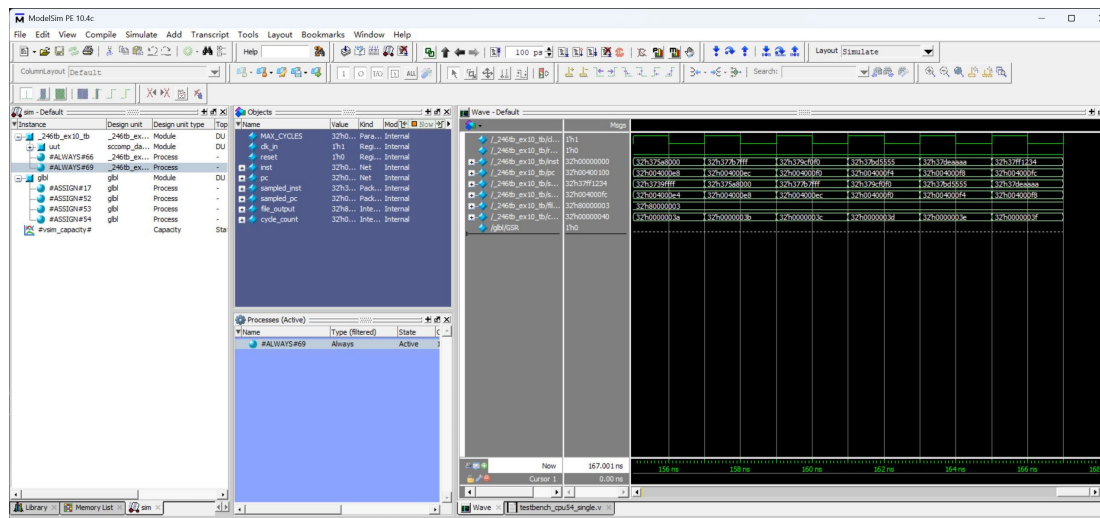
比较结果输出:

在指定检查条件下完全一致。

=====

请按任意键继续...

4.ori



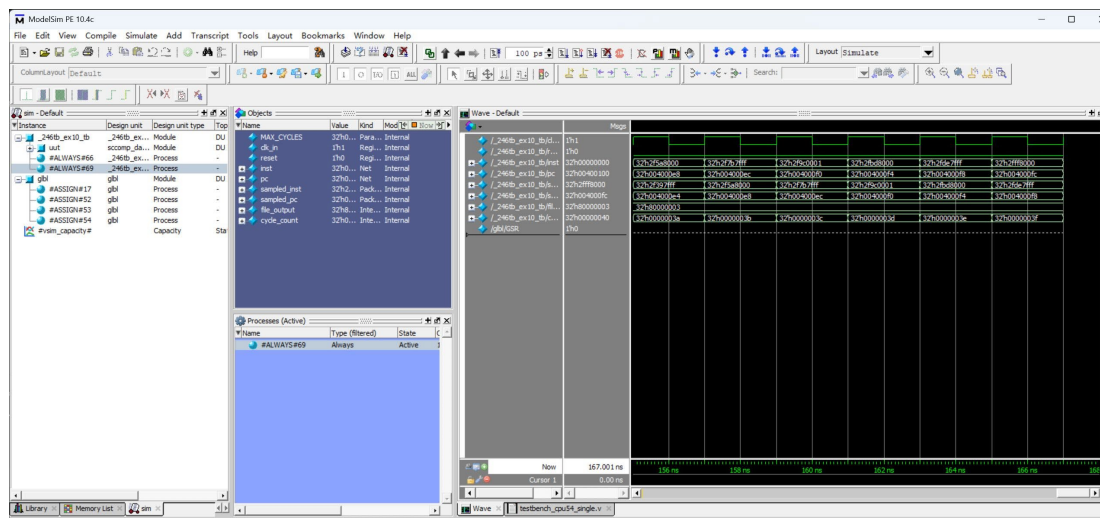
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\4_ori.result.txt"
--file2 "D:\CPU54_test\output\my_4_ori_result.txt"
--display detailed
```

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续...

5.sltiu



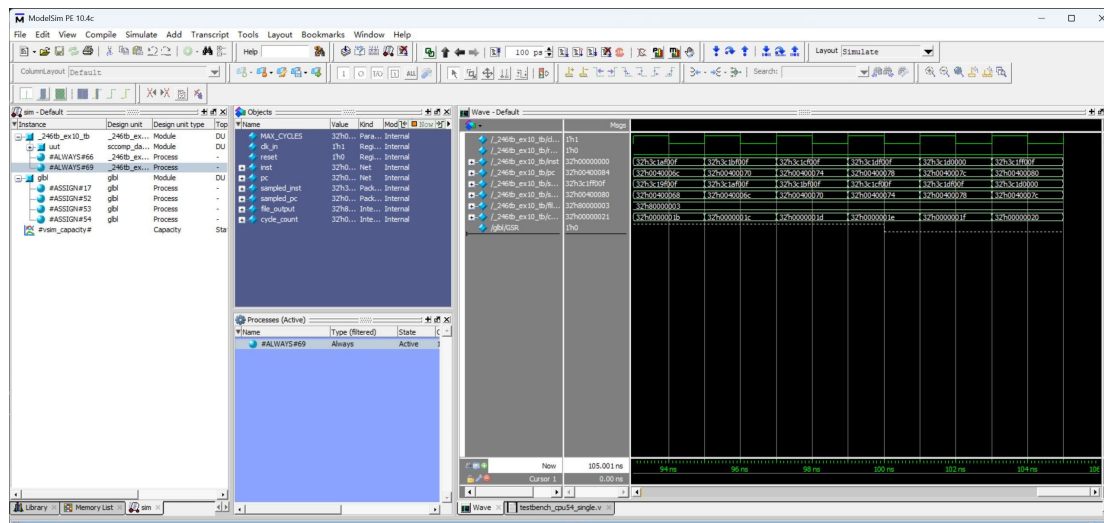
```
执行命令: txt_compare --file1 "D:\CPU54_test\bt\5_sltiu.result.txt"
--file2 "D:\CPU54_test\output\my_5_sltiu_result.txt"
--display detailed
```

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续...

6.lui

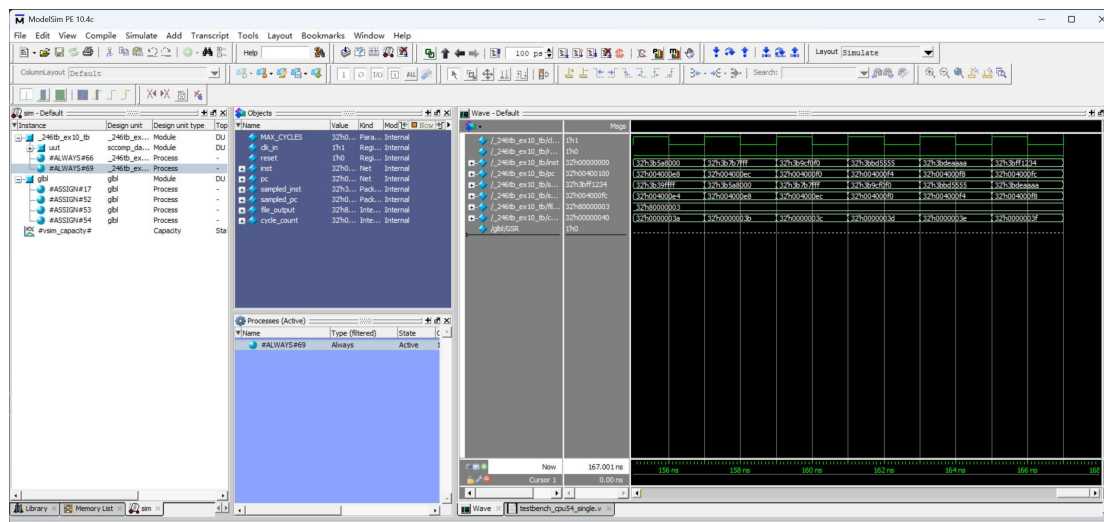


```
执行命令: txt_compare -file1 "D:\CPU54_test\bn6_lui_result.txt"
           -file2 "D:\CPU54_test\output\my_6_lui_result.txt"
           -display detailed

比较结果输出:
=====
在指定检查条件下完全一致
=====

请按任意键继续...
```

7.xori

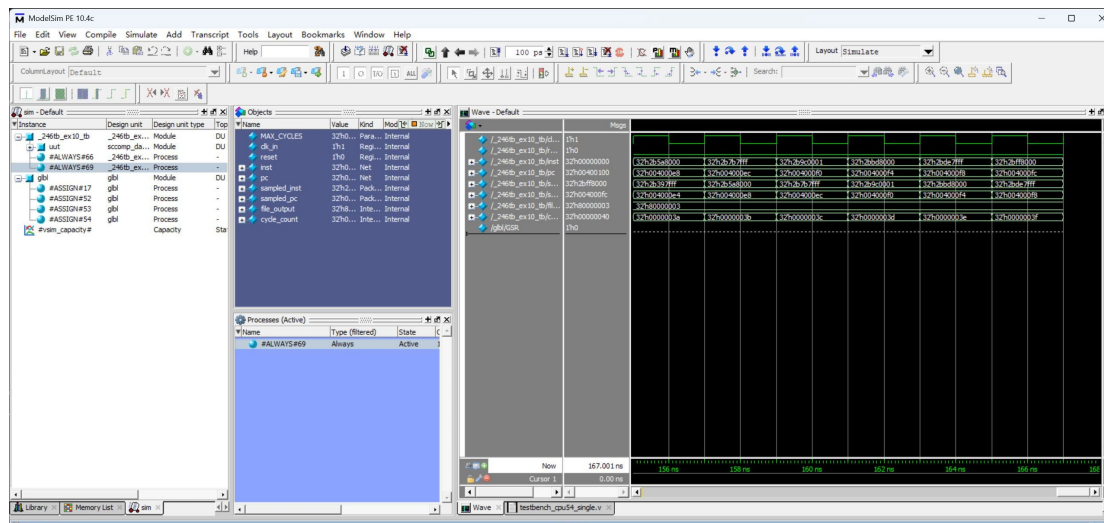


```
执行命令: txt_compare -file1 "D:\CPU54_test\bn7_xori_result.txt"
           -file2 "D:\CPU54_test\output\my_7_xori_result.txt"
           -display detailed

比较结果输出:
=====
在指定检查条件下完全一致
=====

请按任意键继续...
```

8.slti

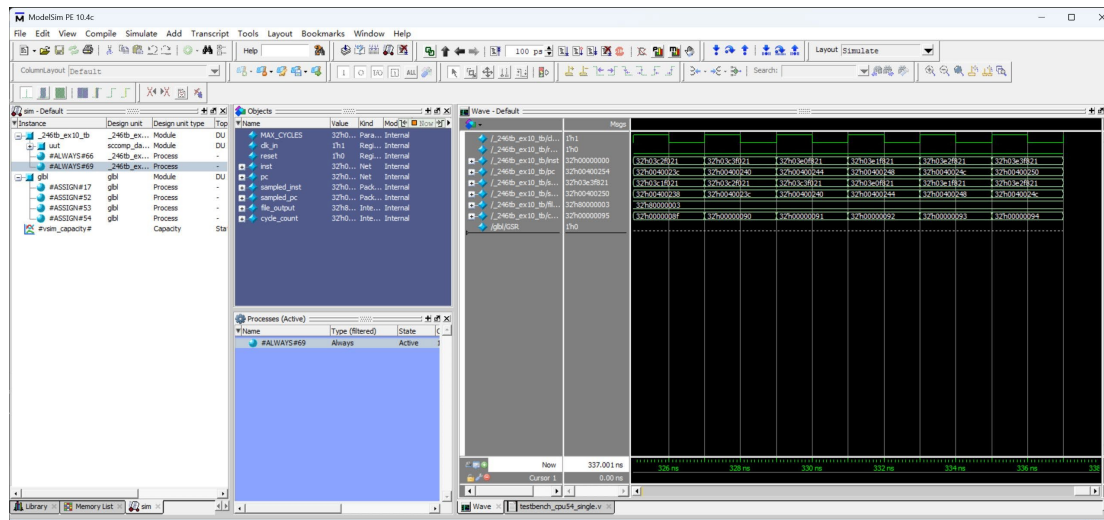


```
执行命令: txt_compare -file1 "D:\CPU54_test\test\8_slti_result.txt"
           -file2 "D:\CPU54_test\output\my_8_slti_result.txt"
           --display detailed

比较结果输出:
=====
在指定检查条件下完全一致
=====

请按任意键继续...
```

9.addu

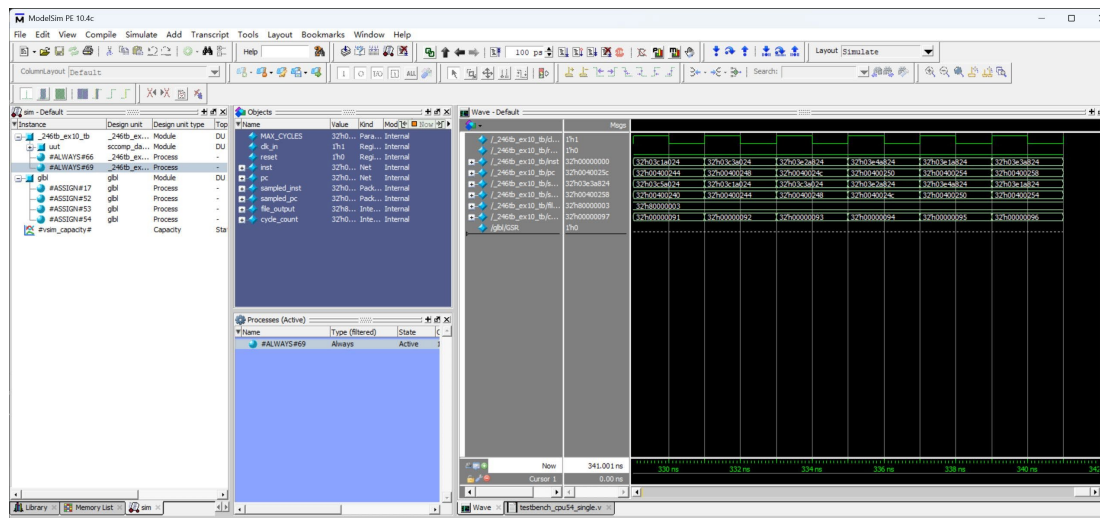


```
执行命令: txt_compare -file1 "D:\CPU54_test\test\9_addu_result.txt"
           -file2 "D:\CPU54_test\output\my_9_addu_result.txt"
           --display detailed

比较结果输出:
=====
在指定检查条件下完全一致
=====

请按任意键继续...
```

10.and



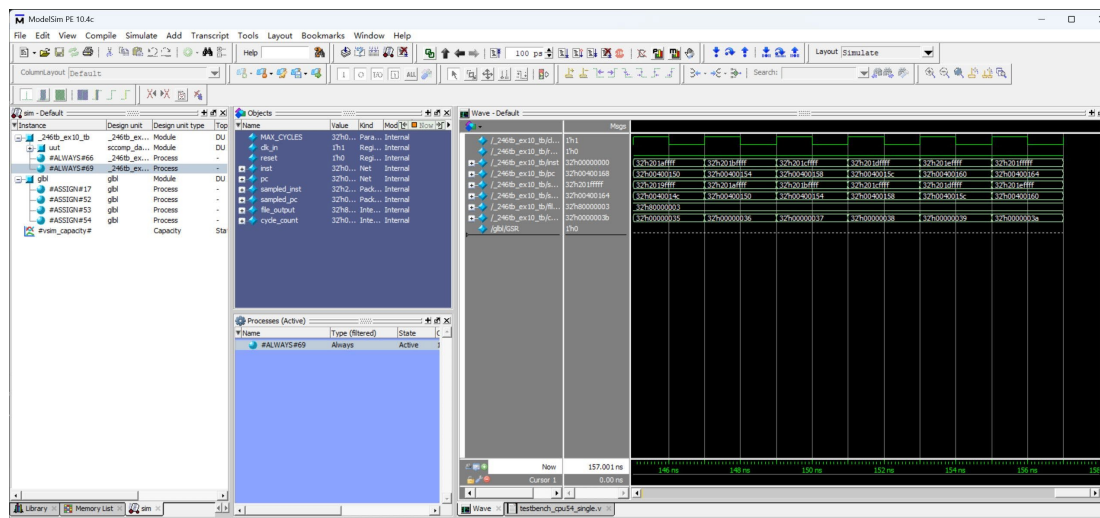
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\10_and_result.txt"
--file2 "D:\CPU54_test\output\my_10_and_result.txt"
--display detailed
```

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续...

11.beq



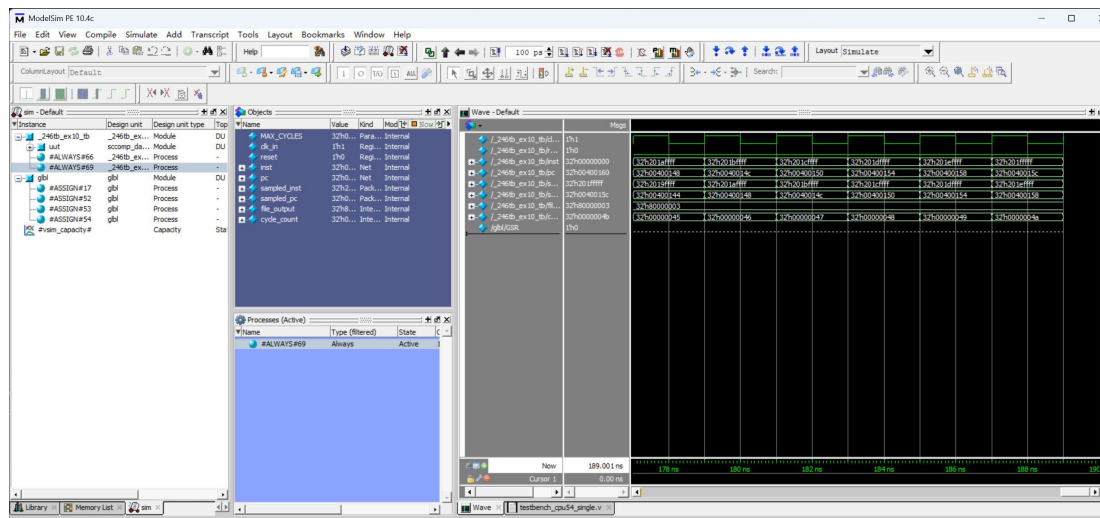
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\11_beq.result.txt"
--file2 "D:\CPU54_test\output\my_11_beq_result.txt"
--display detailed
```

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续...

12.bne



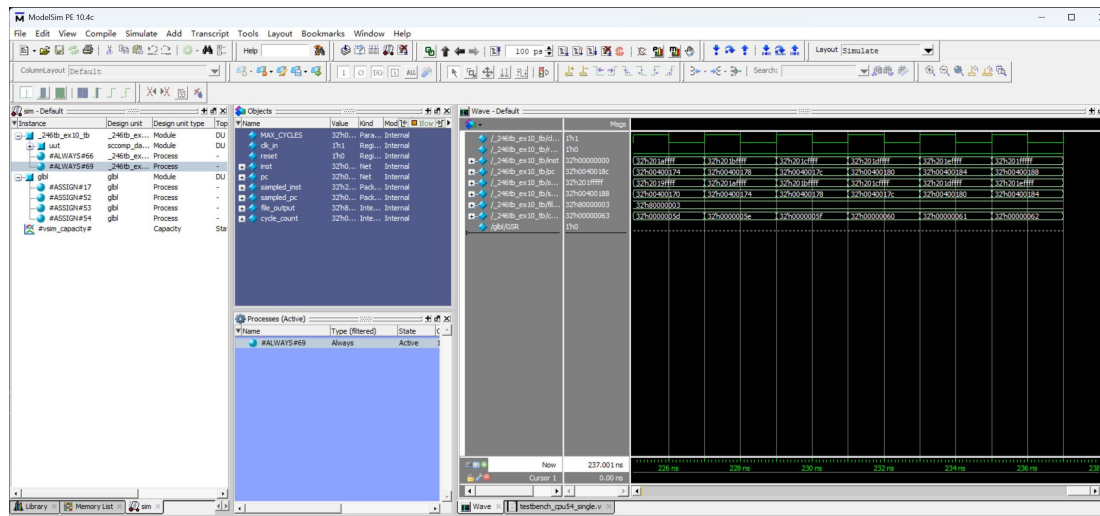
```
执行命令: txt_compare --file1 "D:\CPU54_test\bt\12_bne.result.txt"
--file2 "D:\CPU54_test\output\my_12_bne_result.txt"
--display detailed
```

比较结果输出:

在指定檢查條件下完全一致。

请按任意键继续...

13.j



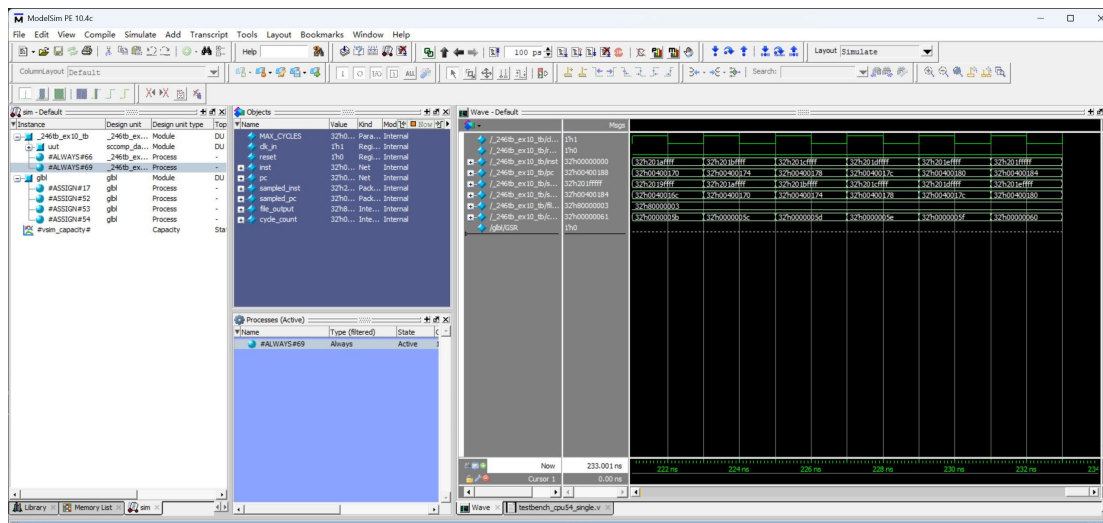
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\13_j_result.txt"  
--file2 "D:\CPU54_test\output\my_13_j_result.txt"  
--display detailed
```

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续...

14.jal



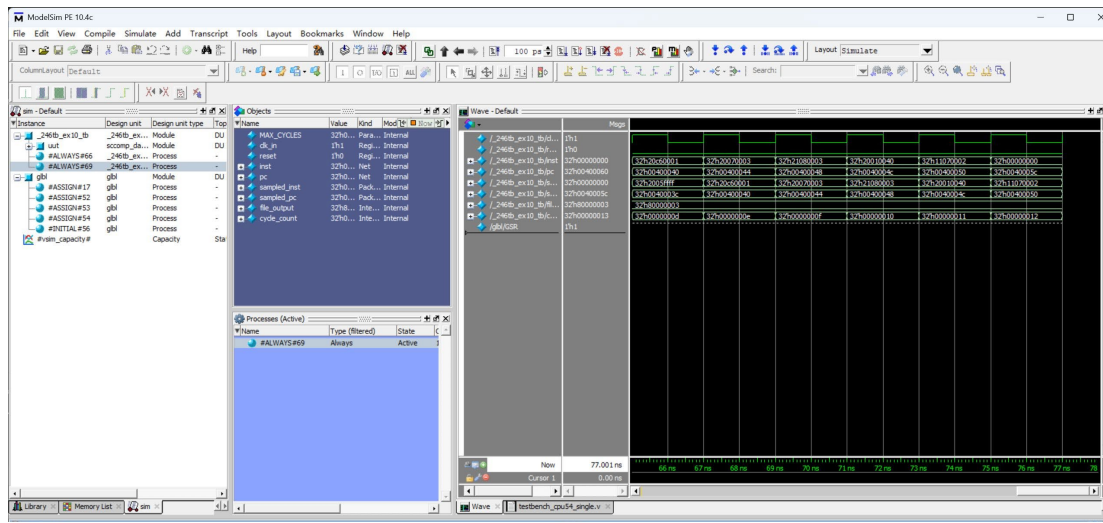
```
执行命令: txt_compare -file1 "D:\CPU54_test\14.jal_result.txt"
           -file2 "D:\CPU54_test\output\my_14.jal_result.txt"
           -display detailed
```

比较结果输出:

在指定检查条件下完全一致

请按任意键继续...

15.jr



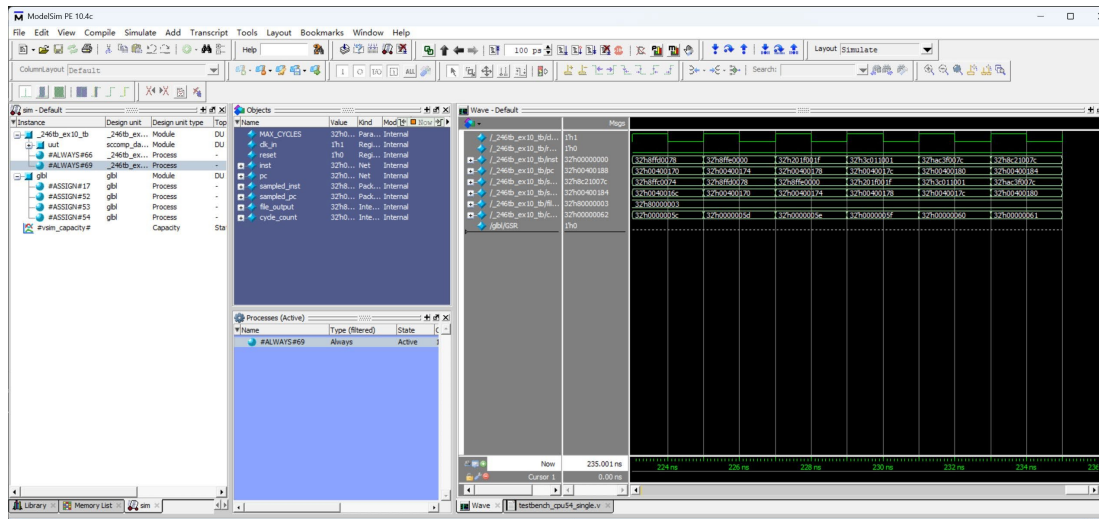
```
执行命令: txt_compare -file1 "D:\CPU54_test\15.jr_result.txt"
           -file2 "D:\CPU54_test\output\my_15.jr_result.txt"
           -display detailed
```

比较结果输出:

在指定检查条件下完全一致

请按任意键继续...

16.26.lwsw



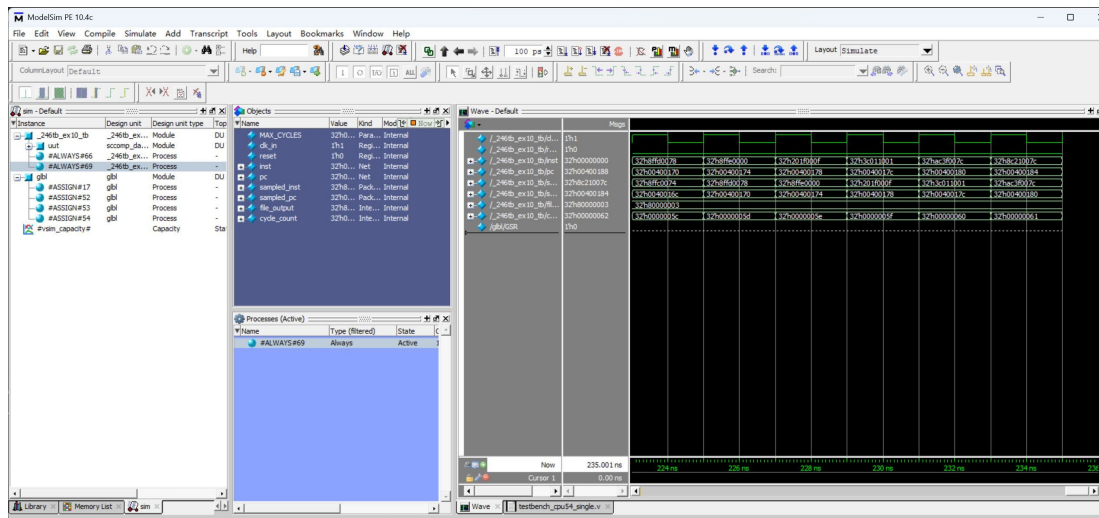
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\16.26_hsws.result.txt"
--file2 "D:\CPU54_test\output\my_16.26_hsws_result.txt"
--display detailed
```

比较结果输出:

在指定檢查條件下完全一致。

请按任意键继续...

16.26.lwsw2



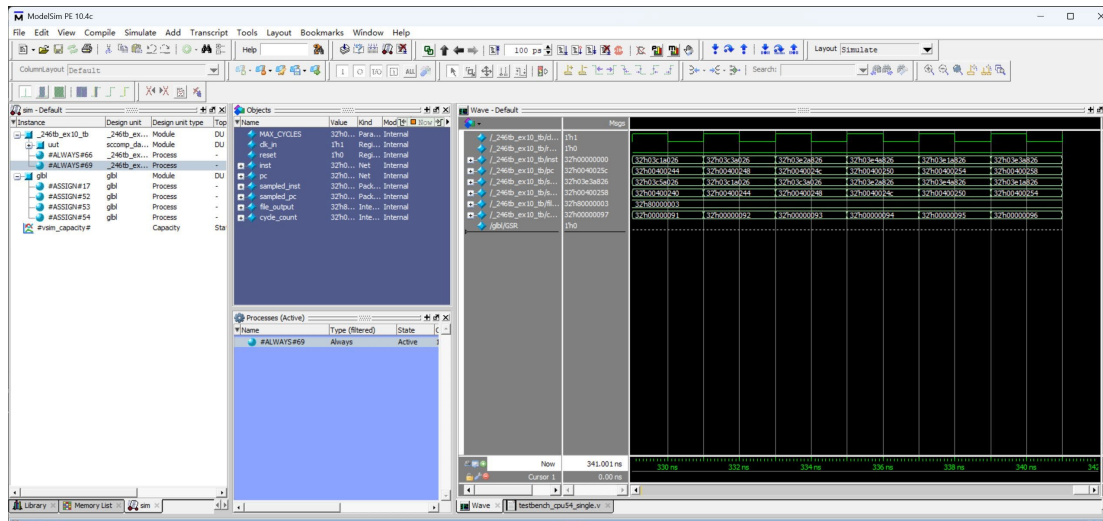
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\16.26_hws2.result.txt"
--file2 "D:\CPU54_test\output\my_16.26_hws2_result.txt"
--display detailed
```

比较结果输出:

在指定檢查條件下完全一致。

请按任意键继续...

17.xor



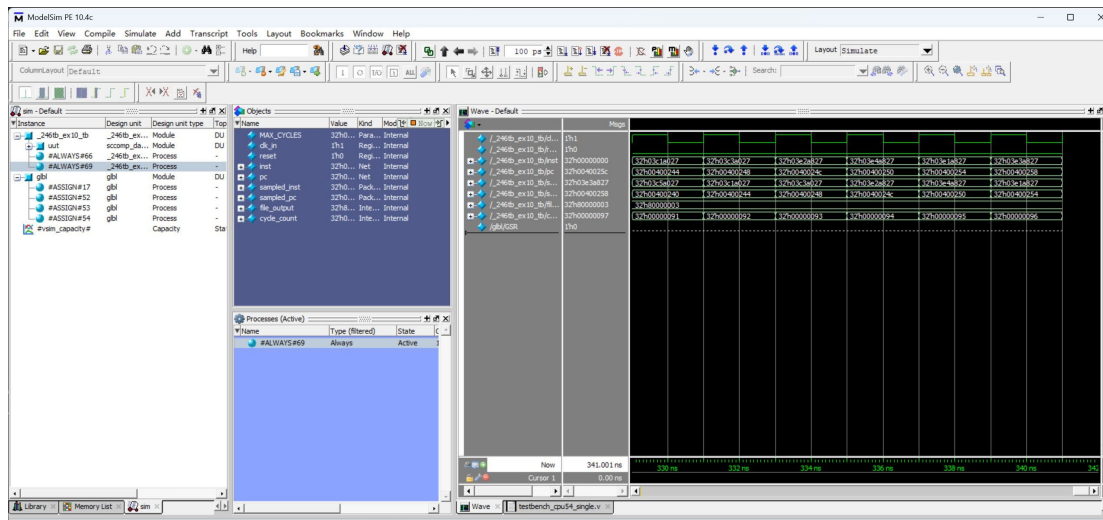
```
执行命令: btx_compare -file1 "D:\CPU54_test\bt17_xor.result.txt"
           -file2 "D:\CPU54_test\output\my_17_xor.result.txt"
           -display detailed
```

比较结果输出:

在指定检查条件下完全一致

请按任意键继续...

18.nor



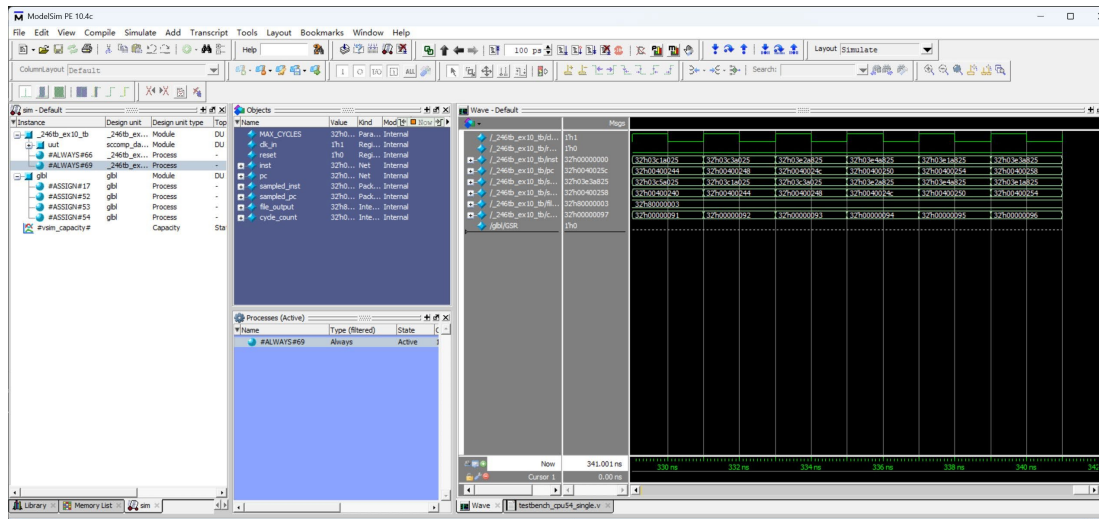
```
执行命令: btx_compare -file1 "D:\CPU54_test\bt18_nor.result.txt"
           -file2 "D:\CPU54_test\output\my_18_nor.result.txt"
           -display detailed
```

比较结果输出:

在指定检查条件下完全一致

请按任意键继续...

19.or



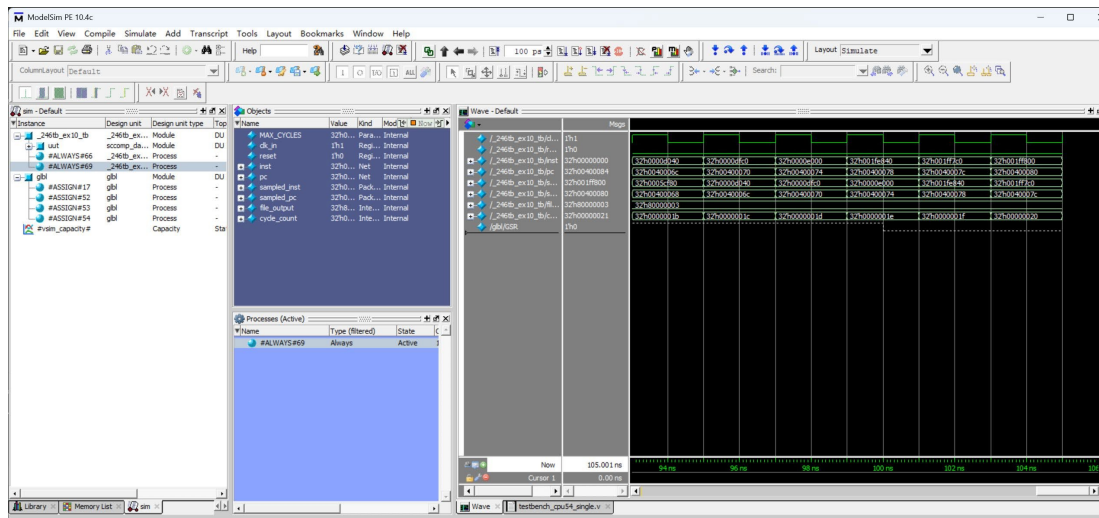
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\19_or_result.txt"
--file2 "D:\CPU54_test\output\my_19_or_result.txt"
--display detailed
```

比较结果输出:

在指定檢查條件下完全一致。

请按任意键继续...

20.sll



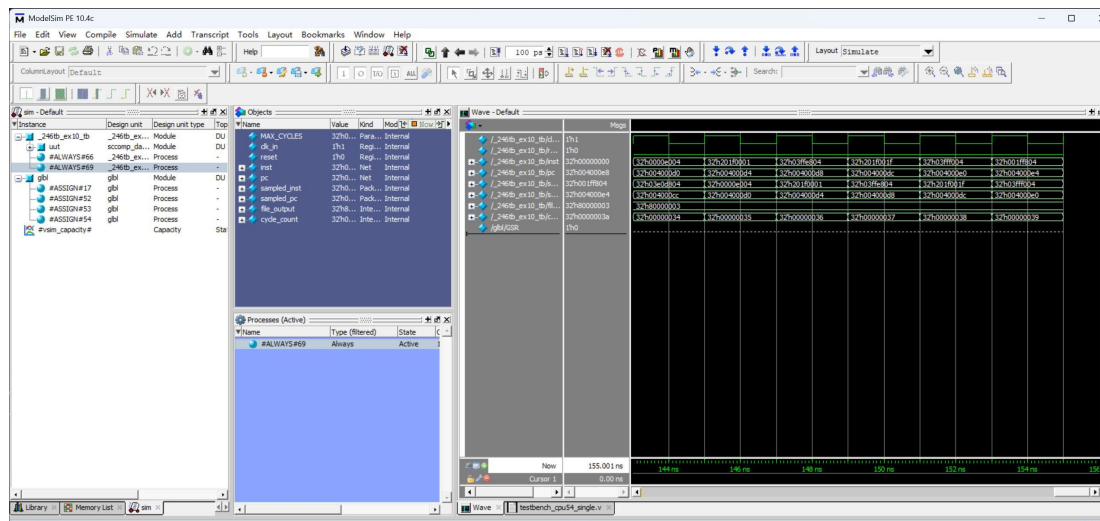
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\20_sll.result.txt"
--file2 "D:\CPU54_test\output\my_20_sll_result.txt"
--display detailed
```

比较结果输出:

在指定檢查條件下完全一致。

请按任意键继续...

21.sllv



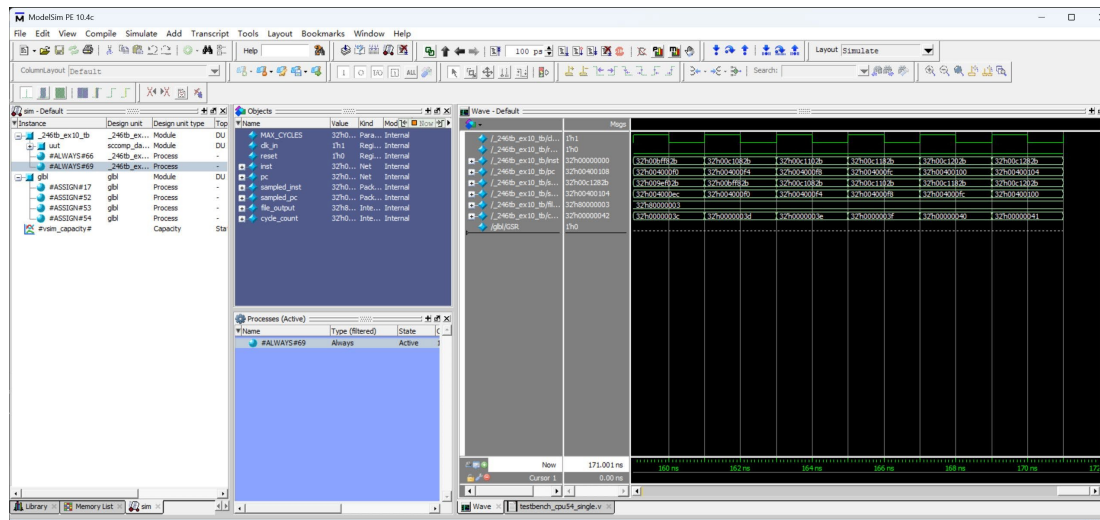
```
执行命令: txt_compare -file1 "D:\CPU54_test\txt21_sllv.result.txt"
           -file2 "D:\CPU54_test\output\my_21_sllv.result.txt"
           -display detailed
```

比较结果输出:

在指定检查条件下完全一致

请按任意键继续...

22.sltu



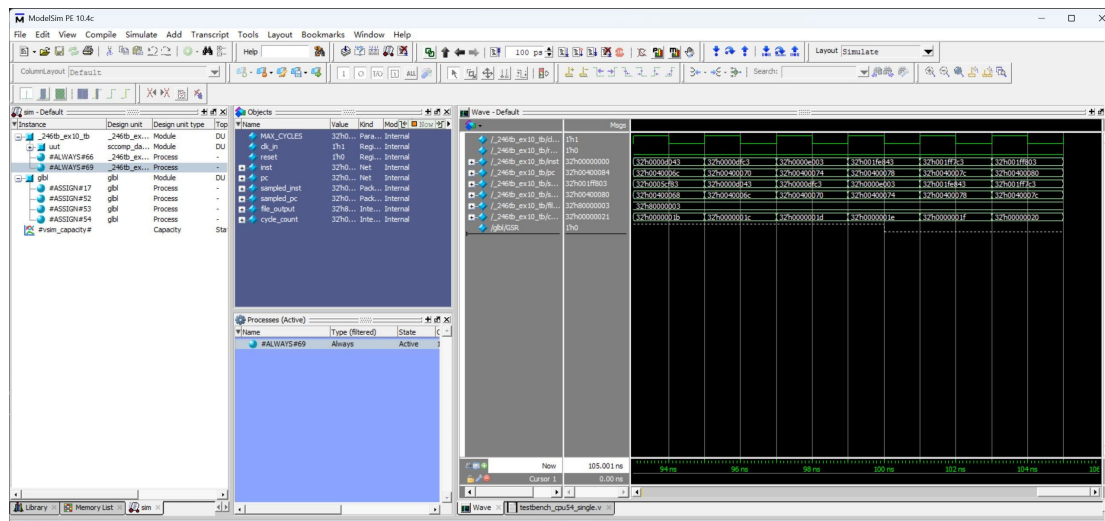
```
执行命令: txt_compare -file1 "D:\CPU54_test\txt22_sltu.result.txt"
           -file2 "D:\CPU54_test\output\my_22_sltu.result.txt"
           -display detailed
```

比较结果输出:

在指定检查条件下完全一致

请按任意键继续...

23.sra



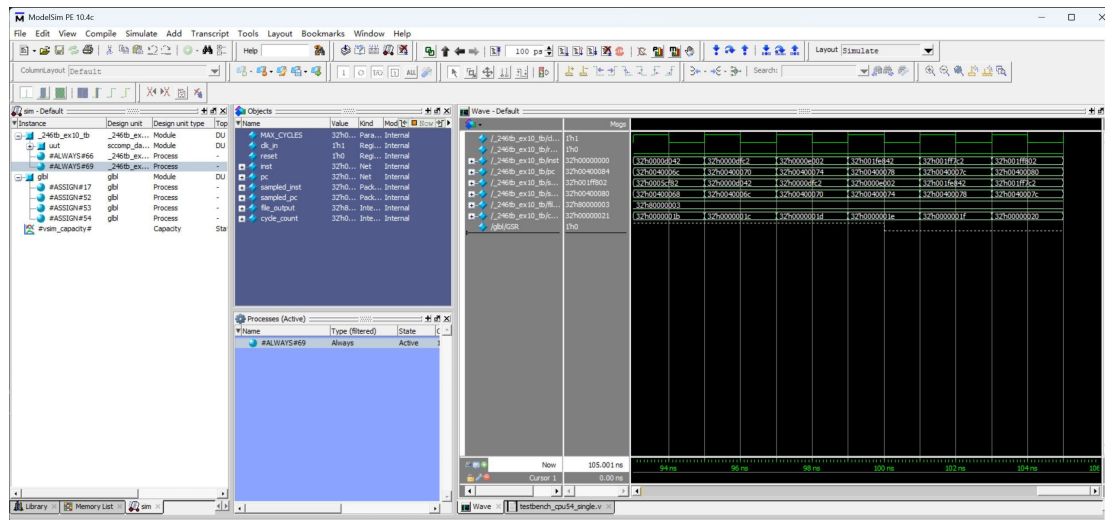
```
执行命令: btt_compare -file1 "D:\CPU54_test\btt23_sra.result.txt"
           -file2 "D:\CPU54_test\output\my_23_sra.result.txt"
           --display detailed
```

比较结果输出:

在指定检查条件下完全一致

请按任意键继续...

24.srl



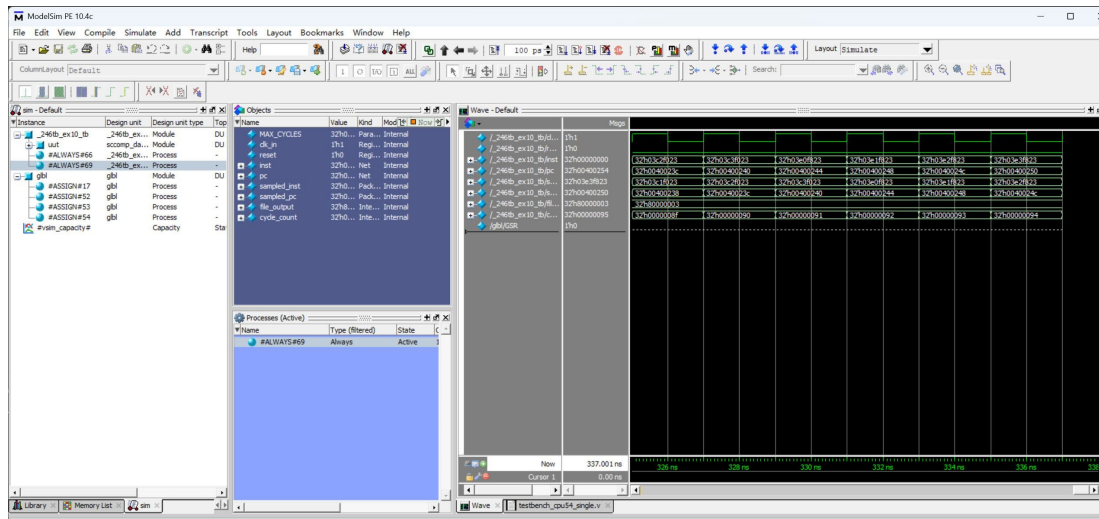
```
执行命令: btt_compare -file1 "D:\CPU54_test\btt24_srl.result.txt"
           -file2 "D:\CPU54_test\output\my_24_srl.result.txt"
           --display detailed
```

比较结果输出:

在指定检查条件下完全一致

请按任意键继续...

25.subu



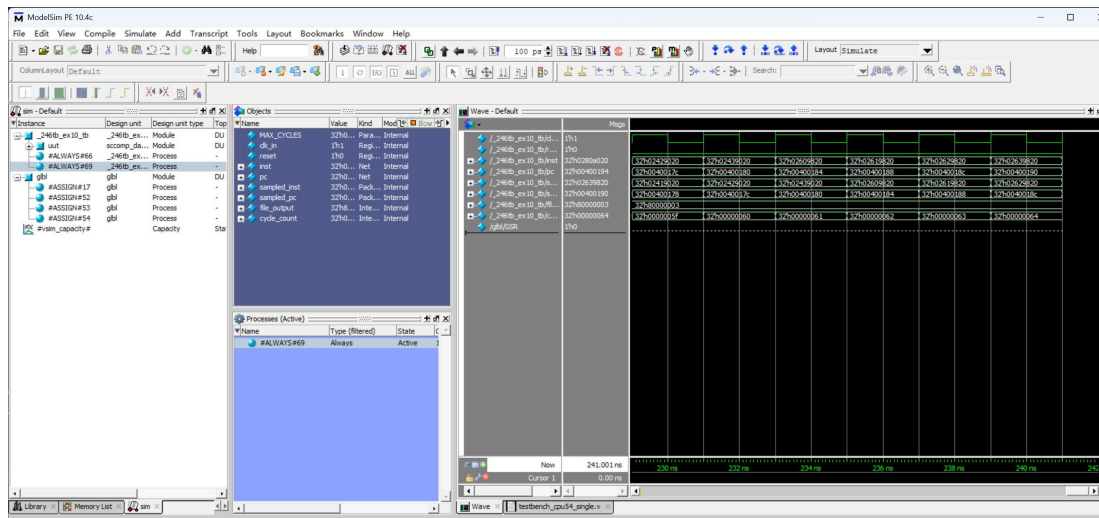
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\25_subu.result.txt"
--file2 "D:\CPU54_test\output\my_25_subu_result.txt"
--display detailed
```

比较结果输出:

在指定檢查條件下完全一致。

请按任意键继续...

27.add



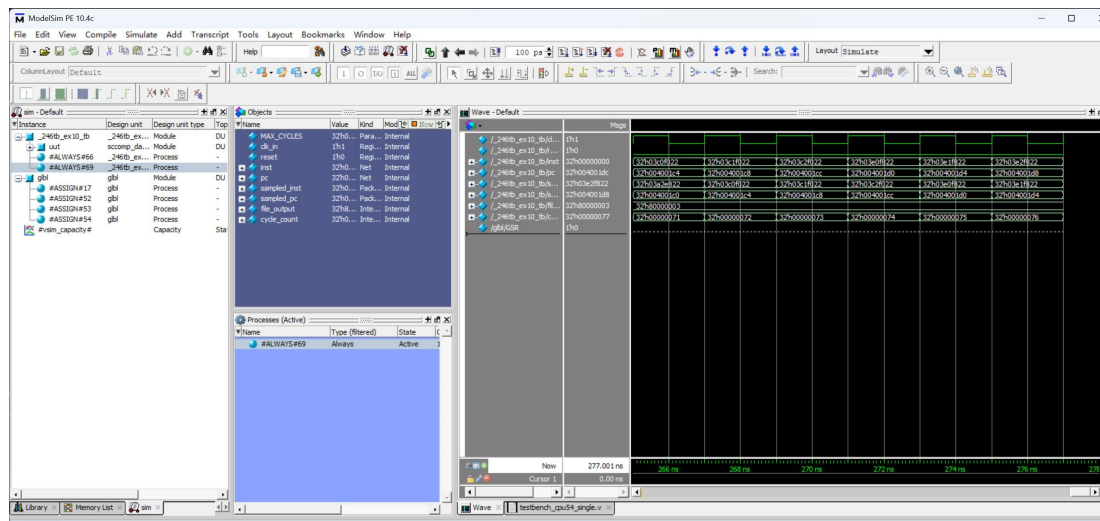
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\27_add.result.txt"
--file2 "D:\CPU54_test\output\my_27_add_result.txt"
--display detailed
```

比较结果输出:

在指定檢查條件下完全一致。

请按任意键继续...

28.sub



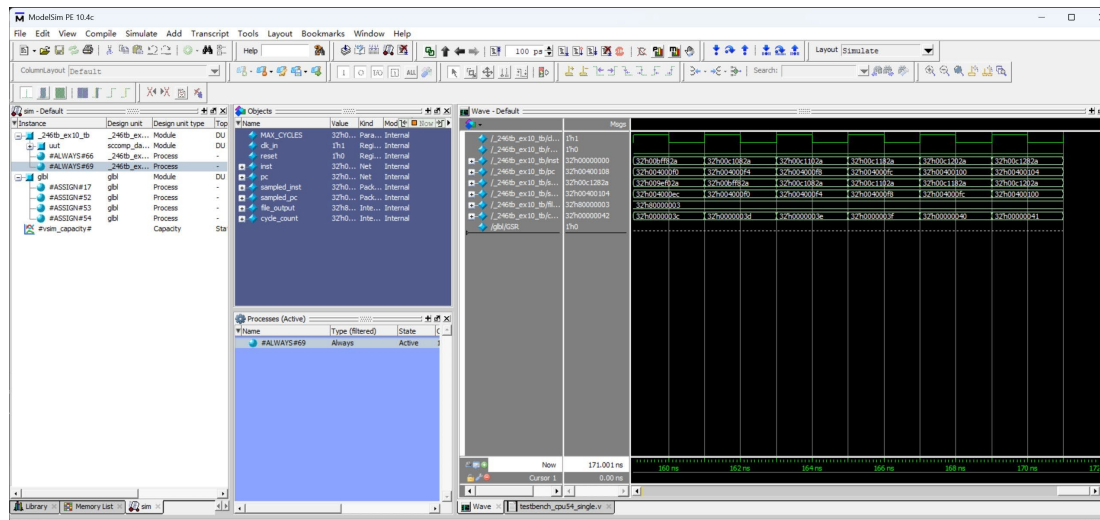
```
执行命令: btx_compare -file1 "D:\CPU54_test\bn28_sub.result.txt"
           -file2 "D:\CPU54_test\output\my_28_sub.result.txt"
           -display detailed
```

比较结果输出:

在指定检查条件下完全一致

请按任意键继续...

29.slt



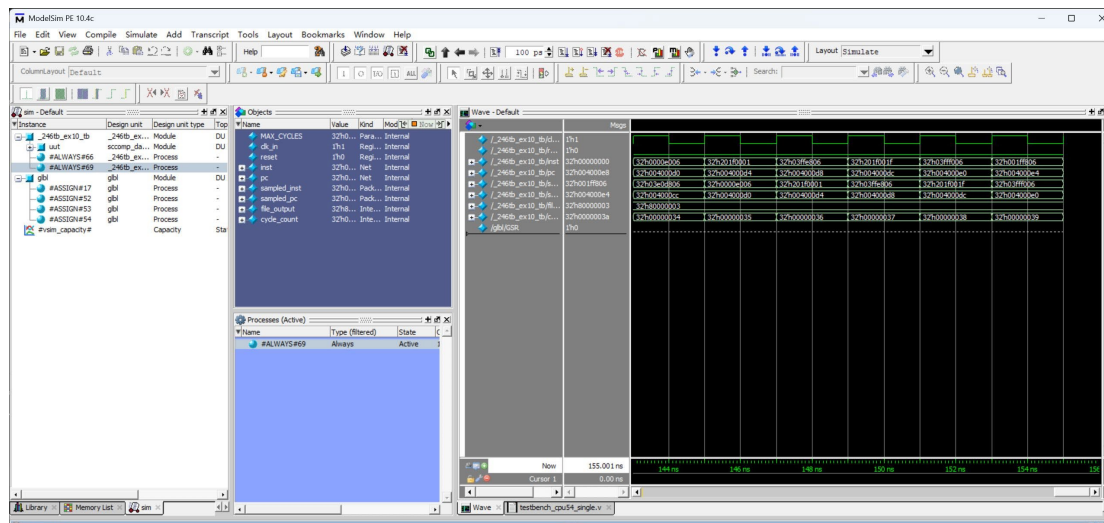
```
执行命令: btx_compare -file1 "D:\CPU54_test\bn29_slt.result.txt"
           -file2 "D:\CPU54_test\output\my_29_slt.result.txt"
           -display detailed
```

比较结果输出:

在指定检查条件下完全一致

请按任意键继续...

30.srlv

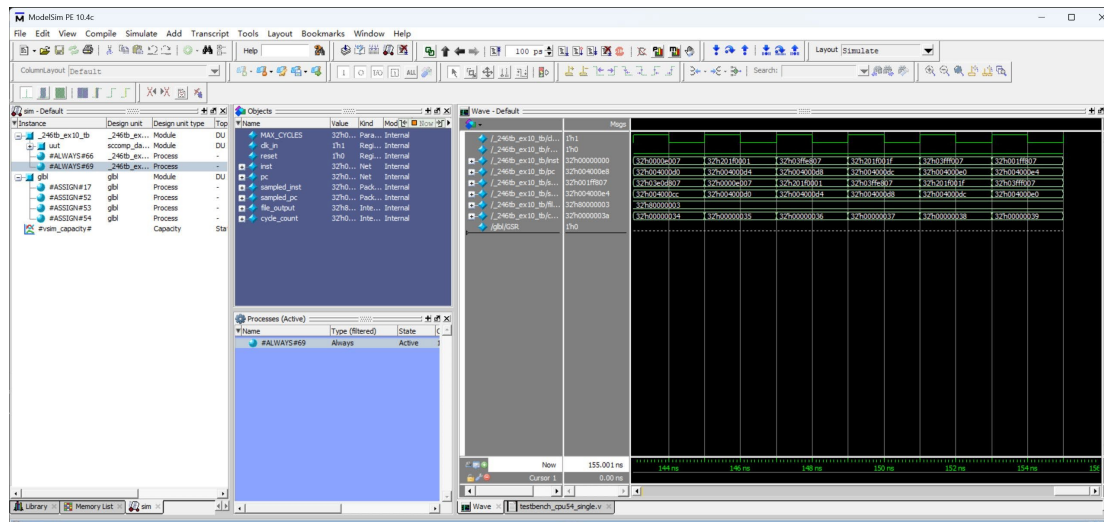


```
执行命令: btx_compare -file1 "D:\CPU54_test\bt30.srv.result.bt"
           -file2 "D:\CPU54_test\output\my_30.srv.result.bt"
           --display detailed

比较结果输出:
=====
在指定检查条件下完全一致
=====

请按任意键继续...
```

31.srvv

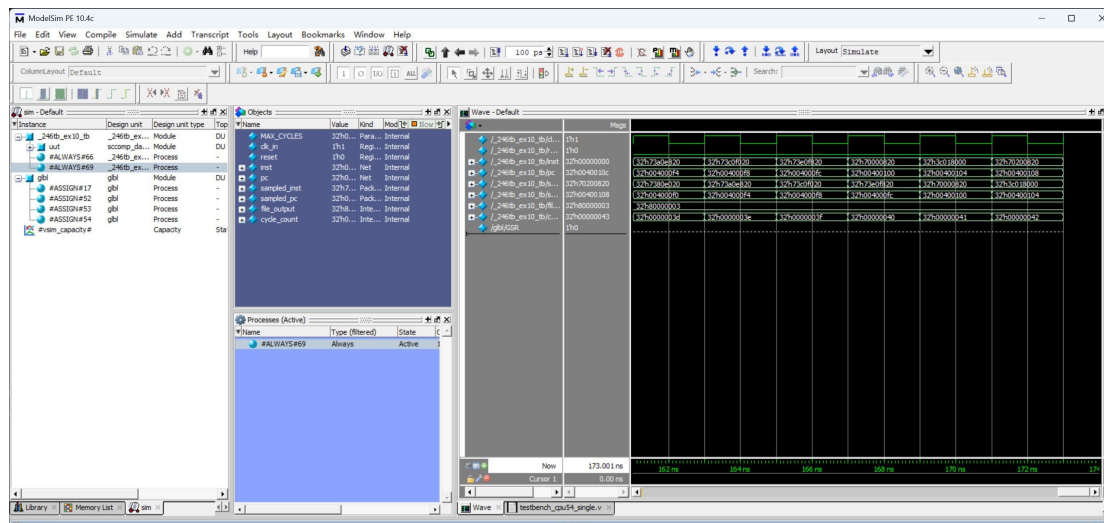


```
执行命令: btx_compare -file1 "D:\CPU54_test\bt31.srv.result.bt"
           -file2 "D:\CPU54_test\output\my_31.srv.result.bt"
           --display detailed

比较结果输出:
=====
在指定检查条件下完全一致
=====

请按任意键继续...
```

32.clz

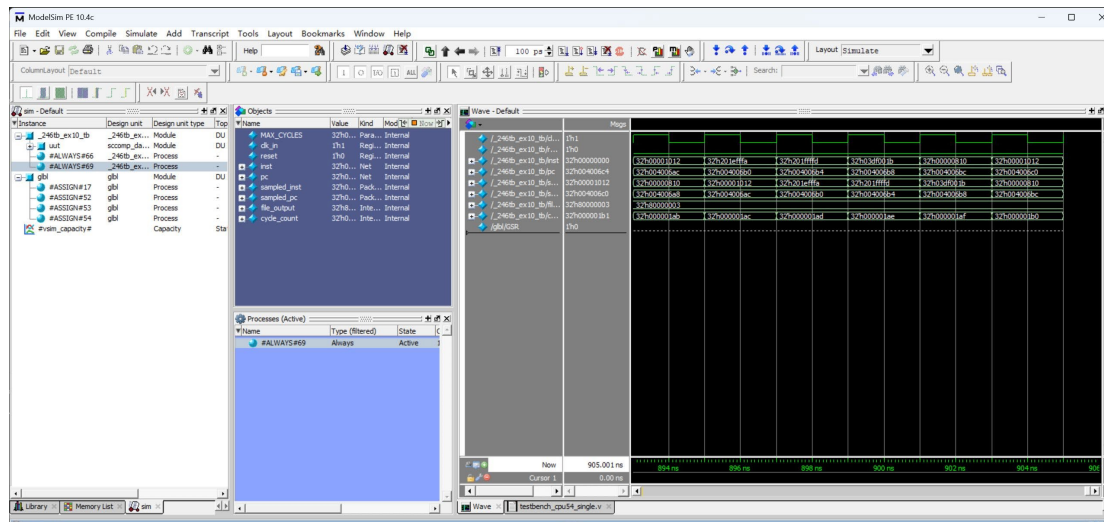


```
执行命令: btx_compare -file1 "D:\CPU54_test\bt32.clz.result.txt"
           -file2 "D:\CPU54_test\output\my_32_clz_result.txt"
           -display detailed

比较结果输出:
=====
在指定检查条件下完全一致
=====

请按任意键继续...
```

33.divu

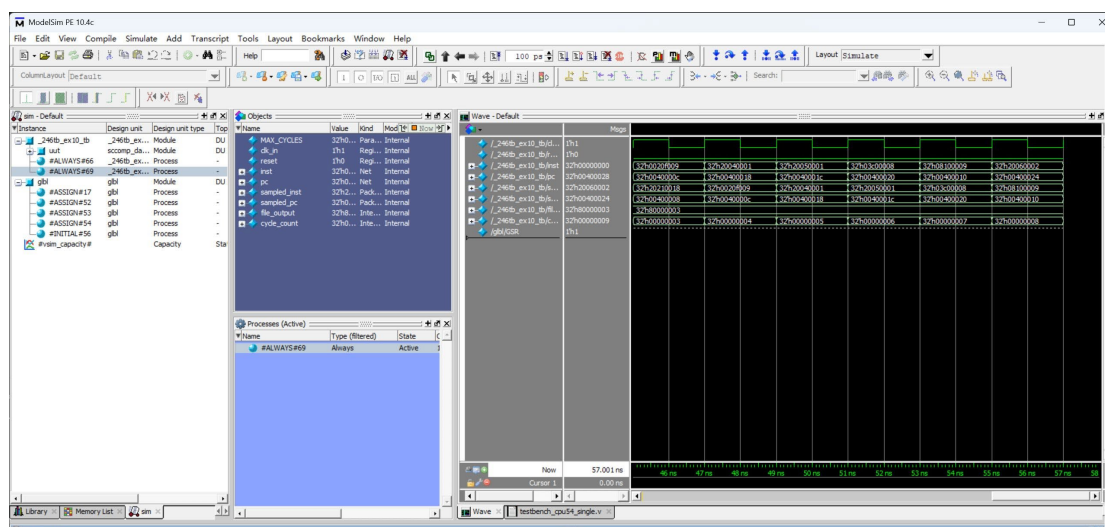


```
执行命令: btx_compare -file1 "D:\CPU54_test\bt33.divu.result.txt"
           -file2 "D:\CPU54_test\output\my_33_divu_result.txt"
           -display detailed

比较结果输出:
=====
在指定检查条件下完全一致
=====

请按任意键继续...
```

35.jalr

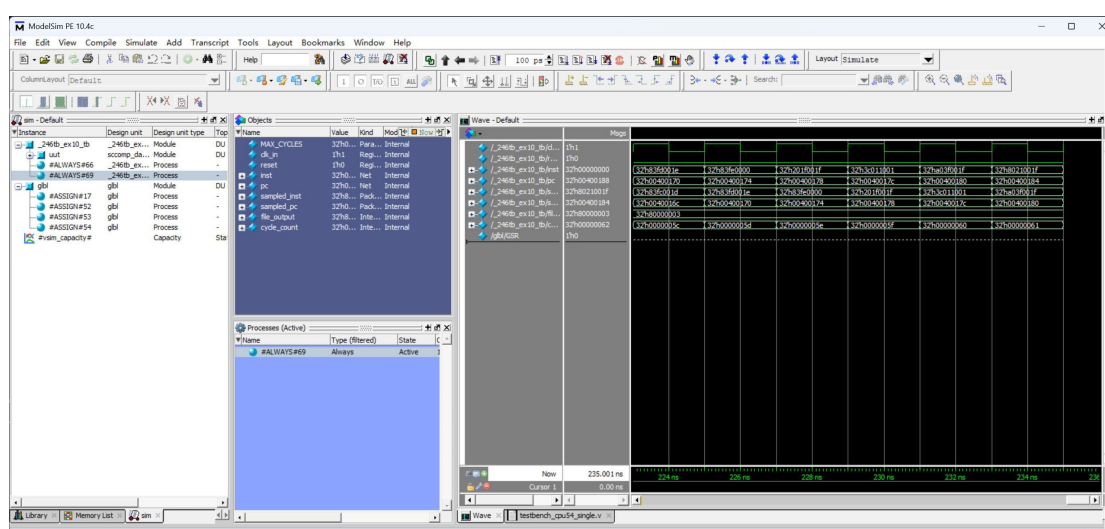


```
执行命令: txt_compare -file1 "D:\CPU54_test\test35_jalr_result.txt"
           -file2 "D:\CPU54_test\output\my_35_jalr_result.txt"
           -display detailed

比较结果输出:
=====
在指定检查条件下完全一致
=====

请按任意键继续...
```

36.39.lbsb

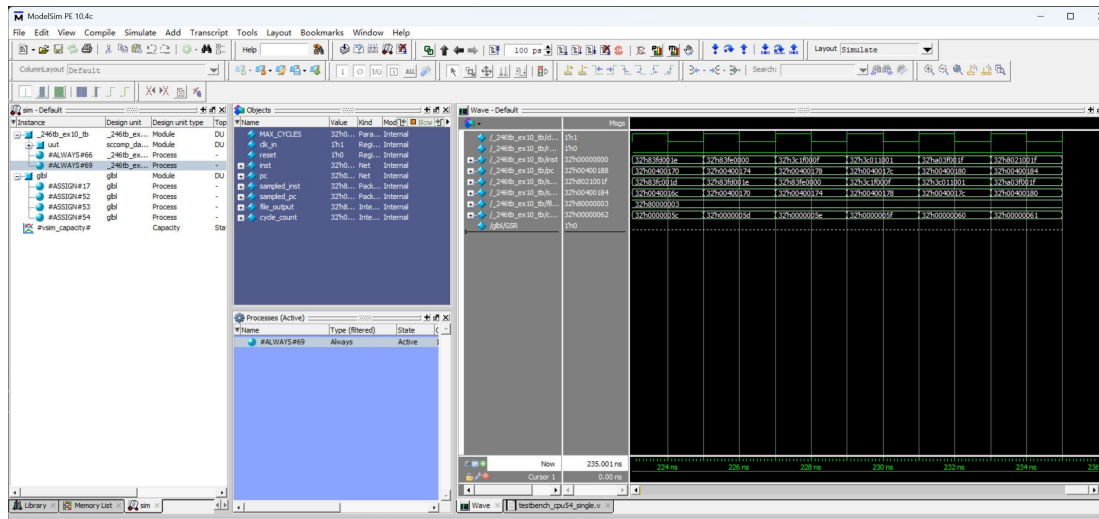


```
执行命令: txt_compare -file1 "D:\CPU54_test\test36.39_lbsb_result.txt"
           -file2 "D:\CPU54_test\output\my_36.39_lbsb_result.txt"
           -display detailed

比较结果输出:
=====
在指定检查条件下完全一致
=====

请按任意键继续...
```

36.39.lbsb2



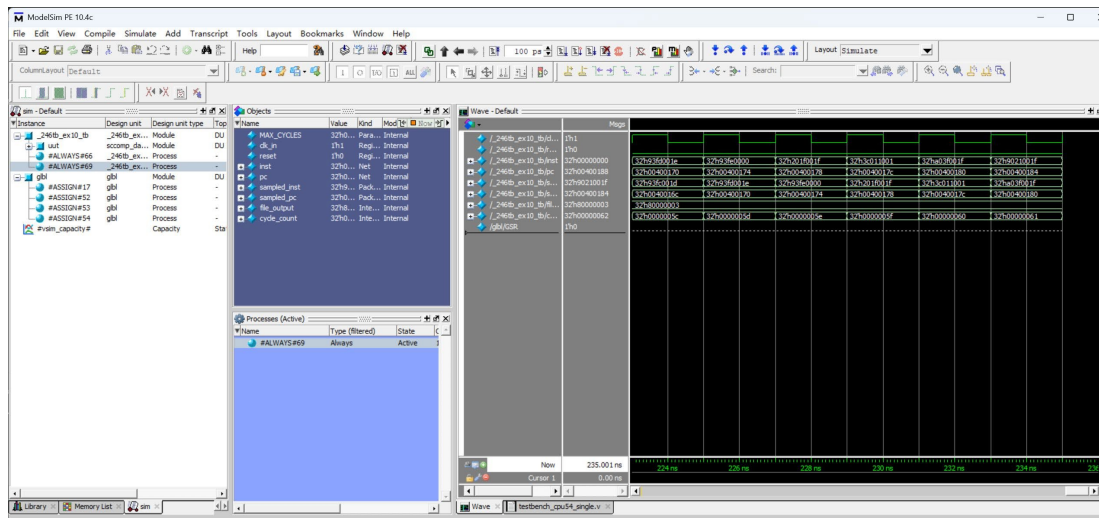
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\36.39_lbsb2.result.txt"
--file2 "D:\CPU54_test\output\my_36.39_lbsb2_result.txt"
--display detailed
```

比较结果输出:

在指定檢查條件下完全一致。

请按任意键继续...

37.lbu



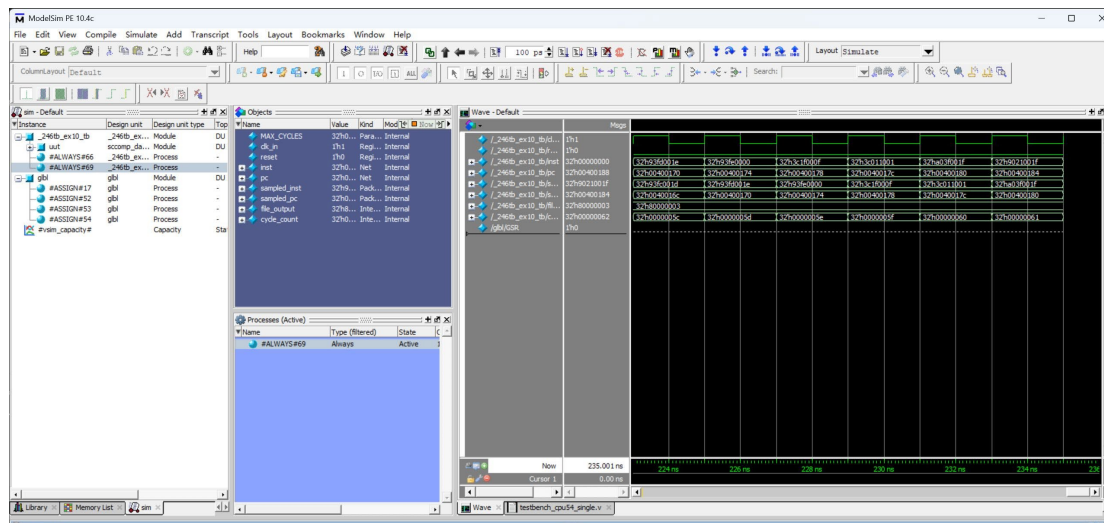
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\37_lbu.result.txt"
--file2 "D:\CPU54_test\output\my_37_lbu_result.txt"
--display detailed
```

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续...

37.lbu2

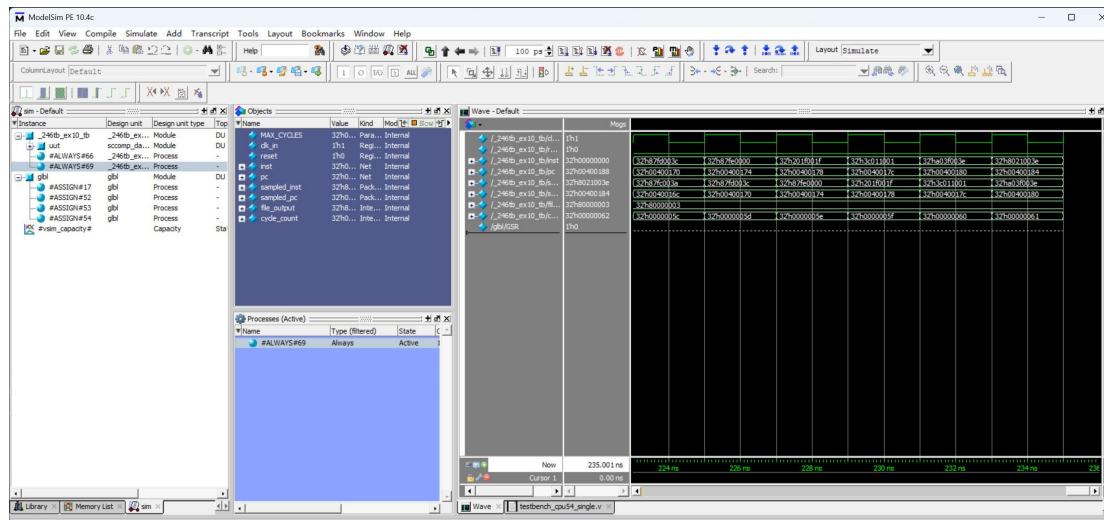


```
执行命令: bxt_compare -file1 "D:\CPU54_test\bxt37_lbu2_result.txt"
           -file2 "D:\CPU54_test\output\my_37_lbu2_result.txt"
           -display detailed

比较结果输出:
=====
在指定检查条件下完全一致
=====

请按任意键继续...
```

38.lhu

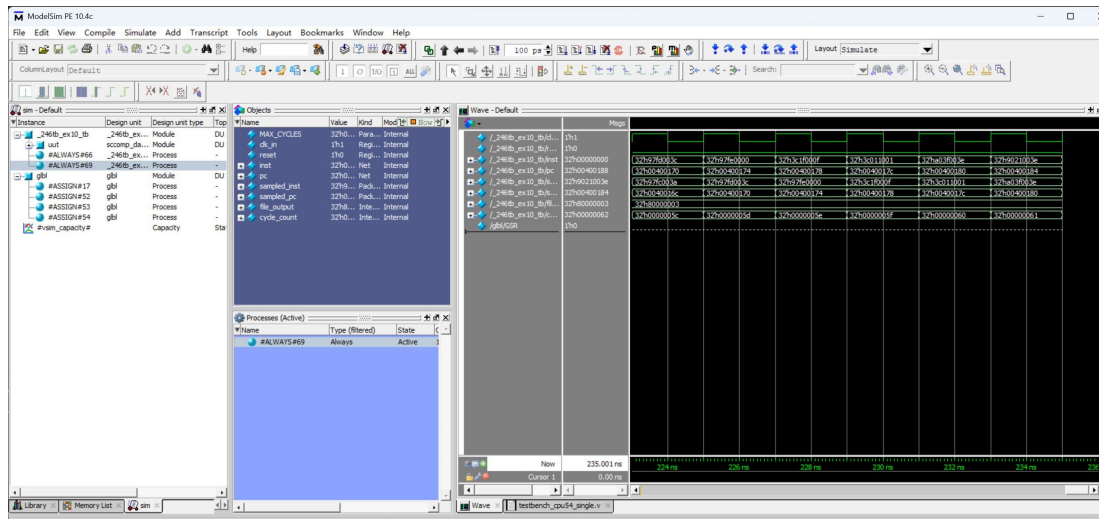


```
执行命令: bxt_compare -file1 "D:\CPU54_test\bxt38_lhu_result.txt"
           -file2 "D:\CPU54_test\output\my_38_lhu_result.txt"
           -display detailed

比较结果输出:
=====
在指定检查条件下完全一致
=====

请按任意键继续...
```

38.lhu2



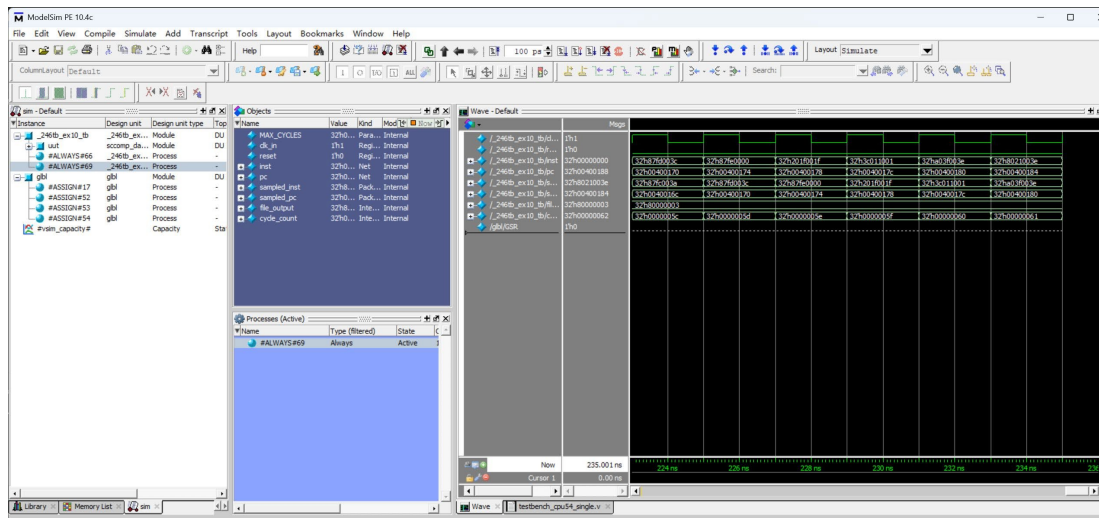
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\38_lhu2.result.txt"
--file2 "D:\CPU54_test\output\my_38_lhu2_result.txt"
--display detailed
```

比较结果输出:

在指定檢查條件下完全一致。

请按任意键继续...

40.41.lhsh



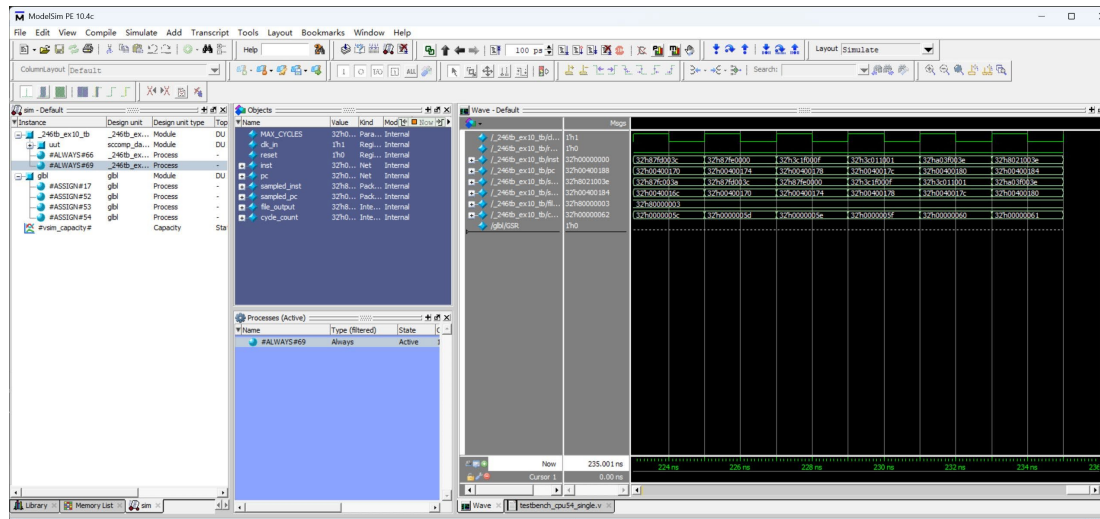
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\40.41_lhsh.result.txt"
--file2 "D:\CPU54_test\output\my_40.41_lhsh_result.txt"
--display detailed
```

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续...

40.41.lhsh2



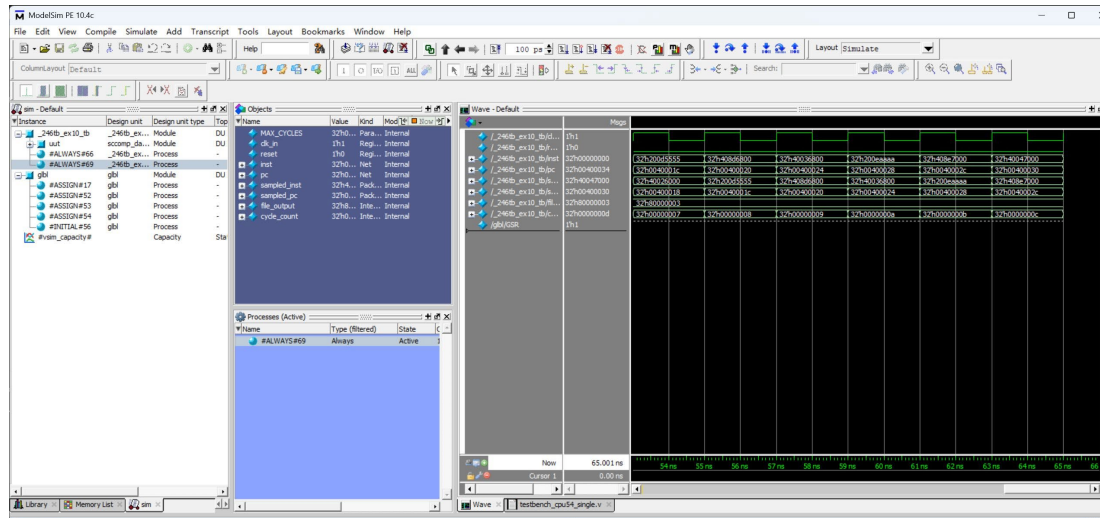
```
执行命令: txt_compare -file1 "D:\CPU54_test\40.41.lhsh2_result.txt"
           -file2 "D:\CPU54_test\output\my_40.41.lhsh2_result.txt"
           -display detailed
```

比较结果输出:

在指定检查条件下完全一致

请按任意键继续...

42.45.mfc0mtc0



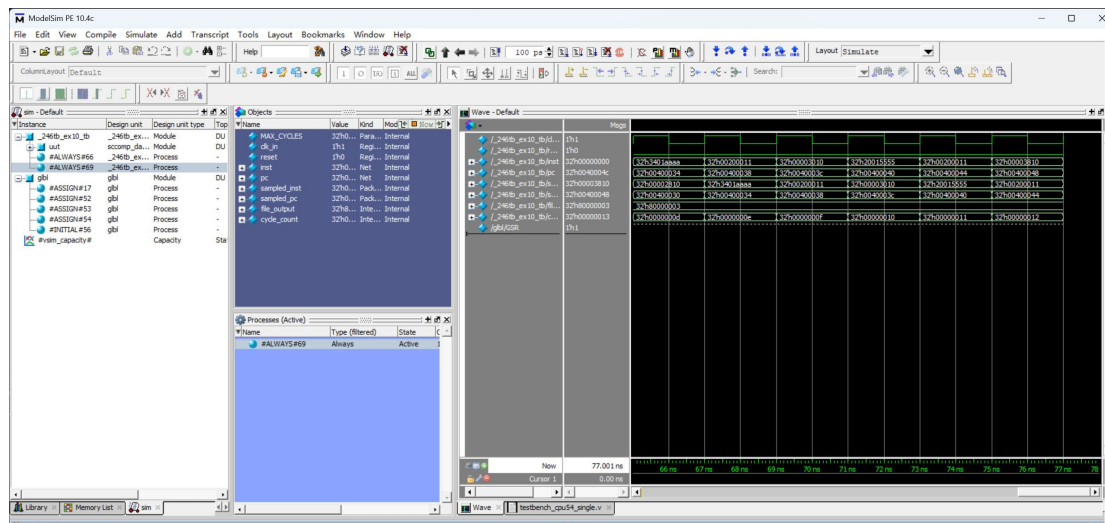
```
执行命令: txt_compare -file1 "D:\CPU54_test\42.45.mfc0mtc0_result.txt"
           -file2 "D:\CPU54_test\output\my_42.45.mfc0mtc0_result.txt"
           -display detailed
```

比较结果输出:

在指定检查条件下完全一致

请按任意键继续...

43.46.mfhi.mthi

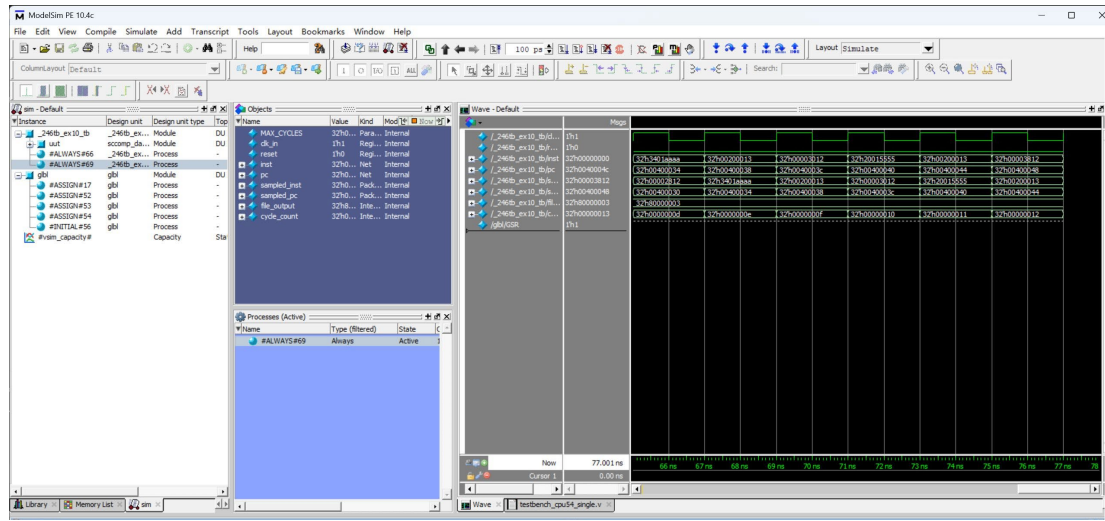


```
执行命令: bt_compare -file1 "D:\CPU54_test\bt\43.46.mfhi.mthi.result.bt"
           -file2 "D:\CPU54_test\output\my_43.46.mfhi.mthi.result.bt"
           --display detailed

比较结果输出:
=====
在指定检查条件下完全一致
=====

请按任意键继续...
```

44.47.mflo.mtlo

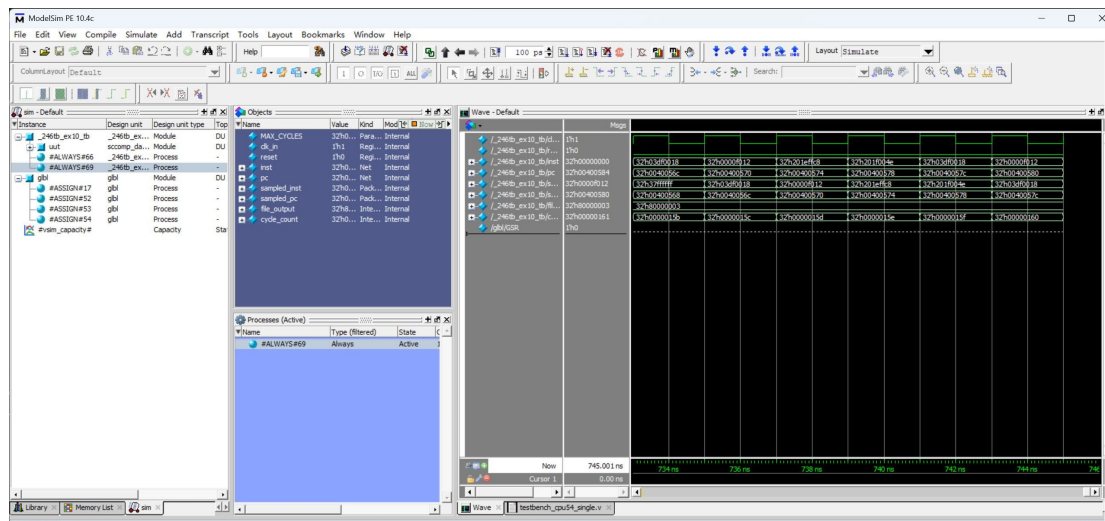


```
执行命令: bt_compare -file1 "D:\CPU54_test\bt\44.47.mflo.mtlo.result.bt"
           -file2 "D:\CPU54_test\output\my_44.47.mflo.mtlo.result.bt"
           --display detailed

比较结果输出:
=====
在指定检查条件下完全一致
=====

请按任意键继续...
```

48.mult



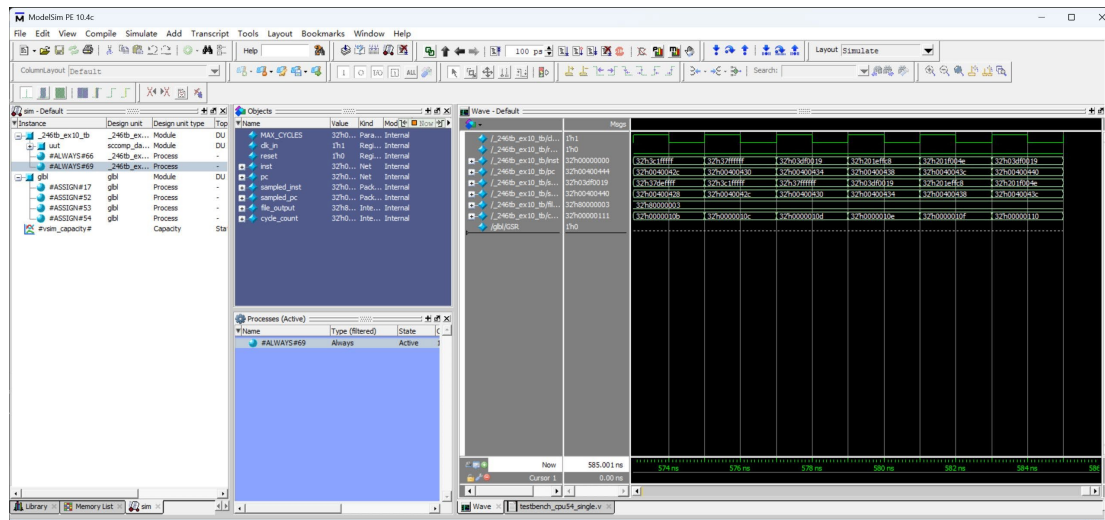
```
执行命令: txt_compare -file1 "D:\CPU54_test\txt\48_mult.result.txt"
           -file2 "D:\CPU54_test\output\my_48_mult.result.txt"
           -display detailed
```

比较结果输出:

在指定检查条件下完全一致

请按任意键继续...

49.multu



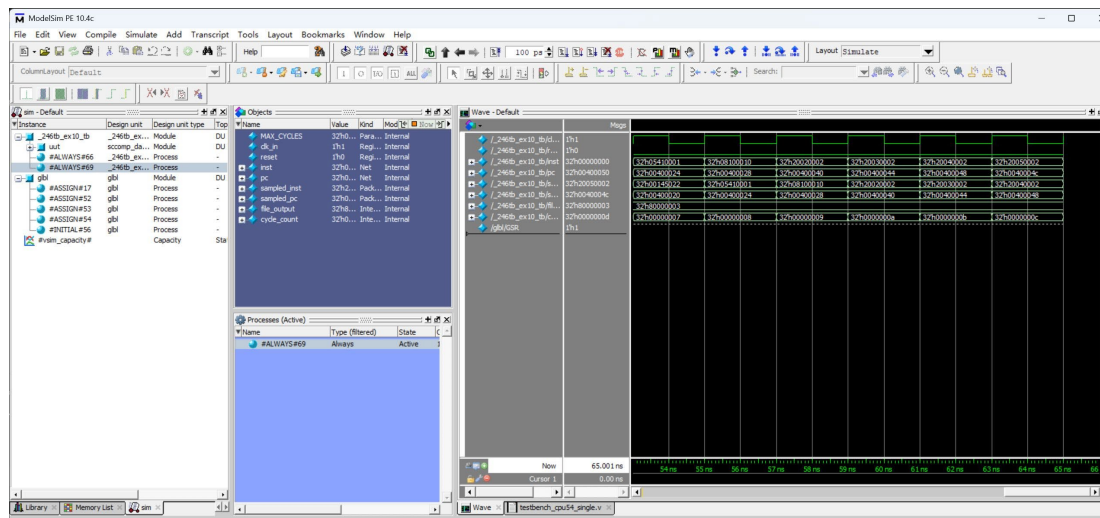
```
执行命令: txt_compare -file1 "D:\CPU54_test\txt\49_multu.result.txt"
           -file2 "D:\CPU54_test\output\my_49_multu.result.txt"
           -display detailed
```

比较结果输出:

在指定检查条件下完全一致

请按任意键继续...

52.bgez



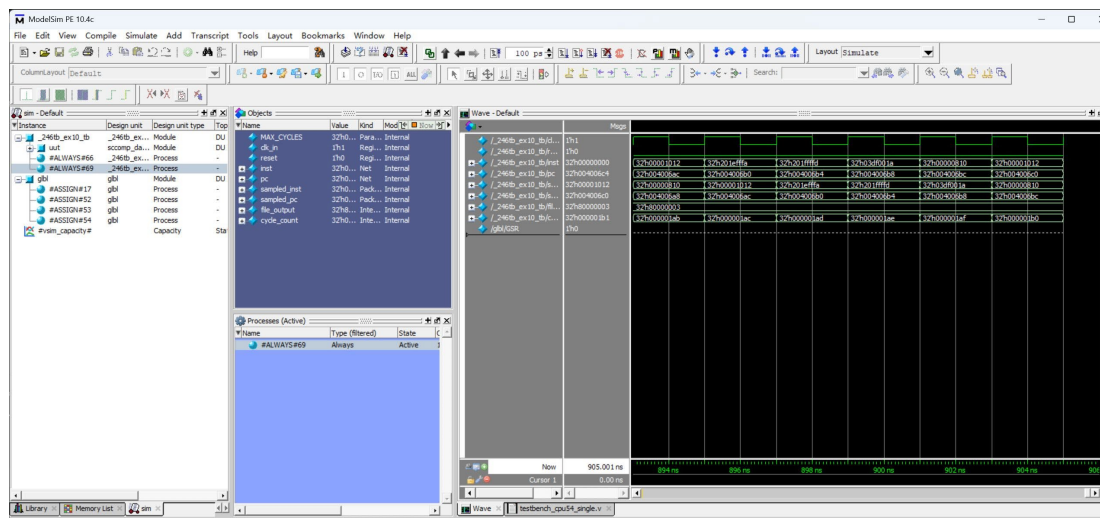
```
执行命令: txt_compare --file1 "D:\CPU54_test\bvt\52_bgez.result.txt"
--file2 "D:\CPU54_test\output\my_52_bgez_result.txt"
--display detailed
```

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续...

54.div



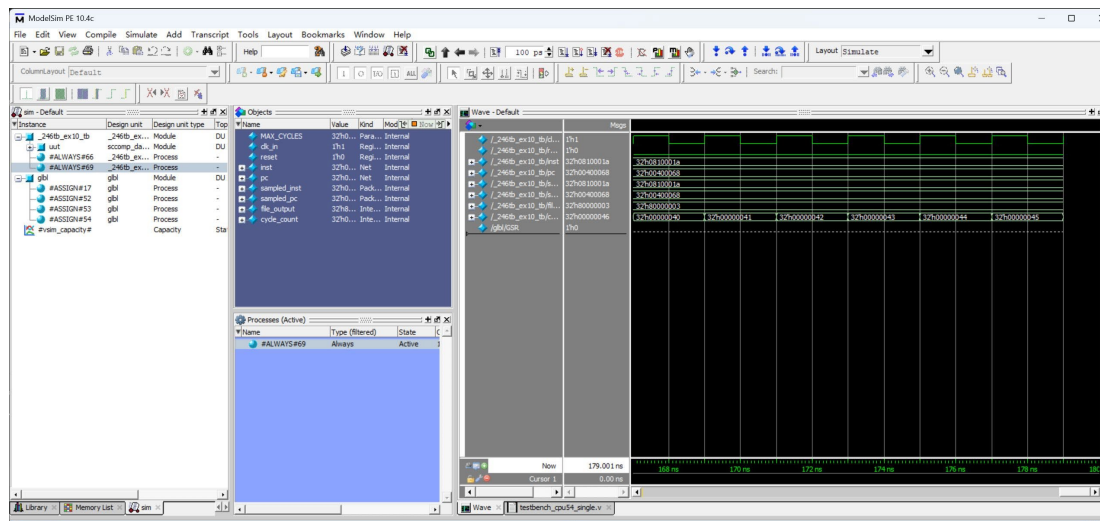
```
执行命令: txt_compare --file1 "D:\CPU54_test\txt\54_div.result.txt"
          --file2 "D:\CPU54_test\output\my_54_div_result.txt"
          --display detailed
```

比较结果输出:

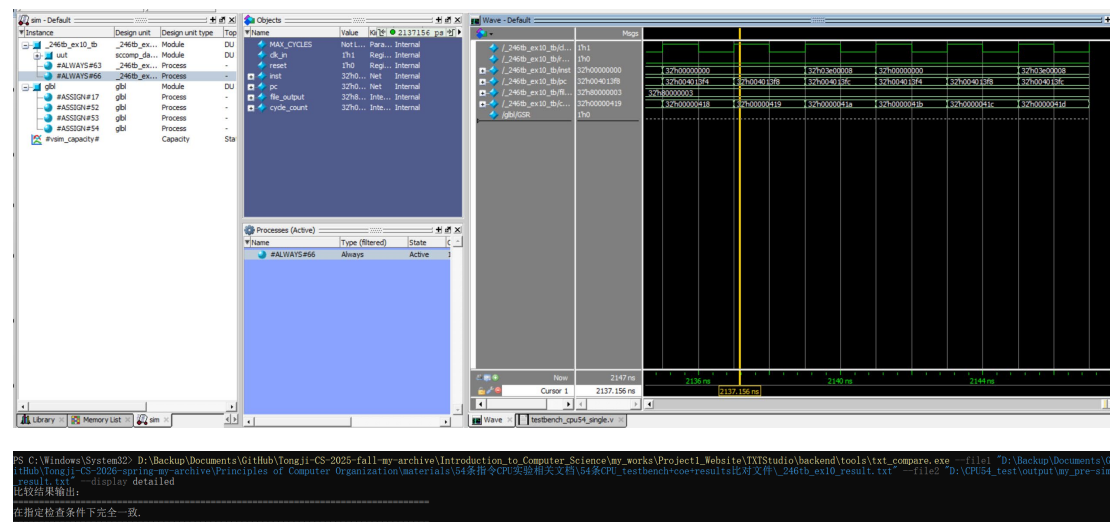
在指定检查条件下完全一致。

请按任意键继续...

CP0test



前仿真 coe



若不终止，最后 pc 会在 004013F8 004013FC 004013F4 之间死循环

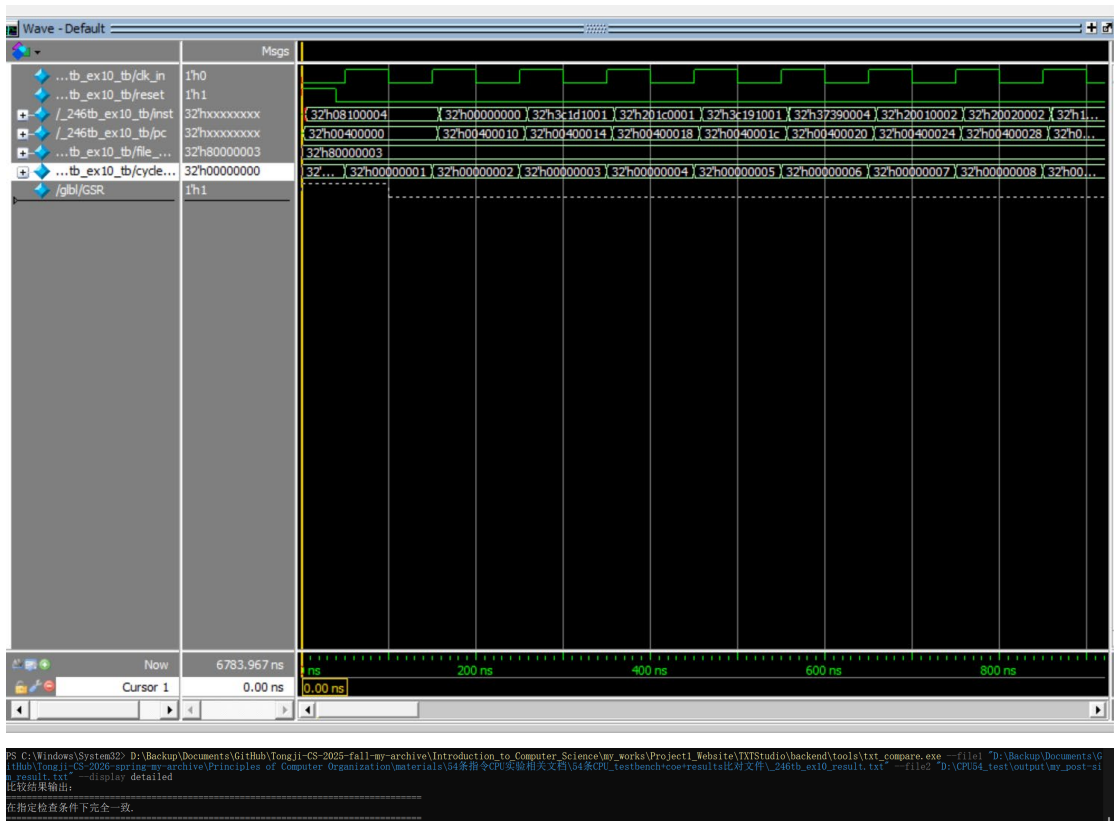
五、CPU 后仿真测试结果

报告要求：

- (1) 对 CPU 后仿真的实验内容和结果进行描述，并给出 modelsim 后仿真波形图的贴图。
- (2) 对时序分析的结果进行分析，并截图。

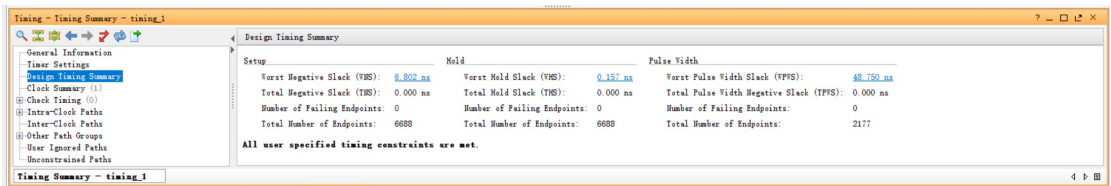
后仿真以综合后的门级网表为对象，使用与前仿真相同的.coe 和 tb，观察时钟、PC 等信号。后仿真中指令执行顺序、PC 变化和运算结果均与前仿真一致，未出现综合引入的功能差异。

(1) 后仿真



Pc 和 inst 如预期在时钟上升沿之后延迟一段时间再更新。最后 pc 会在 004013F8 004013FC 004013F4 之间死循环

(2) 时序分析报告



时序分析采用 100 ns 时钟周期，即 CPU 工作频率为 10 MHz。分析结果显示，最差建立时间裕量 WNS 为 6.802 ns，建立时间总违例 TNS 为 0；最差保持时间裕量 WHS 为 0.157

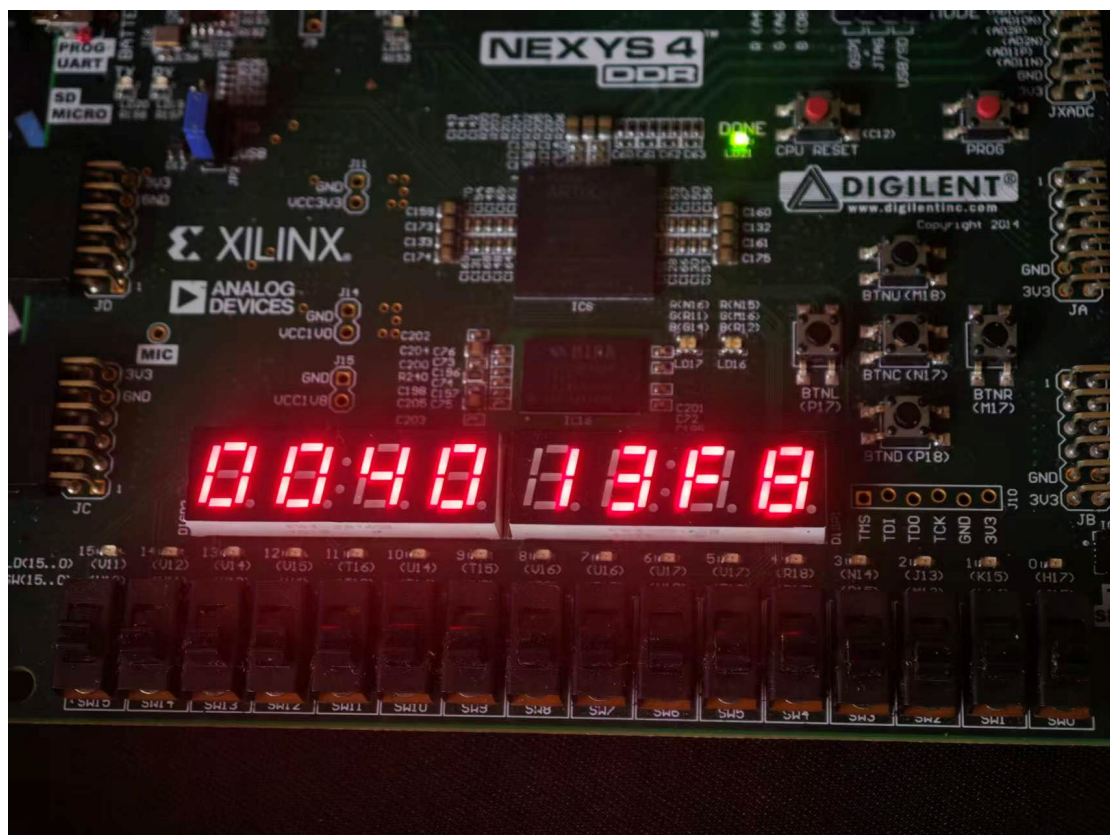
ns, 保持时间总违例 THS 为 0; 最差脉宽裕量 WPWS 为 48.750 ns, 脉宽总违例 TPWS 为 0。所有检查项目的违例端点数均为 0, 且不存在未约束的时钟或内部端点。最差组合路径延迟约为 93.062 ns, 小于 100 ns 时钟周期。因此, 该 CPU 在 10 MHz 时钟约束下满足建立时间、保持时间和脉宽要求, 时序分析通过。

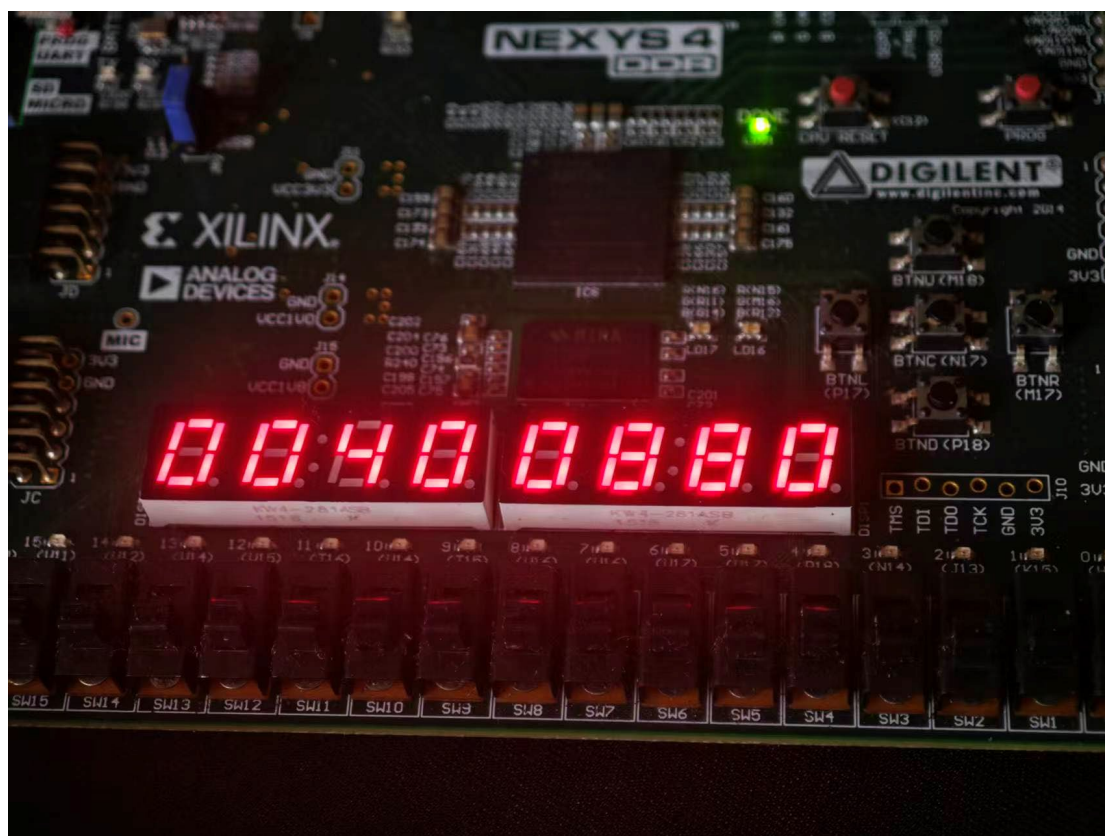
六、CPU 下板结果

报告要求:

- (1) 对 CPU 下板的实验内容和结果进行描述, 并给出下板结果的贴图。
- (2) 实验的分析与结论。

下板实验以 `sccomp_board` 为顶层模块, 将板载 100 MHz 时钟分频后作为 CPU 时钟, 并通过八位数码管显示当前 PC 值。





七段数码管最后显示的 pc 在 004013F8 004013FC 004013F4 之间死循环，与结果文件 _246tb_ex10_result.txt 末尾的 PC 序列完全一致，说明下板结果正确。

七、心得体会及建议

心得体会：

本次 54 条指令单周期 CPU 实验的设计复杂度相比 31 条指令 CPU 有了显著提升。随着指令数量和类型的增加，数据通路、控制信号及其组合逻辑的复杂度也呈现出指数增长的趋势。这让我意识到硬布线控制器的确更适合指令格式规整、控制流程相对简单的 RISC（如 MIPS）体系；当指令系统进一步复杂化时，微程序控制器在设计、维护和扩展方面通常更具优势。

整个 CPU54 实现过程虽然较为繁琐，部分工作也具有较强的重复性，但在绘制数据通路、分析指令执行流程以及填写指令操作时间表的过程中，我对 CPU 内部各部件之间的数据传递、控制信号的生成及每条指令的完整执行过程有了更加深入和系统的理解。此次实验不仅锻炼了我的系统设计与调试能力，也帮助我将计算机组成原理中的理论知识真正落实到了具体的 CPU 设计中。

建议：

（1）（对助教们）下发的指令测试参考 result 文件 pc 是从 0000_0000 开始的，但 Mars 和网站的 pc 又是 0x0040_0000，不一致，需要改改下发的 result 文件